



PROGRAMMING THE TILE-GX PROCESSOR

DOCUMENT RELEASE 1.70
DOC. NO. UG505
JUNE 2014
TILERA CORPORATION

Copyright © 2006-2014 Tilera® Corporation. All rights reserved. Printed in the United States of America.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, except as may be expressly permitted by the applicable copyright statutes or in writing by the Publisher.

The following are registered trademarks of Tilera Corporation: Tilera and the Tilera logo.

The following are trademarks of Tilera Corporation: Embedding Multicore, The Multicore Company, Tile Processor, TILE Architecture, TILE64, TILEPro, TILEPro36, TILEPro64, TILEExpress, TILEExpress-64, TILEExpressPro-64, TILEExpress-20G, iMesh, TileDirect, TILExtreme-Gx, TILExtreme-Gx Duo, TILEmpower, TILEmpower-Gx, TILEmpower-Gx36, TILEmpower-Gx72, TILEncore, TILEncorePro, TILEncore-Gx, TILEncore-Gx9, TILEncore-Gx16, TILEncore-Gx36, TILEncore-Gx72, TILE-Gx, TILE-Gx9, TILE-Gx16, TILE-Gx36, TILE-Gx72, TILE-Gx8072, TILE-Gx3000, TILE-Gx5000, TILE-Gx8000, TILE-Gx8009, TILE-Gx8016, TILE-Gx8036, TILE-Gx3036, DDC (Dynamic Distributed Cache), Multicore Development Environment, Gentle Slope Programming, TMC (Tilera Multicore Components), hardwall, Zero Overhead Linux (ZOL), MiCA (Multicore iMesh Coprocessing Accelerator), and mPIPE (multicore Programmable Intelligent Packet Engine). All other trademarks and/or registered trademarks are the property of their respective owners.

Third-party software: The Tilera IDE makes use of the BeanShell scripting library. Source code for the BeanShell library can be found at the BeanShell website (<http://www.beanshell.org/developer.html>).

This document contains advance information on Tilera products that are in development, sampling or initial production phases. This information and specifications contained herein are subject to change without notice at the discretion of Tilera Corporation.

No license, express or implied by estoppels or otherwise, to any intellectual property is granted by this document. Tilera disclaims any express or implied warranty relating to the sale and/or use of Tilera products, including liability or warranties relating to fitness for a particular purpose, merchantability or infringement of any patent, copyright or other intellectual property right.

Products described in this document are NOT intended for use in medical, life support, or other hazardous uses where malfunction could result in death or bodily injury.

THE INFORMATION CONTAINED IN THIS DOCUMENT IS PROVIDED ON AN "AS IS" BASIS. Tilera assumes no liability for damages arising directly or indirectly from any use of the information contained in this document.

Publishing Information:

Document Number	UG505
Document Release	1.70
Date	26 June 2014

Contact Information:

Tilera Corporation
Information info@tilera.com Web Site http://www.tilera.com

CONTENTS

PREFACE	VII
CHAPTER 1 TILE PROCESSOR ARCHITECTURE	
1.1 Overview	1
1.2 Tiles	2
1.3 Instruction Set Architecture	3
1.4 Memory Subsystem	3
1.5 Inter-Tile Networks	3
1.6 System Software Stack	4
1.6.1 Hypervisor Layer: Hypervisor	4
1.6.2 Supervisor Layer: Tile Linux	5
1.6.3 Applications Layer: User Space	5
1.6.4 Bare Metal Environment (BME)	6
1.7 Tile Classes	6
CHAPTER 2 DESIGNING PARALLEL APPLICATIONS	
2.1 Overview	7
2.2 Decomposition Techniques	7
2.2.1 Functional Decomposition	8
2.2.2 Data Decomposition	8
2.2.2.1 Static Load Distribution	9
2.2.2.2 Dynamic Load Distribution	11
2.3 Choosing a Process Model	12
2.4 Distributing the Work: Spatial Mapping	13
2.5 Communication Mechanisms	13
2.5.1 Choosing Communication Mechanisms	15
2.5.2 Shared Memory	15
2.5.3 Hardware Networks	16
CHAPTER 3 DEVELOPING STANDARD LINUX APPLICATIONS	
3.1 Creating Standard Linux Applications	17
3.2 Creating Parallel Tasks	17

3.2.1 Creating Multiple Threads	18
3.2.2 Creating Multiple Processes	18
3.3 Understanding CPU Affinity	19
3.3.1 Load Balancing Using CPU Affinity	19
3.3.2 Specifying Initial Affinity	20
3.3.2.1 Specifying Initial Affinity By Means of tile-monitor	20
3.3.2.2 Specifying Initial Affinity By Means of the Shepherd	21
3.3.2.3 Specifying Initial Affinity By Means of taskset	21
3.3.3 Setting Affinity Programmatically	22
3.4 Synchronization	22
3.4.1 Pthreads Synchronization	22
3.4.2 TMC Synchronization	23
 CHAPTER 4 MEMORY HOMING AND ALLOCATION CONCEPTS	
4.1 Overview of Memory Homing in the Tiler Environment	25
4.2 Local Homing Strategy	26
4.3 Remote Homing Strategy	27
4.4 Hash-For-Home Strategy	28
4.5 Specified-Attribute Memory Page Allocation	29
4.6 Multiple Heap Allocation (mspaces)	30
4.7 Allocating Shared Memory	30
4.7.1 Standard Linux MAP_SHARED Memory	31
4.7.2 Tiler-Provided Common Memory API, tmc_cmern()	31
 CHAPTER 5 CONTROLLING MEMORY ALLOCATION IN AN APPLICATION	
5.1 Using the tmc_alloc() API to Allocate Memory Pages	33
5.1.1 Functions for Allocating and Freeing Memory	34
5.1.2 Functions for Setting and Clearing Shared Memory	34
5.1.3 Functions for Setting Memory Nodes	34
5.1.4 For More Information About tmc_alloc()	35
5.2 Using the tmc_mspace() API to Allocate Memory Spaces	35
5.2.1 Allocating Separate Memory Spaces	35
5.2.2 Flags Specific to the tmc_mspace() API	36
5.2.3 Setting Allocation Attributes in mspaces	36
5.2.4 Functions for Multiple Heap Allocation	37
5.2.5 Using the tmc_malloc Library to Support malloc() by Means of tmc_mspace	37
5.2.6 For More Information About tmc_mspace()	38

5.3 Using the tmc_cmem() API to Access Common Memory	38
5.3.1 More Information About tmc_cmem()	39
5.4 Memory-Mapping Flags to Control Caching	39
5.4.1 Flags for Setting the Home Cache	39
5.4.2 Flags for Disabling Local Caching	40
5.4.3 Flags for Requesting User-Managed Shared Memory	41
5.4.4 Flags for Sharing Mappings Without the tmc_cmem() API	41
5.5 Using Hash-for-Home	42
5.5.1 Hash-for-Home in Applications	42
5.5.2 Hash-for-Home in Linux	43
5.5.3 Recommendations for Hash-for-Home Settings	43
5.6 Understanding Virtual Memory and Page Size	43
5.7 Configuring Memory Page Sizes under Linux	44
5.7.1 Page Tables and Page Sizes	44
5.7.1.1 Small Pages	44
5.7.1.2 Huge Pages	44
5.7.1.3 Default Huge Page Size	44
5.7.2 Configuring Huge Page Sizes	44
5.7.3 Allocating Huge Pages	46
5.7.4 Allocating Huge Pages on Specific Memory Controllers	47
5.7.5 Determining Current Huge Page Configuration	47
5.7.6 Using Huge Pages Programmatically in Generic Linux	48
5.7.7 Allocating Huge Pages using tmc/alloc.h	48
5.7.7.1 Using TILE-Gx APIs Which Require 64KB Pages	49
5.7.8 Allocating Huge Pages using hugetlbfs	49
5.7.9 Using malloc() to Allocate Huge Pages	49
5.7.10 Basing Program Text, Data, and BSS on Huge Pages	50
5.7.11 Caveats for Using Huge Pages	50
5.8 Allocating Too Much Memory	51
5.9 Configuring TILE-Gx OpenJDK 1.6 Support for UseLargePages	51
5.9.1 Setting the shmmax Parameter	52
5.9.2 Setting the nr_hugepages Size	52
5.10 Unaligned Memory Accesses	52
5.10.1 Vector Library Macros	52
5.10.2 Loading One Unaligned Value	54
5.10.2.1 Accelerating Unaligned Memory Accesses	55
5.10.2.2 Using Packed Struct Fields	56
5.10.3 Loading Multiple Unaligned Values	56
5.10.4 Shifting Over Values Already in Registers	58

5.10.5 Unaligned Stores	60
5.11 Additional Tips on Managing Alignment of Data	60
5.11.1 Using the memalign() Function	60
5.11.2 Using Linker Scripts	60
5.11.3 Using Page Table Special Files to Confirm Allocation	60
5.12 Examples of Dynamic Memory Allocation	61
 CHAPTER 6 DYNAMIC DISTRIBUTED CACHE TECHNOLOGY	
6.1 Overview of DDC	63
6.2 Default Caching Policy	66
6.2.1 System-Wide Cache Defaults	67
6.2.2 Per-Application Cache Defaults	67
6.2.3 Specifying Tiles for Hash-for-Home	67
6.3 Page Attributes in DDC	67
6.3.1 Page Attributes for Allocation Control	68
6.3.2 Page Attribute for Distribution of Cache Blocks	68
 CHAPTER 7 ZERO OVERHEAD LINUX APPLICATIONS	
7.1 Understanding Zero Overhead Linux	69
7.2 Using the Linux Scheduler on Dataplane Tiles	70
7.2.1 Run Queues	70
7.2.2 Scheduler Interrupts	71
7.2.3 Task Migration	71
7.2.4 Dataplane Tiles	71
7.3 Running a Task on a Dataplane Tile	72
7.4 Interrupting a Dataplane Core	73
7.5 Programming Models for Dataplane Mode	73
7.6 Using the set_dataplane() API	74
7.7 Spin-Based Shared Memory Synchronization	75
7.8 Avoiding System Calls	75
7.9 User Dynamic Network (UDN)	75
7.9.1 Basic UDN Usage	75
7.9.2 Avoiding UDN Deadlock	76
7.10 Troubleshooting ZOL Applications	77
7.11 Dataplane Example	78
 INDEX	 79

PREFACE

About This Guide

This guide describes the basic concepts involved in developing applications for the Tile Processor™ and also provides specific implementation details.

This guide applies to the Multicore Development Environment™ (MDE) Release 4.2.

Intended Audience

This guide is intended for use by Tile Processor application developers.

Note: This guide assumes that you have already read *Getting Started with the Multicore Development Environment* (UG504) and the *Multicore Development Environment User Guide* (UG501).

Document Organization

This guide is organized as follows:

- [Chapter 1: Tile Processor Architecture](#)
Provides an overview of the Tile Processor architecture and system software.
- [Chapter 2: Designing Parallel Applications](#)
Describes the steps involved in decomposition, mapping, and communication in designing parallel applications.
- [Chapter 3: Developing Standard Linux Applications](#)
Describes concepts and tasks involved in developing applications using Tiler's standards-based Linux environment.
- [Chapter 4: Memory Homing and Allocation Concepts](#)
Describes memory homing strategies and introduces Tiler's APIs for memory allocation.
- [Chapter 5: Controlling Memory Allocation in an Application](#)
Describes methods of allocating memory that apply to the Tiler environment.
- [Chapter 6: Dynamic Distributed Cache Technology](#)
Describes DDC™ (Dynamic Distributed Cache), the caching subsystem for the Tile Processor.
- [Chapter 7: Zero Overhead Linux Applications](#)
Describes how to implement real-time applications using Zero Overhead Linux™ (ZOL).

The guide also provides an [Index](#).

MDE Documentation

Note: `TILERA_ROOT` is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is required by many Tiler tools and examples. We recommend that you use the `tile-env` script described in *Getting Started with the Multicore Development Environment* (UG504) to set it correctly.

Tilera MDE Document Index

The Tilera MDE Document Index provides links to online copies of the MDE documentation, in both HTML and PDF formats:

```
$TILERA_ROOT/doc/index.html
```

Location of PDF Files in the Kit

The PDF versions of MDE documents can be found at this location:

```
$TILERA_ROOT/doc/pdf/
```

Man Pages and Info Pages

To view a man page for an x86 tool, run:

```
$ tile-man toolname
```

To view a man page for a Tile tool, run this on a TILE system:

```
$ man toolname
```

To view an info page for a Tile tool, run this on a TILE system:

```
$ info toolname
```

IDE Online Help

The IDE has its own [Tilera IDE Online Documentation](#), which you can access when you use the IDE.

To get to the online help:

- Start `tile-eclipse`.

Select the menu command [Help → Help Contents](#).

- When the online help browser is displayed, select the document [Tilera IDE Online Documentation](#) in the left navigation pane, if it is not already selected.

Technical or Customer Support

You can reach Tilera customer support in either of the following ways:

- Visit the Tilera Web site at <http://www.tilera.com>.
- E-mail questions to support@tilera.com.

Notation Conventions

The following table lists notation conventions used in this guide.

Example	Description
{ }	Alternative required items in syntax descriptions appear within curly brackets. If the contents are separated by a vertical bar, you must choose one of the items.
[]	Optional items in syntax descriptions appear within brackets. If the contents are separated by a vertical bar, you must choose one of the items.
...	An ellipse in syntax specifies that the preceding element can be repeated an arbitrary number of times.
\$	This is the system prompt for host-side commands.
#	This is the system prompt for all Tile-side commands and for those x86 host-side commands that must be run as <code>root</code> .
command	A command name appears in the Courier New font
computer output	Computer output appears in the Courier New font.
filename	Non-keyword placeholders appear in italics.
GUI item	A graphical user interface (GUI) item such as a button or menu name appears in red text with the Arial font.
new term	An important word or phrase appears in italics.
Tile Processor™	A processor in the TILE-Gx processor™ family.
TILERA_ROOT	TILERA_ROOT is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is expected by many Tiler tools and examples. We recommend that you use the <code>tile-env</code> script described in <i>Getting Started with the Multicore Development Environment</i> (UG504) to set it correctly.

CHAPTER 1 TILE PROCESSOR ARCHITECTURE

This chapter provides an overview of the Tile Processor architecture and system software:

- [Section 1.1 Overview](#)
- [Section 1.2 Tiles](#)
- [Section 1.3 Instruction Set Architecture](#)
- [Section 1.4 Memory Subsystem](#)
- [Section 1.5 Inter-Tile Networks](#)
- [Section 1.6 System Software Stack](#)
- [Section 1.7 Tile Classes](#)

For more information about the Tile Processor™ architecture, see the *Tile Processor Architecture Overview for the TILE-Gx Series* (UG130) and the *Tile Processor and I/O Device Guide for the TILE-Gx Family of Processors* (UG404).

1.1 Overview

The Tile Processor hardware comprises an array of *tiles*. Each tile has its own processor engine, local cache subsystem, and direct connections to the four nearest-neighbor tiles to form the iMesh™ multicore on-chip interconnect.

For the Tile Processor, each on-tile processor is a 64-bit processor engine that implements Tiler's proprietary, 3-way VLIW instruction set. Tiles on the edges of the chip have iMesh connections to I/O *shims*, which connect to hardware I/O controllers. Each I/O shim has one or more connections to the iMesh interconnect.

[Figure 1-1](#) shows the hardware architecture for two models of the TILE-Gx36™ processor.

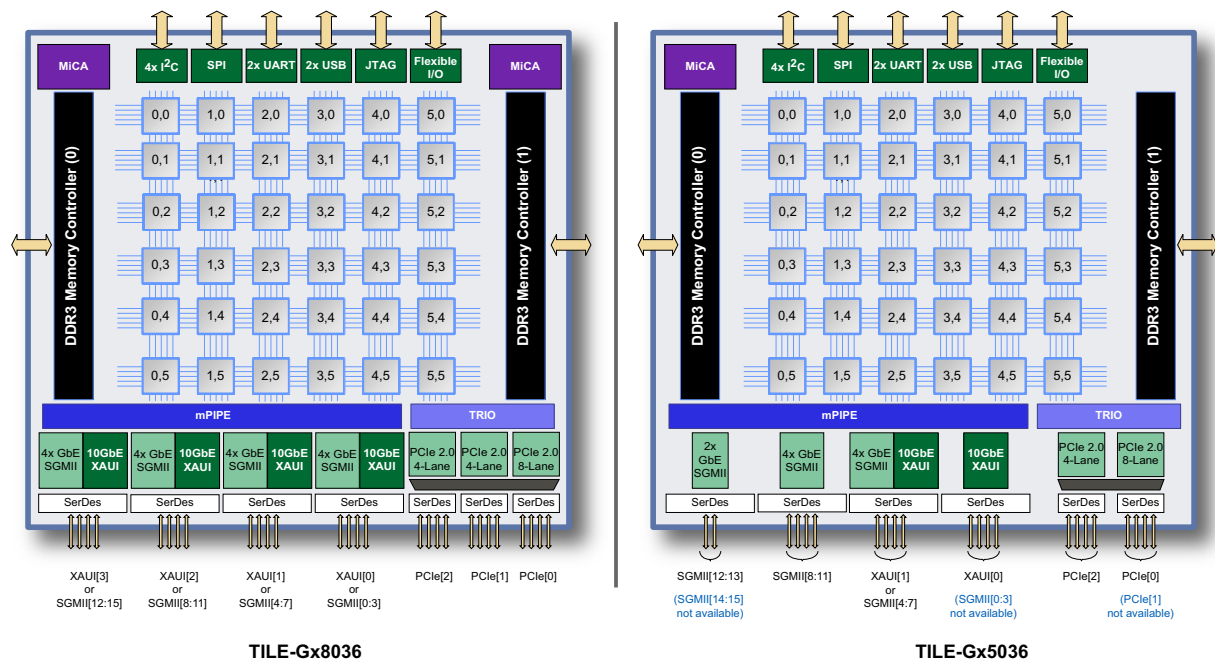


Figure 1-1: TILE-Gx36 Processor Hardware Architecture Block Diagram

1.2 Tiles

At the chip level, the Tile Processor contains a two-dimensional array of identical tiles or cores. Each tile is a powerful, full-featured computing system that can independently run an entire operating system, such as SMP Linux. Each tile implements a 64-bit integer processor engine utilizing a three-way Very Long Instruction Word (VLIW) architecture with its own program counter (PC), cache, and DMA subsystem. An individual tile is capable of executing up to three operations per cycle.

Individual tiles are identified by means of (column,row) notation (for example, tile 0,0 to tile 5,5).

Figure 1-2 shows a single tile for the TILE-Gx36 processor.

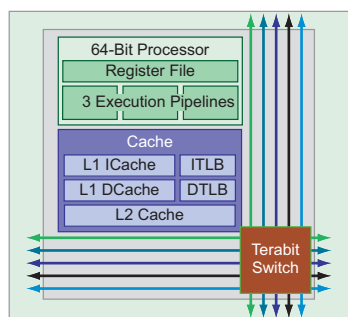


Figure 1-2: Single Tile on a TILE-Gx36 Processor

Each tile in the two-dimensional array connects to other tiles using multiple mesh networks implemented by the network routers in each tile. Because the tiles are identical and connected by short point-to-point connections in the iMesh, the tile array is both scalable and power-efficient.

At the edge of the tile array, the mesh connects into I/O shims. The I/O shims convert data from the on-chip network emanating out of the tile array perimeter to specific data formats. These data formats are required by controllers such as memory controllers, XAUI controllers, GbE controllers, or PCIe controllers, for example.

1.3 Instruction Set Architecture

Each tile contains a conventional three-way VLIW processor with three instructions per bundle and full memory management, protection, and OS support. Compile-time scheduling of VLIW operations results in lower power consumption than dynamically scheduled superscalar processors require.

The Tile Processor includes special instructions to support commonly used embedded operations in DSP, video and network packet processing. These instructions aid in performing such operations as:

- Sum of absolute differences (SAD)
- Hashing and checksums
- Encryption
- SIMD instructions for sub-word parallelism
- Saturating arithmetic
- Unaligned access acceleration

1.4 Memory Subsystem

The Tile Processor contains per-tile caches and supports cache-coherent shared memory. Each tile contains a level 1 data cache (L1D), a level 1 instruction cache (L1I), and a shared level 2 cache (L2). The tile's execution pipe accesses those caches by means of load and store instructions. Each load or store instruction checks the L1D and L2 caches for data to be read or updated. If the requested cache line cannot be found in the tile's local caches, the memory subsystem forwards the operation to either a remote tile (for cache-coherent operations) or to one of the two on-chip DDR3 memory controllers.

The Tile Processor supports a fully-protected memory model. Every load or store instruction is translated by a translation lookaside buffer (TLB) that converts application virtual addresses (the addresses used in a program) into the physical addresses (the actual addresses of DDR memory locations) used by the cache subsystem. The system software stack controls access to the TLB so that applications are only provided TLB entries for memory that they are allowed to access. The TLB's translation for each virtual address can also include various attributes used to tune memory subsystem performance.

For more information, see these chapters:

- [Chapter 4: Memory Homing and Allocation Concepts](#)
- [Chapter 5: Controlling Memory Allocation in an Application](#)
- [Chapter 6: Dynamic Distributed Cache Technology](#)

1.5 Inter-Tile Networks

The Tile Processor provides two program-accessible, inter-tile networks:

- User Dynamic Network (UDN)
- I/O Dynamic Network (IDN)

These networks transmit packets across a mesh network, routing along the X axis first and then the Y axis until reaching the destination core. At the destination, the packet data words are stored in a “demux FIFO” with a capacity of 118 words. Each network packet contains 1 to 128 data words. Cores inject packets into the network by writing words to special registers. When data arrives in the destination’s demux buffer, it is routed to one of four *demux queues*. The receiving core then reads the incoming data by reading one of four registers, each mapped to a different demux queue.

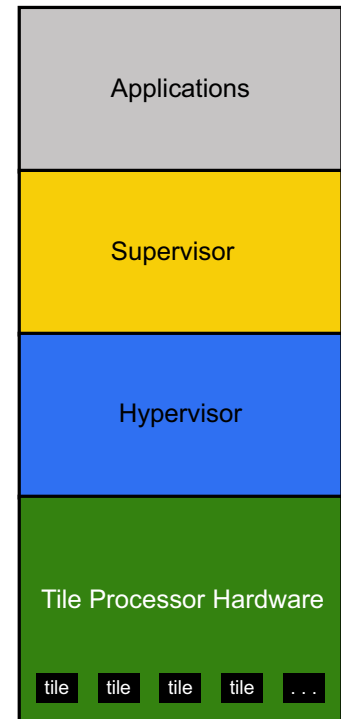
Applications generally perform bulk data transfers by means of shared memory and use the UDN for synchronization and control operations that require high throughput and low latency. For example, networking applications often transmit packet pointers between tiles by means of the UDN while storing the packet data in shared memory.

1.6 System Software Stack

The Tiler system software and components provide an operating system structure and user-level run-time software to enable applications to run on the Tile Processor. Tiler has developed architecture-dependent code to support the Tile Processor architecture.

The Tiler run-time software stack includes the following layers:

- The *hypervisor layer* abstracts some hardware details of the Tiler chip on which it is running from the supervisor, managing communication between tiles, from tiles to I/O controllers, and providing a low-level virtual-memory system.
- The *supervisor layer* provides system calls and I/O devices for user-space applications and libraries. This layer is typically the Tile Linux operating system (OS). This layer enables multi-process applications and multi-threaded processes to exploit multiple tiles for increased performance. The OS software manages hardware resources and provides higher-level services, such as processes and virtual memory allocation.
- The *applications layer* runs standard libraries, such as the C run-time library and Tiler Multicore Components (TMC) library, in user space.



1.6.1 Hypervisor Layer: Hypervisor

The Tiler hypervisor is the lowest layer of the system stack. The hypervisor infrastructure includes the booter, which sets up the initial software environment on all tiles in the system, and a variety of service routines that are exported to client operating systems. The hypervisor is the only layer that can access the TLBs and IDN.

A separate hypervisor instance runs on each tile. The hypervisor software provides the following services:

- Booting and hardware discovery. The hypervisor boots the Tile Processor from ROM, USB or PCIe (PCI Express), probes available memory, and discovers which I/O devices are connected in order to determine the machine configuration.
- Providing a virtual hardware interface to the supervisor. This simplifies OS porting and can also enable multiple OS instances.

- Managing low-level I/O drivers. While many device control functions are performed by the supervisor or even by application programs, the hypervisor performs initial configuration, inter-supervisor resource management, and some types of error handling.
- Managing low-level virtual memory. From the supervisor's perspective, the hypervisor provides an abstraction of TLB fill hardware, handling TLB miss faults and providing an in-memory virtual page table for the supervisor.
- Enabling communication between supervisor tiles. The hypervisor provides facilities that allow supervisor code on one tile to send messages to supervisor code on other tiles. For example, Linux uses this facility to implement interprocessor interrupts.

The hypervisor is designed to support multiple supervisors, including multiple side-by-side instances. Some tiles can optionally be dedicated to an application running under the Bare Metal Environment. For more information, see [Section 1.6.4 “Bare Metal Environment \(BME\)” on page 6](#).

An application developer can configure the hypervisor by means of a “hypervisor configuration file” (`.hvc`). This configuration file controls which I/O drivers run, the tiles on which the driver runs, and what configuration parameters are passed to driver at startup. It also specifies the boot parameters to be passed to Linux and the set of tiles on which Linux will run. For more information about how to write a `.hvc` file, see the *Multicore Development Environment System Programmer's Guide* (UG509).

1.6.2 Supervisor Layer: Tile Linux

Tilera has implemented a version of Symmetric Multiprocessing (SMP) Linux, also known as *Tile Linux*.

Tile Linux is based on the standard open-source Linux version 3.10. This architecture-dependent code supports the Tile Processor and provides many features for OS support, including virtual memory management, *translation lookaside buffers* (TLBs), and hardware protection of system resources.

Tile Linux has several important extensions that enable development of high-performance applications. These include:

- Zero Overhead Linux (ZOL) — Allows applications to run on designated *dataplane tiles* that entirely avoid Linux interrupt overhead, so long as the application runs one task per tile and avoids making system calls. For more information, see [Chapter 7: Zero Overhead Linux Applications](#).
- Cache homing `mmap()` flags — Allows an application to specify the tile(s) on which particular memory allocations are homed.
- Protected UDN access — Allows applications to send and receive UDN packets; Linux will clean up any left over packets when the application exits.

For more information on configuring Linux, specifying Linux boot arguments, or building a custom Linux image, see the *Multicore Development Environment System Programmer's Guide* (UG509).

1.6.3 Applications Layer: User Space

Most application code is written to run in user space, taking advantage of the standard Linux environment. User space programs can invoke Linux system calls and link against standard libraries just as on any other Linux platform. Tilera also provides a variety of libraries and tools to support user space application development. While it is possible to write custom code (usually drivers) for the hypervisor and Linux layers, user space provides the most complete set of libraries and debugging tools and is generally used for the bulk of application development.

1.6.4 Bare Metal Environment (BME)

Tilera provides a Bare Metal Environment (BME), which is a run-time environment that allows advanced users to run applications that require direct access to the hardware. The user application runs in place of the hypervisor, and essentially acts as the operating system for the tiles on which it runs. You the user are fully responsible for all services provided on the BME tiles.

You specify the virtual and physical memory layout for the application and the set of tiles on which it runs. (There is one address space per tile.) Once started, the BME application has full control of the tiles on which it runs. This allows you to leverage all of the Tilera boot and startup infrastructure, and lets your application coexist on a processor with Linux or hypervisor dedicated driver tiles.

Developing a bare metal application requires extensive knowledge of the Tile Processor architecture and generally requires much more development time than developing with the Linux stack. Thus, it is not recommended for most applications.

For more information, see the BME chapter in the *Multicore Development Environment System Programmer's Guide* (UG509).

BME support for the TILE-Gx processor is still under development.

1.7 Tile Classes

Tiles fall into four classes. Each tile's class is specified by the hypervisor configuration (`.hvc`) file and cannot be changed after booting.

- **Standard** — This type of tile uses the normal Linux scheduler and can run any task. Linux automatically load balances tasks across all standard tiles. Hypervisor configurations should always provide at least one standard tile to run miscellaneous Linux daemons and development tools.
- **Dataplane** — This type of tile minimizes operating system overhead for the tasks they run, but as a consequence they do not run the full Linux scheduling algorithm. See [Chapter 7: Zero Overhead Linux Applications](#).
- **Dedicated** — Some tiles are permanently devoted to high-performance hypervisor drivers and are not available for Linux at all. Thus, it is not possible to run tasks on them. From Linux's perspective, these tiles are offline and represent holes in the CPU numbering space.
- **Bare Metal Environment (BME)** — When the hypervisor finishes booting the chip, these tiles begin executing a custom, user-created run-time environment with full access to all hardware resources. From Linux's perspective, these tiles are offline and represent holes in the CPU numbering space.

CHAPTER 2 DESIGNING PARALLEL APPLICATIONS

This chapter discusses the steps involved in decomposition, mapping, and communication in designing parallel applications:

- [Section 2.1 Overview](#)
- [Section 2.2 Decomposition Techniques](#)
- [Section 2.3 Choosing a Process Model](#)
- [Section 2.4 Distributing the Work: Spatial Mapping](#)
- [Section 2.5 Communication Mechanisms](#)

2.1 Overview

Once an application is running in the Tile environment, the standard `pthread_create()` or `fork()` APIs can be used to “go parallel”, creating multiple tasks to take advantage of the Tile Processor’s many tiles. In many cases, this initial parallelization of the application is an incremental step that simply runs the same program on each tile, feeding in different data to each instance. In other cases, the program is broken into smaller pieces that can be run in parallel on multiple tiles.

2.2 Decomposition Techniques

Converting a serial, single-core application into a parallel, multi-core application requires a plan for dividing the workload among many worker tasks (processes or threads).

The first step in this *parallelization* process is to choose a decomposition technique, a method for dividing an application into smaller subtasks that can be executed in parallel. There are often many possible decompositions for an application, but the best decompositions split work evenly among the available workers and minimize the time that a worker needs to wait for some other worker’s results.

Once a decomposition method is selected, the programmer implements that decomposition by creating multiple tasks, mapping those tasks to tiles, and using shared memory or hardware networks to communicate between those tasks.

Some typical decomposition techniques are:

- **Functional decomposition** — Different parts of an application that accomplish different functions are divided to run in parallel.
- **Data decomposition** — The data processed by an application is split up among multiple worker tasks, each of which executes the same algorithm on its own subset of the data.
- **Work queues** — If data decomposition results in many more work items than there are tiles, work queues can be used to spread work items among the worker tiles while minimizing wasted cycles due to load imbalance.

Many applications use a combination of these techniques. For example, a networking application might use functional decomposition to implement a sequence of tiles that perform fast-path processing on every packet, and then use a work queue model to distribute the more expensive processing that remains.

2.2.1 Functional Decomposition

Functional decomposition divides an application into functions that can be performed sequentially on each data item. Once the function A has finished working on object 1, function B can work on object 1 and function A can start on object 2, and so forth. One sub-category of functional decomposition is streaming communication, in which one component communicates with the next by means of a long stream of objects. Objects can be quite small (for example individual data words) or larger (such as network packets or macroblocks).

Figure 2-1 depicts Pipeline decomposition, a simple one-dimensional chain in which each task performs a portion of the overall work and then forwards partial results to the next task. This is the simplest form of functional decomposition.

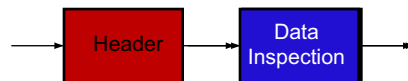


Figure 2-1: Pipeline Decomposition

In reference to Figure 2-1, consider a simple networking application. Suppose that the processing of each network packet involves two steps: packet header processing followed by packet data inspection. In pipelined decomposition, designers can use two tiles for this application. All packets from an I/O device start their processing at the first tile. The first tile performs the header processing function on each packet and then passes the packet to the second tile. The second tile completes the processing by performing the data inspection function. Of course, while the second tile is performing data inspection on the first packet, the first tile can begin header processing for the next packet. This pipeline continues as packets are streamed from the first tile to the second.

2.2.2 Data Decomposition

Data decomposition divides application data into subsets that can be processed independently. This method is associated with a data parallelism style, which executes the same algorithm in parallel over many individual subsets of a whole dataset, as shown in Figure 2-2. In a pure data-parallel style, there is no communication during processing among the individual subsets. Data parallelism ranges from very fine-grain models, in which individual data words or loop iterations are executed in parallel, to coarse-grain models in which the individual data items are larger objects such as whole network packets or digital images.

In a typical data parallel application structure, a load distributor takes incoming work items, such as network packets, and distributes them across a set of identical worker tasks. Each worker task completes the work item and forwards the results to be aggregated by a gatherer task, as shown in Figure 2-2. In the networking space, this programming approach has been termed run-to-completion because a single task running on one tile performs all the processing steps required for a given packet. Tilera's mPIPE library provides a ready-to-use, high performance implementation of a packet load distributor for use in run-to-completion applications.

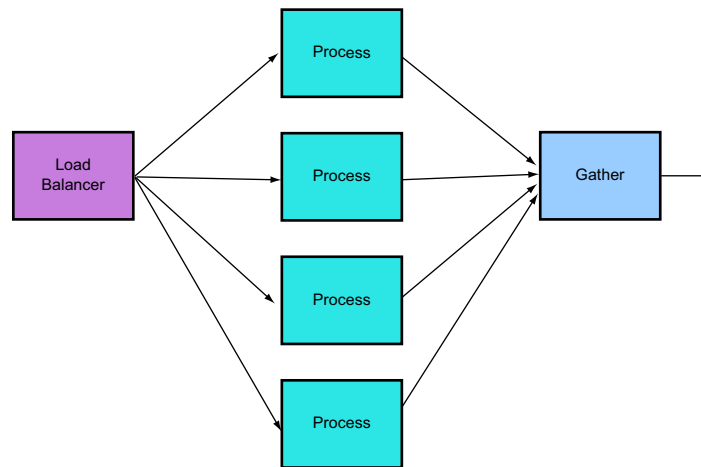


Figure 2-2: Processing Subsets of Data in a Run-to-Completion Model

The choice of load distribution algorithm can have a significant impact on the performance of data parallel applications. Bad load distribution can result in most or all of the work being directed to a single worker, effectively limiting performance to that achieved by a single tile. The following sections describe a variety of load distribution schemes that can be used in parallel applications.

2.2.2.1 Static Load Distribution

Static load distribution algorithms spread work items among worker tasks without any consideration of how busy each worker is. These algorithms tend to be easy to implement, but can suffer performance problems when work items can take a variable amount of time to complete. Examples of static load balancing algorithms include:

- **Static Round Robin** — As shown in [Figure 2-3](#), this load distribution algorithm goes around the workers in a cycle, assigning each new work item to the next worker. In a scheme with three workers, item 1 goes to worker 1, item 2 to worker 2, and item 3 to worker 3. The load distributor then goes back around the cycle, assigning item 4 to work 1, item 5 to worker 2, and so forth. This scheme is straightforward to implement, but it assumes that any worker can handle any incoming work item and that work items are independent (that is, there is never a requirement that the work on packet N be finished before work can begin on packet (N+1)).

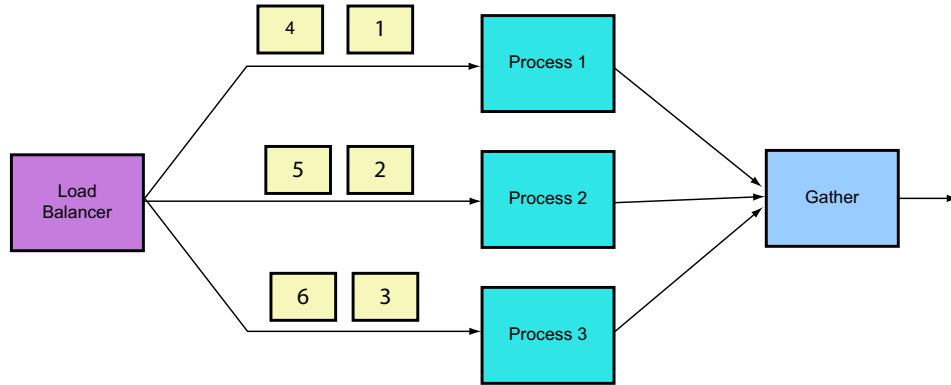


Figure 2-3: Static Round Robin Load Distribution

- **Static Flow Affinity** — Some networking applications work with packet *flows*, where each incoming packet is associated with a “connection” between two network addresses. Each connection is composed of many packets of bidirectional traffic, and it is often convenient to guarantee that all packets from a particular connection will always go to the same worker. That way, each worker can maintain connection-related state without having to synchronize it with any other workers. As shown in Figure 2-4, static flow affinity load distribution enables such applications by computing a *flow hash* based on the headers of each incoming packet. Each hash value is statically assigned to a particular worker, and that worker handles all packets whose headers produce that particular hash value. Tiler’s multicore Programmable Intelligent Packet Engine (mPIPE) library includes an implementation of static flow affinity packet distribution for use in networking applications.

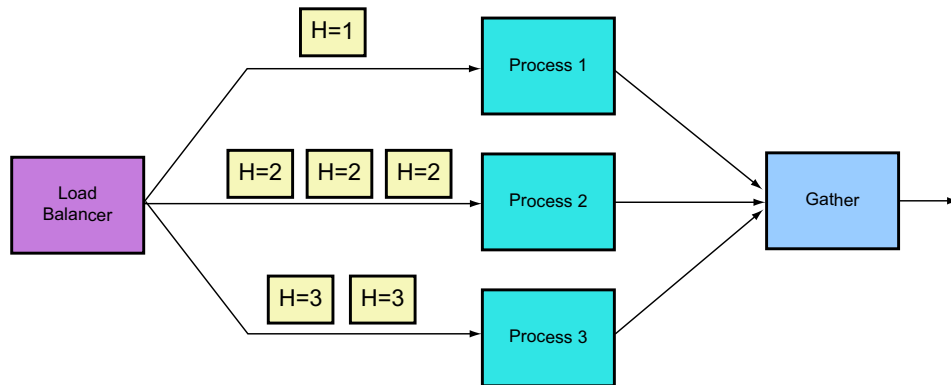


Figure 2-4: Static Flow Affinity

Notice that both of the load distribution algorithms described above can suffer performance problems if some work items or flows require more processing cycles than others. In the case of static round robin load distribution, performance can drop if, say, all of the items going to worker 1 happen to take twice as long as those going to other workers. Similarly, static flow affinity can suffer if one particular flow has more work than the others or many flows happen to hash to the same worker.

2.2.2.2 Dynamic Load Distribution

Dynamic load distribution algorithms fix the shortcomings of static load distribution algorithms by taking into account which workers are heavily loaded and which are idle. Some common dynamic load distribution schemes include the following:

- **Dynamic Round Robin** — This scheme differs from the static round robin algorithm in that work items are only forwarded to “unloaded” workers. If a worker is busy processing a work item, or has too many items queued up, it is skipped whenever its turn comes around.

As seen in [Figure 2-5](#), work items are initially distributed among all the workers. However, if, say, worker 2 becomes overwhelmed, the load balancer skips worker 2 and distributes packets among the remaining workers (1 and 3, in this case). The multicore Programmable Intelligent Packet Engine (mPIPE) library provides a dynamic round robin packet distribution algorithm for use in networking applications.

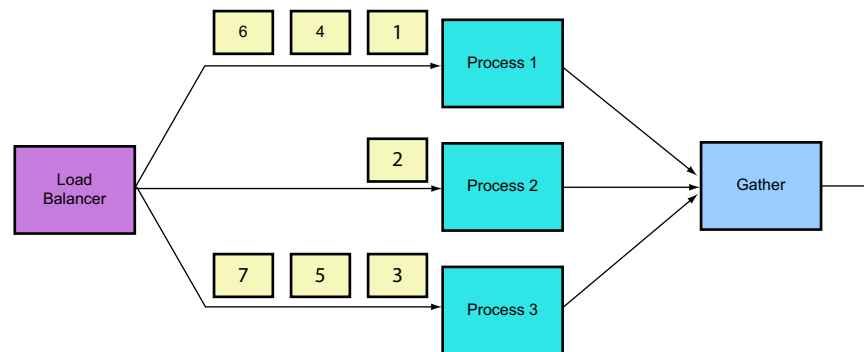


Figure 2-5: Dynamic Round Robin

- **Work Queues** — The work queue is a shared memory data structure that allows workers to dynamically redistribute work items based on which workers have free cycles. In a work queue scheme, some “producer” task constructs a shared-memory list of work items that can be executed by any worker. Each worker then pulls an item off the list, works on it, and then gets another. If one work item happens to take ten times longer than the others, the worker that is handling it will end up processing just one item while the other workers handle ten times as many.

[Figure 2-6](#) gives an example of this structure. In that figure, workers 1 and 2 each take work items that require a lot of processing. Worker 3 takes the next item, which requires fewer cycles, and when it finishes it goes back to the work queue and pulls off item number 4.

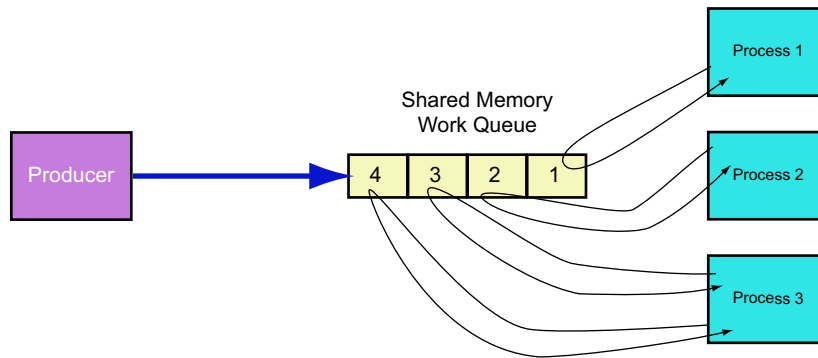


Figure 2-6: Work Queue Scheme

Work queue load distribution can be helpful in any data decomposition where work items take a variable number of cycles to complete. For example, pattern matching to check for viruses in network packets can take a variable amount of time depending on whether or not a match is found, what patterns are being checked, and the size of the packet being inspected. Similarly, in the video space, motion estimation can take a variable amount of time.

2.3 Choosing a Process Model

Once a decomposition has been selected, the programmer must decide how to author that decomposition, implementing it in terms of separate processes or threads. Different components of a decomposition, such as load balancers, workers, and monitors, can be implemented as different binary programs or as separate code within the same binary.

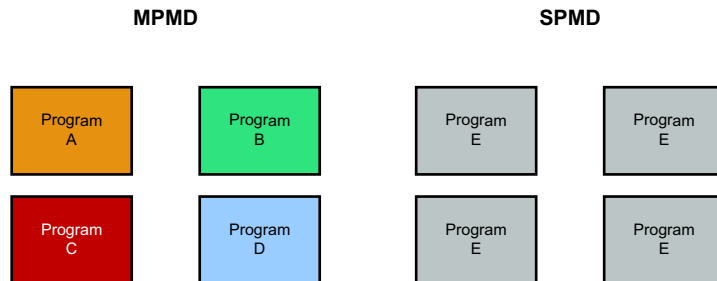


Figure 2-7: Multi-Process Program Models

As illustrated in [Figure 2-7](#), this presents two possible patterns for dividing tasks among binary programs:

- **Multiple Program Multiple Data (MPMD)** — In this model, each process is a different program, running entirely different code corresponding to different functions of the overall application. Pipelined applications are sometimes written in the MPMD style, with each phase having a separate binary. MPMD programs start by using the `fork()` and `exec()` system calls to create separate processes running different binaries.
- **Single Program Multiple Data (SPMD)** — In this model, each task is an instance of the same program. All tasks can be doing approximately the same thing, typically on different parts of the problem space, and thus implement a data decomposition. Alternatively, each thread could run

a different function, creating a functional decomposition. SPMD programs are implemented by either creating threads with `pthread_create()` or by using the `fork()` call to create multiple processes running the same program code.

Tilera supports both the MPMD and SPMD process models. In general, the SPMD approach is recommended because it makes the compilation and application startup code simpler. In particular, threaded applications make shared memory setup easier, and having a single binary is generally simpler than having to build and launch several different binaries. However, feedback-based optimization can be more effective in the MPMD model because each sub-program can be tuned in isolation without having to simultaneously tune performance for some other sub-function contained in the same binary. For more information on feedback-based optimization, see the *Multicore Development Environment Optimization Guide* (UG515).

2.4 Distributing the Work: Spatial Mapping

Once the abstract decomposition of the application has been designed, it can be mapped onto the many tiles provided by the Tile Processor. [Figure 2-8](#) shows how separate tasks can run on a specified series of tiles. By default, Linux will automatically schedule tasks (processes or threads) on available tiles and attempt to run tasks on different tiles as long as there are fewer tiles than tasks.

For improved performance, the programmer can bind each task to a specific tile. This allows each task to run continuously and independently, without being interrupted by the other tasks. Applications that use the UDN or high-performance I/O libraries like multicore Programmable Intelligent Packet Engine (mPIPE) must bind each of their tasks to a single tile. See [Section 3.3 “Understanding CPU Affinity” on page 19](#) for more information about binding tasks to tiles.

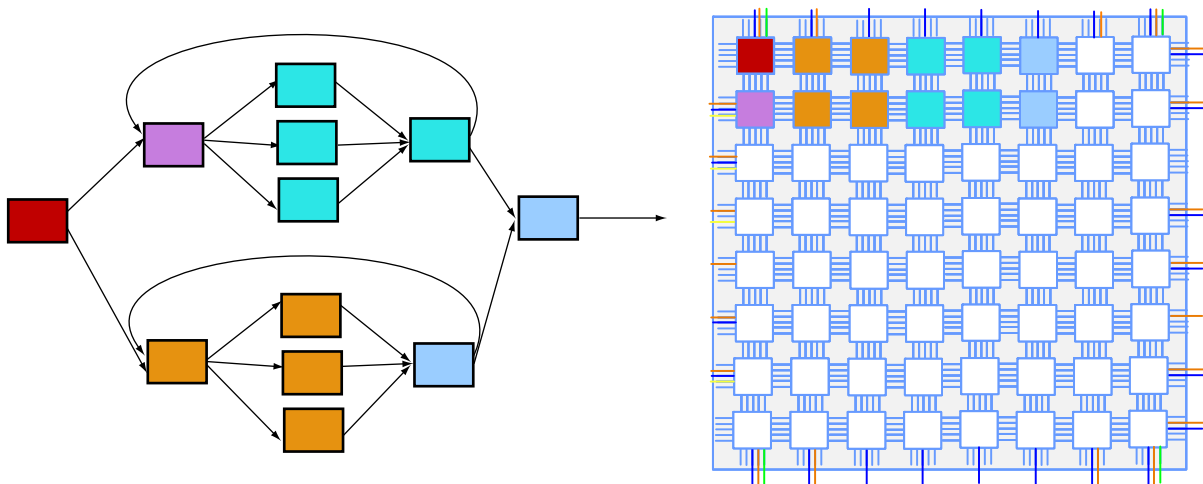


Figure 2-8: Spatial Mapping — Matching the Methodology to the Architecture

2.5 Communication Mechanisms

Using a multiprocessor effectively requires not only deciding which tiles will run each task, but also deciding how those tasks will communicate. The following sections describe the communication mechanisms available on the Tile Processor and how to choose between those mechanisms.

The Tile Processor supports two categories of communication mechanisms: shared memory and hardware networks. As depicted in [Figure 2-9](#), shared memory communication provides a shared data store accessible to many tasks. With shared memory, any task can access any piece of data at any time. Users can implement data access ordering and exclusive data access among tasks by using mutexes.

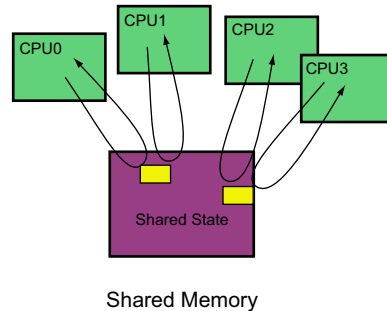


Figure 2-9: Shared Memory Communication

The second category of communication, hardware networks, provides a point-to-point messaging capability between two tiles. As is evident from [Figure 2-10](#), the hardware networks are convenient for producer-consumer relationships, in which one tile produces data that is then used as input to another tile's computation. When applicable, point-to-point communication by means of the hardware networks can improve performance by allowing the programmer to transfer words directly from the registers of one tile to the registers of another, without having to pass through the cache subsystem.

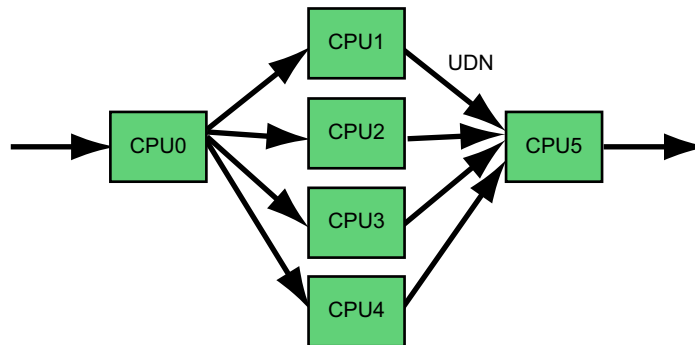


Figure 2-10: Streaming Communication

As illustrated in [Figure 2-11](#), most applications use a combination of point-to-point hardware messages and shared memory communication.

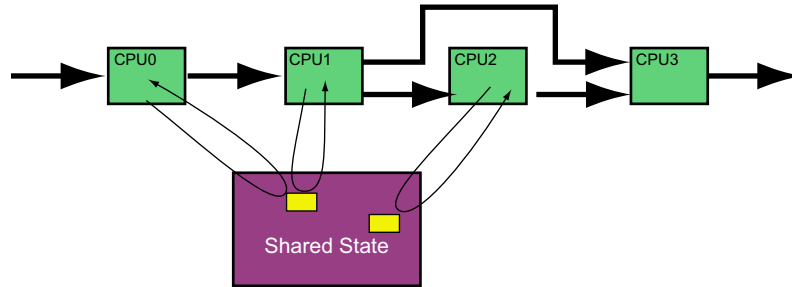


Figure 2-11: Combined Shared Memory and Streaming Mechanisms Model

2.5.1 Choosing Communication Mechanisms

The Tile Processor fully supports the cache-coherent programming model expected by many parallel applications. The processor also includes program-accessible, hardware networks that applications can use to transmit tile-to-tile messages with very low latencies. This section gives an introduction to both the shared memory and hardware network approaches to inter-tile communication.

2.5.2 Shared Memory

Shared memory is the most flexible communication mechanism provided by the Tile Processor. It can be used for both bulk data storage, in which data objects are saved aside for later use at some indeterminate time and tile, and for point-to-point communication, in which one tile produces a data item that is consumed by some other tile.

Shared memory communication allows each task in a parallel application to load and store values from a globally visible region of memory. Each task in the application can access any object in shared memory at any time. However, this flexibility comes at a price — access to shared memory objects must be synchronized so that only one task accesses a shared object at a time. Failing to properly synchronize access to shared memory can lead to difficult-to-detect bugs in which two tasks update an object at the same time, leaving it in an inconsistent state.

Shared memory is the only communication method that can be used for bulk data storage. The Tile Processor's cache subsystem automatically brings data into the on-chip caches when that data is accessed, potentially displacing older data from the caches when storage is needed for the newly accessed data. The cache subsystem's ability to preferentially store recently used data can significantly improve performance when a program maintains a large shared dataset (for example, a flow table representing a million flows), but at any given point in time only a small part of data (for example, the state representing a given flow) is being accessed. Other examples of data generally stored in bulk shared memory include network packets and video frames.

Point-to-point communication primitives can also be implemented by means of shared memory. These include a variety of producer-consumer data structures such as:

- Single-sender, single-receiver FIFOs.
- Multiple-sender, single-receiver *sink* FIFOs.
- Single-sender, multiple-receiver work queues.

These primitives are generally implemented using a ring buffer and two counters, one for the number of items produced and another for the number of items consumed. In cases with multiple senders or receivers, a shared memory mutex is used to synchronize access.

The Tile Processor's cache-homing mechanism, described in [Chapter 4: Memory Homing and Allocation Concepts](#), can be used to tune the performance of shared memory FIFOs. Cache-homing allows the user to specify a particular tile that holds the canonical version of each cacheline. Whenever data is written to that cacheline, the write is immediately forwarded to this "home tile". Data structures can be carefully homed to take advantage of two important principles:

- Loads are faster when issued to data that is homed on the issuing tile.
- Stores are fire-and-forget; storing to a remote cache has minimal performance impact because the main processor does not have to wait for a read result to come back.

For performance-critical FIFO data structures, cache homing can be used to home data so that stores go to remote tiles and loads are issued on the home tile. In particular, the ring buffer that stores the data objects is generally homed at the consumer, so that the consumer finds the data in its L2 cache when it issues load instructions. The producer generally suffers no performance impact when storing data to a remote cache because store instructions do not block the execution of subsequent instructions. Similar optimizations can be made for the producer and consumer counts.

2.5.3 Hardware Networks

In addition to shared memory, the Tile Processor provides a set of hardware networks for sending messages between tiles, primarily the UDN, which you use for user space messages.

Most applications use only the UDN because it is available to user programs.

These hardware networks are generally used to optimize one or two latency-critical, point-to-point communication tasks. The hardware networks offer improved performance because data is sent directly from registers on one tile, across the network, to registers on another tile, without having to go through the cache subsystem (which can take ~10 cycles on each tile). On the other hand, attempting to use the networks for too many purposes at once can lead to deadlock problems. Avoiding deadlock generally requires that the program's hardware network send and receive operations be carefully designed so that either:

- All possible packets destined for a particular tile can fit into that tile's receive-side demux buffer, so packets do not "back up" into the network, or
- Each network link is only used to transmit messages between a unique pair of tiles.

Maintaining either of these guarantees generally requires placing strict limits on the types of packets that are sent over the UDN. Thus, we recommend that application developers start by using shared memory for point-to-point communication. Then, if one or two point-to-point primitives prove to be particularly critical for application performance, they can be re-implemented by means of the UDN. For example, many applications find that load balancing is the most critical synchronization operation and choose to implement that communication by means of the UDN.

CHAPTER 3 DEVELOPING STANDARD LINUX APPLICATIONS

This chapter describes how to develop applications using Tilera’s standards-based Linux environment:

- [Section 3.1 Creating Standard Linux Applications](#)
- [Section 3.2 Creating Parallel Tasks](#)
- [Section 3.3 Understanding CPU Affinity](#)
- [Section 3.4 Synchronization](#)

For information about programming in the Tilera-specific Zero Overhead Linux™ (ZOL) environment, see [Chapter 7: Zero Overhead Linux Applications](#).

3.1 Creating Standard Linux Applications

The Tilera® Multicore Development Environment™ (MDE) provides a standard Linux 3.10 operating environment for the Tile Processor™, including a full standard C library and numerous common Linux libraries. Many applications can be ported to the platform with a simple recompile using the MDE tools, with little to no source code modification.

When compared with the Zero Overhead Linux (ZOL) and bare metal environments, the *standard* Linux environment described in this chapter provides the most flexible development environment. Applications running on standard Linux tiles are free to make system calls, run multiple tasks per core, and invoke whatever I/O routines they choose. ZOL’s *dataplane* tiles, by contrast, should only run one task per core, and zero overhead is only maintained if that task avoids system calls and uses specialized I/O routines. The bare metal environment provides no Linux compatibility at all.

Thus, the standard Linux environment is the best target for easy porting of pre-existing proprietary and open-source applications. For many applications, moving to the Tile environment is as simple as cross-compiling, uploading the program, and running. Once an application is running in the standard Linux environment, developers can choose to add incremental, Tile-specific optimizations. Some of these optimizations are applicable on standard Linux tiles, as described in this chapter. In other cases, a developer may choose to move tasks with more strict real-time requirements onto ZOL *dataplane* tiles or even port those code portions to run on bare metal tiles.

3.2 Creating Parallel Tasks

The standard Linux environment supports both thread-based and process-based multi-core applications.

Choosing to use threads or processes to express parallelism depends on a combination of application characteristics and personal preference. Threads share a common memory address space, so allocating and accessing shared memory tends to be easier in threaded programs. Threads also tend to be more familiar to many programmers.

On the other hand, processes have a private-default memory model. This fits particularly well when the parallelization strategy is to simply run parallel copies of a program on different data streams. Because global variables are private to each process, there is no need to worry about synchronizing access to static data. Using processes also allows an application to map more than 3 gigabytes of memory into an application by limiting the amount of shared memory mapped into the virtual address space and allocating large amounts of private memory in each process.

3.2.1 Creating Multiple Threads

The `pthread_create()` API is familiar to most programmers. It creates a new thread that runs a function specified in the `pthread_create()` parameters. For example, the following code, taken from `examples/tmc/memory_perf/memory.c`, can be used to create an application with `count` threads:

```
// This method creates (count - 1) child threads and calls 'func' in
// each thread, passing the rank to each thread. The caller is
// assigned the lowest rank.
//
void
run_threads(int count, void*(*func)(void*))
{
    for (int i = 1; i < count ; i++)
    {
        if (pthread_create(&threads[i], NULL, func, (void*)i) != 0)
            tmc_task_die("pthread_create() failed.");
    }

    (void) func((void*) 0);
}
```

3.2.2 Creating Multiple Processes

Similarly, the `fork()` system call can be used to create parallel processes executing the same program code. The child process is cloned at the point of the `fork()` call, which returns 0 in the child and the child's PID in the parent. The following code, taken from `examples/tmc/multi_tile/multi_tile.c`, shows how to create an application with a total of `count` processes:

```

// This method forks off (count - 1) child processes and returns the
// 'rank' in each process. The parent has rank 0, and children have
// rank 1, 2, 3, etc.
int
fork_processes(int count, cpu_set_t* cpus)
{
    for (int i = 1; i < count; i++)
    {
        pid_t child = fork();
        if (child < 0)
            tmc_task_die("fork() failed.");

        if (child == 0)
        {
            // We're a child; return our rank.
            return i;
        }
    }

    // The parent always returns rank 0.
    return 0;
}

```

3.3 Understanding CPU Affinity

Each Linux task (a process or thread) has a set of processors on which it is allowed to run. This is known as its *CPU affinity set*. CPU affinity sets allow tasks to be restricted to run only on particular processors, allowing the programmer to guarantee that a task will not migrate to other cores and interfere with whatever is running there.

3.3.1 Load Balancing Using CPU Affinity

The Linux task scheduler automatically load balances tasks across available processors. As a result, most throughput-oriented applications can avoid explicitly setting each task's affinity. This is particularly true for applications that have more tasks than cores or that spend a lot of time blocked on syscalls. In such cases, the Linux load balancer can improve performance by migrating tasks to free cores as they become available. By contrast, most embedded applications benefit from manually specifying task affinity, often for one of the following reasons:

- **Dataplane tiles** — Programs that wish to execute a task on a ZOL *dataplane* tile can only do so by explicitly setting its affinity to that tile.
- **Program configuration** — Some programs can spawn a variable number of tasks. A good convention for such programs is to have them query their initial CPU affinity on startup by calling `tmc_cpus_get_my_affinity()`, count the number of tiles in that set by calling `tmc_cpus_count()`, and spawn that many tasks. This way both the number of tasks and where they are located on the chip can be controlled by setting the initial affinity of the entire application to the desired tiles when launching it. For more information, see the TMC `cpu_set_t` documentation in the *Application Libraries Reference Manual* (UG527).

When a program uses its affinity set to control the number of tasks spawned, it is often a good idea to lock each task to a separate tile in that set by having each task call `tmc_cpus_find_nth_cpu()` to find its CPU, and then call `tmc_cpus_set_my_cpu()` to lock itself to that CPU. This prevents Linux from randomly migrating tasks around within the tiles in the affinity set, which can be slightly inefficient and make some debugging information harder to follow. For an example of this pattern, see `$TILERA_ROOT/examples/tmc/multi_tile/`.

- Linux command `taskset` — The Linux standard command `taskset` allows a more popular and general way to affinitize a program onto a specific tile. For more information, see [Section 3.3.2.3 “Specifying Initial Affinity By Means of `taskset`” on page 21](#).
- UDN — The header of UDN packets addresses them to a specific tile, so it is critical that the receiving task be running on that tile and never move. The only way to accomplish this is by setting its affinity to a set containing only that tile. The Linux UDN hardware feature (see the `tmc/udn.h` documentation in the *Application Libraries Reference Manual* (UG527)) enforces that tasks lock themselves to one tile before they can use the UDN. For more information, see [Section 7.9 “User Dynamic Network \(UDN\)” on page 75](#).
- High-speed I/O — The Tile Processor includes on-chip I/O controllers, including DDR3 memory, XAUI and GbE Ethernet, PCIe, and other assorted I/O. Separate I/O shims on the edges of the Tile Processor manage each actual hardware interface, and have one or more connections to the iMesh™ interconnect. Commands and responses flow over the processor’s memory networks, which connect all tiles and I/O shims. Various APIs allow the programmer to control the I/O.
- Memory controller optimizations — Certain high-performance programs might want to locate tasks on tiles near memory controllers they use heavily. This maximizes memory network bandwidth by avoiding unnecessary link congestion. On the Tile Processor, being on the same left/right half of the chip as the memory controller is particularly helpful. This optimization is most relevant to applications that use PCIe to transfer high-bandwidth data between DRAM and the I/O subsystem. The optimization is usually unnecessary for an application’s heap memory (for instance, memory returned by `malloc()`), because Linux automatically prefers memory from a nearby controller.

3.3.2 Specifying Initial Affinity

Recall from [Section 2.3.2 “Tile Classes” on page 9](#) that each tile can be classified as a *hypervisor dedicated tile*, a *BME tile*, a *standard Linux tile*, or a *dataplane Linux tile*. Within the set of Linux tiles, standard tiles run the normal Linux scheduler, and dataplane tiles disable various Linux services in order to minimize interrupt overhead. If no affinity is specified when running a program, it defaults to the set of *standard* tiles (that is non-dataplane tiles available for Linux to use). This is a convention enforced by the shepherd and the default Linux `init` process.

The remainder of this section describes three techniques that can be used to set a program’s initial affinity to something other than the default, standard-tile-only affinity.

3.3.2.1 Specifying Initial Affinity By Means of `tile-monitor`

`tile-monitor` takes a `--tile` option that indicates the set of tiles on which the following program (and only that program) should be run. For example:

```
tile-monitor --tile 1,2 -- /bin/my_program
```

would run the program `/bin/my_program` on tile 1,2. This invocation:

```
tile-monitor --tile 3x4@1,2 -- /bin/my_program
```

would run it in a 3x4 rectangle whose top left corner was 1,2. Of course, specifying a multi-tile set like this makes sense mostly for multi-threaded or multi-process applications.

By default, the initial affinity for all programs is the set of standard Linux tiles. If your hypervisor configuration file specifies dataplane tiles, you can use the following command to run on the dataplane tiles:

```
tile-monitor --tile dataplane -- /bin/my_program
```

You can specify multiple `--tile` options per invocation, in which case the affinity set is the union of those sets. For example the following command runs an application on tiles 2,2 and 3,3:

```
tile-monitor --tile 2,2 --tile 3,3 -- /bin/my_program
```

`tile-monitor` resets the current affinity set back to the default (*standard affinity*) each time it runs something, so be sure to specify the desired affinity immediately before running each program. For example:

```
tile-monitor --tile 1,1 --run ++ /bin/prog1 +- --tile 2,2 --run +- /bin/prog2
+-
```

3.3.2.2 Specifying Initial Affinity By Means of the Shepherd

`tile-monitor` runs on the host and communicates with the program `shepherd` running on the Tile Processor. `shepherd` takes care of all of the actual system calls to implement commands sent from `tile-monitor`. See the `tmc/task.h` documentation in the *Application Libraries Reference Manual* (UG527) for more information on the `shepherd`.

You can run the `shepherd` process directly, without a `tile-monitor`. One use for it is to run a program on a set of tiles, using the same `--tile` syntax as `tile-monitor`. For example:

```
shepherd --tile dataplane /bin/my_program
```

Note that the `shepherd` process continues to run as long as the spawned program is alive, to watch for crashes. It is a lightweight process, but this may be unwanted. `taskset` provides a way to run a completely unwatched program on designated tiles.

3.3.2.3 Specifying Initial Affinity By Means of taskset

`taskset` is a command-line program that runs on the Tile Processor, making it suitable for use when `tile-monitor` is not available. Its sole function is to run another program with a specified affinity set. In contrast to the `shepherd`, it does not leave a process around watching the spawned program. Its syntax is different from the `shepherd`'s but accomplishes the same thing.

`taskset` works in terms of CPU numbers, the same ones used when indexing affinity sets programmatically. The CPU number for a tile is computed as $x + (y * \text{width})$, so the top left CPU is number 0, the one to its right is number 1, and on an 6x6 chip, the first CPU on the second row is number 6, and so forth.

So on a 6x6 chip, to run something on the third tile on the second row, you would say:

```
taskset -c 8 /bin/my_program
```

`taskset` can also specify multiple CPUs. For example:

```
taskset -c 1-5,10,14 /bin/my_program
```

The file `/sys/devices/cpu/dataplane` contains a comma- and dash-separated list of dataplane CPU numbers. In order to specify the initial affinity of an application as the set of dataplane tiles, you can say, for example:

```
taskset -c `cat /sys/devices/cpu/dataplane` cocryptographic mmand args ...
```

Note that any program started from the shell inherits the shell's affinity by default. To set the affinity of the current Bash shell to CPU 8:

```
taskset -cp 8 $$
```

Or to set the affinity of the current process to all the tiles:

```
taskset -cp 0-63 $$
```

3.3.3 Setting Affinity Programmatically

A task can change its own affinity to anything it likes by calling `tmc_cpus_set_my_affinity()`, or, for programs not using the TMC library, by calling `sched_setaffinity()`. It is also possible to modify the affinity set of another task. The TMC library contains a variety of `tmc_cpus_` functions useful for querying and modifying affinity sets. For information, see the *Application Libraries Reference Manual* (UG527).

Note: The Linux man page for the `sched_setaffinity` function implies that the set of available CPUs is contiguous starting with CPU ID 0. That is incorrect; no assumptions should be made about particular CPUs being available, or the set of CPUs being contiguous, because CPUs can be taken offline dynamically or be otherwise absent. In Tile Linux, dedicated hypervisor and bare metal environment (BME) tiles are always unavailable.

When a task's affinity is changed the new affinity is not constrained to be a subset of its current affinity. It can *escape* if it wants to, although by convention that is rarely done. Typically one task spawns other tasks and locks them to distinct tiles in its original affinity set, so that some outer tool like `taskset` can easily control the set of tiles used by the entire application.

When one task wants to spawn a task with a particular affinity set, a typical implementation is to execute a `fork()` and have the child task set its own affinity before calling `execve()`. This is how the command-line tools described in the previous section work. Other implementations are of course possible, such as changing the child task's affinity after it calls `execve()`, but that approach tends to leave the child process running on the parent's CPU for many cycles because of the expense of the `execve()` system call.

Similarly, when spawning threads that need to be locked to a tile it is customary to have each thread call `tmc_cpus_set_my_affinity()` as soon as it starts running, based on a CPU number passed to it by the spawning thread.

For an example of how to manage affinity in a thread-based application, see `examples/tmc/memory_perf/memory.c`. For an example using multiple processes, see `examples/tmc/multi_tile/multi_tile.c`.

3.4 Synchronization

3.4.1 Pthreads Synchronization

Most standard, threaded applications use the Pthreads (POSIX Threads) APIs to synchronize access to shared memory data structures. Tiler's implementation of the Pthreads library includes support for mutexes (`pthread_mutex_t`), condition variables (`pthread_cond_t`), and barriers (`pthread_barrier_t`).

The most frequently used of these primitives is the mutex, which provides mutual exclusion for shared memory objects. A mutex is *locked* by invoking `pthread_mutex_lock()`. Once a thread obtains the lock, no other thread can obtain that lock until the holding thread invokes `pthread_mutex_unlock()`. This mechanism is most frequently used to prevent more than one thread from accessing an object at the same time. For example, a mutex is often used to guard counters so that two threads will not load the count value at the same time, both add one (getting the same result), and both write back that value, thus incrementing the counter once instead of twice.

Condition variables and barriers are less frequently used, but still useful on occasion. The condition variable is generally used to synchronize producer/consumer relationships: the producing thread invokes `pthread_cond_signal()` to wake the consuming thread from `pthread_cond_wait()`. The `pthread_barrier_t` allows many threads to guarantee that they have all reached some phase transition before continuing.

All of the pthread synchronization primitives may put a thread to sleep if a resource (mutex, barrier, and so on) is unavailable. This behavior is desirable in cases where more than one thread is running on the same CPU — putting one thread to sleep allows another thread to run. Other techniques, for example the `tmc_spin_mutex_t` primitive discussed later in this book, might be more appropriate in cases where there is only one thread per core.

3.4.2 TMC Synchronization

The Tiler Multicore Components (TMC™) library provides synchronization primitives that work with both thread-based and process-based applications. This capability is particularly useful if an application wishes to avoid linking with Pthreads.

TMC provides both a mutex (`tmc_sync_mutex_t`) and a barrier (`tmc_sync_barrier_t`). Both of these primitives use the same algorithm as the Pthreads version and thus have similar performance characteristics — a task will be put to sleep if the desired resource is unavailable.

TMC also provides *spinning* synchronization primitives that are generally faster than the `tmc_sync` implementations. These primitives (`tmc_spin_mutex_t`, `tmc_spin_queued_mutex_t`, and `tmc_spin_barrier_t`) are tuned for use in applications that have a single task per tile. They are intended for use with Zero Overhead Linux *dataplane* tiles and are described in [Chapter 7: Zero Overhead Linux Applications](#).

CHAPTER 4 MEMORY HOMING AND ALLOCATION CONCEPTS

This chapter describes memory homing strategies and introduces Tiler's APIs for memory allocation:

- [Section 4.1 Overview of Memory Homing in the Tiler Environment](#)
- [Section 4.2 Local Homing Strategy](#)
- [Section 4.3 Remote Homing Strategy](#)
- [Section 4.4 Hash-For-Home Strategy](#)
- [Section 4.5 Specified-Attribute Memory Page Allocation](#)
- [Section 4.6 Multiple Heap Allocation \(mspaces\)](#)
- [Section 4.7 Allocating Shared Memory](#)

See also:

- [Chapter 5: Controlling Memory Allocation in an Application](#)
- [Chapter 6: Dynamic Distributed Cache Technology](#)

4.1 Overview of Memory Homing in the Tiler Environment

Each physical address of memory in a Tiler system is associated with a *home tile*.

The home tile is a tile designated to maintain and track sharing and coherence information for a particular physical address. For example, if two tiles are both caching the same address, *P*, this information is tracked at the home tile for address *P*.

From an implementation perspective, this information is stored and managed by a hardware structure called a directory. In the event that an entity (processor or I/O device) in the system modifies address *P*, the sharing information at the home tile is queried in order to:

1. Identify which tiles within the system are caching address *P*.
2. Properly notify those tiles that address *P* has been modified.

The home tile is part of the cache coherency system, and is a requirement for properly maintaining a coherent view of memory across all of the tiles and I/O devices. Shared memory programming models, such as Pthreads, require that the underlying system support cache coherency, enabling two threads to communicate by means of physical memory.

In addition to tracking sharing information, the home tile often caches the data for which it is serving as a *home*. In these scenarios, a cache line might be cached by several sharing tiles, as well as by the home tile. When the home tile is caching the data, we call this a level 3 (L3) cache, because the home tile can be thought of as a level of cache beyond the local L2 cache.

In the Tiler architecture, the union of all of the L2 caches serves as the distributed L3 cache. There is no disparate hardware cache for caching homed blocks. Instead, the level 3 blocks are stored in the home tile's L2 cache. In a sense, the *home tile* mechanism provides a virtual L3 cache on top of the normal L2 cache system. This enables scenarios where, after an L2 cache miss, the home tile might be queried for data, and the data serviced from the home tile, rather than from DDR memory.

With the L3 cache, for each cache line you can tell it where the home tile is. If some tile finds that a copy of that cache line is not present in its L2, it will ask the home tile for a copy. If some other tile writes new data to the cache line, the home tile is responsible for invalidating all the other copies out there, so they will re-fetch a copy of the modified version the next time they need it.

The Tile Processor provides a programmatic means for controlling the homing attributes for a given physical address. This is done by means of the TLB within each tile. In the Tiler environment, TLBs not only translate and map a virtual address to a physical address, but they also use the same mapping mechanism to determine the home tile for a given address. Therefore, the result of a virtual address translation by means of the TLB is not just the corresponding physical address but the home tile for the physical address as well.

Because the TLBs are controlled through software (a TLB miss handler in the hypervisor), the hypervisor needs to ensure that all TLB entries through the system are consistent. In other words, if two different cores are accessing the same physical address, P, the TLB entries on those two cores must be identical, ensuring that both cores have the same home tile for address P.

While the *home tile* mechanism is used primarily for cache coherency, it also is important for application performance. In particular, the homing strategy has an impact on the latency of memory read instructions that miss in the L2 cache. In some homing scenarios, on an L2 cache miss, the home tile might be queried for data. In other scenarios, a miss results in a direct access to memory, ignoring other L2 caches in the system.

The Tiler memory subsystem has three different classes of homing, each useful for certain types of memory:

- Local homing (see [Section 4.2 “Local Homing Strategy” on page 26](#))
- Remote homing (see [Section 4.3 “Remote Homing Strategy” on page 27](#))
- Hash-for-home (see [Section 4.4 “Hash-For-Home Strategy” on page 28](#))

Note: In general, hash-for-home provides the best out-of-the-box performance, and is the default homing policy for many different classes of memory.

4.2 Local Homing Strategy

With the *local homing* strategy, an entire page of memory (64KB or 16MB) is homed on the same tile that is accessing the memory. In this scenario, when an access to address P misses in the L2 cache, a request is sent directly to DDR memory in order to retrieve the data.

This scheme differs from the remote homing and hash-for-home schemes: for those schemes, L2 read misses result in messages being sent to another tile, rather than directly to memory (as is the case for an L2 miss to a locally homed page).

Local homing has the advantage of potentially having a lower L2 miss latency than the other schemes. However, with local homing, you cannot use other tiles' caches as a backing L3 cache, so the capacity of the cache is limited to the size of a single L2.

[Figure 4-1](#) shows how, on an L2 read or write miss to locally homed memory, a request is sent directly to the DDR memory.

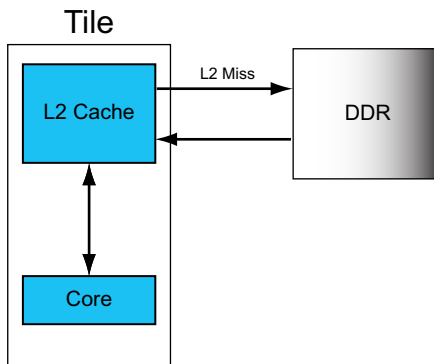


Figure 4-1: Example of Local Homing

Uses

Local homing is useful for private data that would not benefit from using another core's cache as a backing L3 (either because of a low L3 hit rate or because the data fits in the local L2).

Advantages

For small private working sets, local homing can reduce overall cache occupancy by avoiding having a copy of the data at both the reader and home. Also, for data that never hits in the cache, the local homing strategy provides the lowest latency.

Disadvantages

Local homing is not good for highly shared data or limited cache capacity.

Defaults

Local homing is used by default for the stack for processes and threads.

4.3 Remote Homing Strategy

With the *remote homing* strategy, an entire page of memory (65 KB or 16 MB) is homed on a different tile than the tile accessing the data. In this scenario, when an access to address P misses in the L2 cache, a request is sent to the home tile (sometimes called the L3). At the home tile, the coherency information is updated. Additionally, if the data is present in the home tile, then the request is serviced directly with the data from the home tile. If the data is not present, a request is sent to memory and the memory controller services the request.

If the data is present at the home tile, the latency of the L2 miss is significantly lower than going to memory. However, if the data is not present in the home tile, then the L2 miss latency is longer than that of the locally homed scenario. On L2 write misses, the write issues through directly to the home, and updates the value at the home.

The remote homing strategy can be viewed as the same as the locally homed strategy, except from the point of view of a different tile accessing the data (for example, a thread in a pthreaded program accessing another thread's stack data).

The main usage model for remotely homed memory is for producer/consumer cases. In the case of a shared memory FIFO, with a single consumer reading the FIFO, it is best to home FIFO data on the consuming tile. With this configuration, the producing tiles will issue stores directly to the consuming tile's cache.

Figure 4-2 shows the scenario for a remotely homed page. On read/write misses in the L2, a request is sent to the home tile, where the request might be serviced (if the data is present), or passed along to DDR memory (if the data is not present).

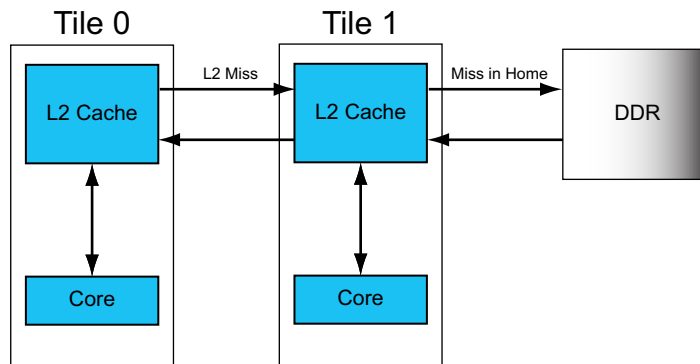


Figure 4-2: Example of Remote Homing

Uses

Shared memory FIFOs provide a good use for remote homing: Home the FIFO data on the consuming tile, enabling the producing tiles to write directly to the consuming tile's cache.

Advantages

For shared memory FIFOs, a producer can write directly to a consumer's cache, without polluting the producer's cache.

Disadvantages

Remote homing is only useful for shared memory FIFOs with a single reader.

Defaults

If a thread in a Pthreads program accesses another thread's stack data, that will be a remotely homed access (assuming the other thread is scheduled on a different tile).

4.4 Hash-For-Home Strategy

The *hash-for-home* strategy is very similar to the remote homing strategy. However, for remote homing, an entire page is mapped on a single home tile; with hash-for-home, the entire page is hashed across a set of tiles within the system, at a cache line granularity. This spreads the memory across multiple tiles, enabling an even traffic distribution through the on-chip networks, and effectively load balancing the cache traffic across a set of tiles.

As with the other strategies, the TLB translates a given virtual address to a physical address. If the page is marked as hash-for-home, upper bits of the physical address are hashed together and used to index a table that will then specify the appropriate home. The set of tiles used to home hash-for-home pages can be programmatically controlled (see below).

Hash-for-home is a good default policy, providing excellent performance for shared instruction and data memory, and reasonable performance for private data memory. The reason hash-for-home performs well is because it effectively utilizes the aggregate L2 cache bandwidth of the entire chip, and avoids overloading any single tile with too many requests.

In general, an ideal setup uses hash-for-home memory for shared instruction and data memory, and homes private data locally, on the core responsible for reading the private data. Remote homing can also be used for high performance single-reader shared memory FIFOs.

Uses

Hash-for-home is useful for any data or instruction memory that is shared across multiple threads/processes.

Advantages

By spreading the memory across a set of tiles, network congestion is alleviated and it is less likely that a given tile's L2 cache controller will be overwhelmed with requests from other cores.

Disadvantages

Hash-for-home is not suitable for private, single-reader data.

Defaults

Static and dynamic data and instruction memory are all hash-for-home by default, for all types of applications.

For more information, see [Section 5.5 “Using Hash-for-Home” on page 42](#).

4.5 Specified-Attribute Memory Page Allocation

The Tile Processor allows application developers to tune memory allocation policies in order to improve application performance. The programmer can specify how data should be stored in on-chip caches, which memory controller to use, and what TLB page size to use.

The TMC™ (Tilera Multicore Components) library's `tmc/alloc.h` routines provide a unified API for controlling these parameters by means of the `tmc_alloc_t` structure. A `tmc_alloc_t` initialized to `TMC_ALLOC_INIT` results in memory with a set of default properties. A variety of setter methods allow you to modify these default policies to tune for particular performance considerations.

For example, the `tmc_alloc_set_home()` method controls the CPU or CPUs on which pages are homed:

```
// Allocate memory homed on this task's CPU, and dynamically rehome
// the pages if this task moves to another CPU.
tmc_alloc_t alloc = TMC_ALLOC_INIT;
tmc_alloc_set_home(&alloc, TMC_ALLOC_HOME_TASK);
p1 = tmc_alloc_map(&alloc, size);
// Allocate hash-for-home memory.
tmc_alloc_set_home(&alloc, TMC_ALLOC_HOME_HASH);
p2 = tmc_alloc_map(&alloc, size);
```

Similarly, the `tmc_alloc_set_huge()` method configures the allocator to allocate huge (16MB) memory pages. Multiple setter methods can be invoked on the same allocator object:

```
// Allocate huge page, hash-for-home memory.
tmc_alloc_t alloc = TMC_ALLOC_INIT;
tmc_alloc_set_huge(&alloc);
tmc_alloc_set_home(&alloc, TMC_ALLOC_HOME_HASH);
p3 = tmc_alloc_map(&alloc, size);
```

A `tmc_alloc_t` structure can also be passed to the `tmc_mspace` heap allocator in order to allow custom memory allocation by means of a familiar interface much like that available for use with the `malloc()` routine. See [Section 4.6 “Multiple Heap Allocation \(mspaces\)”](#) on page 30.

For more information, see [Section 5.1 “Using the `tmc\_alloc\(\)` API to Allocate Memory Pages”](#) on page 33.

4.6 Multiple Heap Allocation (mspaces)

The TMC library provides a heap-based memory allocation API that can be used in addition to the standard `malloc()` or the `free()` APIs in `libc`.

An *mspace* (memory space) is also known as a heap or arena. Memory space objects allow the programmer to create multiple memory allocation heaps, and, if desired, set different memory attributes for each heap. The `tmc_mspace_create_special()` API allows the creation of an *mspace* that gets its memory from the system using `tmc_alloc_map()` and a specified `tmc_alloc_t`.

For example, to request memory on controller 2, you might do this:

```
tmc_alloc_t alloc = TMC_ALLOC_INIT;
tmc_alloc_set_node_preferred(&alloc, 2);
msp = tmc_mspace_create_special(0, 0, &alloc);
```

Note: To enable choosing the memory controller programmatically, you must disable the default *memory striping* mode; see the section on “Controlling Memory Striping” in the *Multicore Development Environment System Programmer’s Guide* (UG509).

For more information, see [Section 5.2 “Using the `tmc\_mspace\(\)` API to Allocate Memory Spaces”](#) on page 35.

4.7 Allocating Shared Memory

Applications that run multiple tasks often require *shared memory* regions that are accessible to all tasks in the application.

Pthreads-style shared memory is the natural state in a program with multiple threads. All memory is shared by default. Changes written to memory in any thread are immediately visible to any other thread. Any `mmap()` calls made by one thread can immediately be used by another thread.

In process-based applications, memory allocations must be specially configured in order to be shared. By default, memory allocations are private to whatever process allocates the memory.

The Tilera MDE provides two mechanisms for sharing memory between processes:

- Standard Linux `MAP_SHARED` memory (see [Section 4.7.1 “Standard Linux `MAP\_SHARED` Memory”](#) on page 31)
- Tilera-provided “common memory” API, `tmc_cmem()` (see [Section 4.7.2 “Tilera-Provided Common Memory API, `tmc\_cmem\(\)`”](#) on page 31)

4.7.1 Standard Linux MAP_SHARED Memory

The Linux MAP_SHARED flag allows two or more processes to share memory by creating a writable mapping of a particular file. Changes to the mapped memory in either process are reflected in the file and in the memory of the other process. However, all participating processes must separately issue their own mmap() calls to get access to this memory, and they must each tolerate the mapping appearing at different parts of their address space, or somehow agree on a good address to use and specify that address.

MAP_SHARED memory will be shared between the allocating process and any child processes created after the allocation using fork(). As a result, it is useful for applications that can allocate all of their memory at startup, but is not appropriate for applications that need to allocate more shared memory after creating many parallel processes. MAP_SHARED memory is also inappropriate for applications that call execve() after fork(), because the exec system calls erase all memory mappings (in particular the shared ones).

This type of memory can be allocated by means of tmc_alloc_map() using a tmc_alloc_t allocator object that has been passed to the tmc_alloc_set_shared() API to set the MAP_SHARED flag. (For more information, see [Section 5.1.1 “Functions for Allocating and Freeing Memory” on page 34.](#))

4.7.2 Tilera-Provided Common Memory API, tmc_cmем()

Common memory is shared memory that is mapped on all participating processes at the same virtual address. This property allows pointers into common memory to be passed between processes or stored in common memory. This is more powerful than the normal idiom of cross-process shared-memory mappings, because the OS locates available address space in all participating processes and maps the memory in all participating processes with a single call. In addition, any process can create new mappings that become visible in all related processes, rather than the usual model where forked children can only see mappings that were created in the parent before the fork occurred.

The tmc_cmем() API has similarities with the normal Pthreads-style of sharing memory and with the MAP_SHARED style of sharing memory and provides the advantages of both.

Note: Initializing and managing the common memory subsystem can be somewhat complicated, and using the tmc_cmем() common memory allocator is significantly slower than allocating memory in a normal fashion. Thus, we recommend that applications use the normal Pthreads-style of sharing memory or MAP_SHARED shared memory if possible.

For more information, see [Section 5.3 “Using the tmc_cmем\(\) API to Access Common Memory” on page 38.](#)

CHAPTER 5 CONTROLLING MEMORY ALLOCATION IN AN APPLICATION

The chapter contains these topics:

- [Section 5.1 Using the `tmc\_alloc\(\)` API to Allocate Memory Pages](#)
- [Section 5.2 Using the `tmc\_mspace\(\)` API to Allocate Memory Spaces](#)
- [Section 5.3 Using the `tmc\_cmem\(\)` API to Access Common Memory](#)
- [Section 5.4 Memory-Mapping Flags to Control Caching](#)
- [Section 5.5 Using Hash-for-Home](#)
- [Section 5.6 Understanding Virtual Memory and Page Size](#)
- [Section 5.7 Configuring Memory Page Sizes under Linux](#)
- [Section 5.7.1 Page Tables and Page Sizes](#)
- [Section 5.7.2 Configuring Huge Page Sizes](#)
- [Section 5.7.3 Allocating Huge Pages](#)
- [Section 5.8 Allocating Too Much Memory](#)
- [Section 5.10 Unaligned Memory Accesses](#)
- [Section 5.11 Additional Tips on Managing Alignment of Data](#)
- [Section 5.12 Examples of Dynamic Memory Allocation](#)

Note: For a discussion of memory issues, see the chapter on “Memory Considerations for High Performance” in the *Multicore Development Environment Optimization Guide* (UG515).

- [Chapter 4: Memory Homing and Allocation Concepts](#)
- [Chapter 6: Dynamic Distributed Cache Technology](#)

5.1 Using the `tmc_alloc()` API to Allocate Memory Pages

Tilera provides a wrapper API for the traditional `mmap()` and `mbind()` APIs, which allow you to allocate and release pages of memory from the operating system.

The `tmc_alloc()` API handles some aspects of low-level page-based memory allocation that can be challenging. Examples of these are allocating huge (16MB) pages, guaranteeing that mapped memory is present and usable for the application before returning *success*, handling the choice of memory controller and *home cache*, and so forth.

The API is based on a `tmc_alloc_t` data structure, which is configured to specify how memory should be cached, what size pages to use, and so forth. This structure encapsulates all the information about the parameters to use when allocating memory.

The `tmc_alloc()` API provides many functions for allocating memory pages. This section describes some of the most important.

5.1.1 Functions for Allocating and Freeing Memory

The functions for allocating and freeing memory are:

- `void* tmc_alloc_map_at(const tmc_alloc_t *alloc, void *start, size_t length)`
Allocate pages of memory from the operating system sufficient to hold the requested number of bytes at the specified address.
- `void* tmc_alloc_map(const tmc_alloc_t *alloc, size_t length)`
Allocate pages of memory from the operating system sufficient to hold the requested number of bytes.
- `int tmc_alloc_unmap(void *start, size_t length)`
Return the pages holding the requested number of bytes to the operating system.
- `void* tmc_alloc_remap(void *addr, size_t old_len, size_t new_len, int flags, ...)`
Change the length and/or address of a memory mapping from `tmc_alloc_map()` with the given `tmc_alloc_t`.

5.1.2 Functions for Setting and Clearing Shared Memory

For information about shared memory, see the *Multicore Development Environment Optimization Guide* (UG515).

The functions for setting and clearing shared memory are:

- `tmc_alloc_t * tmc_alloc_set_shared(tmc_alloc_t *alloc)`
Set allocated memory to be shared.
- `tmc_alloc_t * tmc_alloc_clear_shared(tmc_alloc_t *alloc)`
Set allocated memory to be private.

5.1.3 Functions for Setting Memory Nodes

By default, Linux boots with a single *striped* virtual memory controller (also known as a NUMA node). It is possible to boot in a way that exposes each of the physical controllers as separate NUMA nodes, as documented in the “Controlling Memory Striping” section of the *Multicore Development Environment System Programmer’s Guide* (UG509).

When separate NUMA nodes are available, Linux associates a *local* node with each core, dividing the cores evenly among the nodes in a way that generally associates each core with its closest NUMA node. By default, Linux allocates standard *small* 64KB pages on the local memory controller, and *huge* (16MB) pages round-robin among the available controllers.

The `tmc_alloc` object can override these defaults by calling the following functions:

- `unsigned long tmc_alloc_get_available_nodes(void)`
Return a mask of the memory controller nodes available for use.
- `tmc_alloc_t * tmc_alloc_set_nodes(tmc_alloc_t *alloc, unsigned long nodes)`
Set the required memory nodes from which to acquire memory.

This function restricts the set of nodes that memory can be allocated from (with the default still being the CPU's local node).

- `tmc_alloc_t * tmc_alloc_set_node_preferred(tmc_alloc_t *alloc, int node)`

Set the preferred memory node from which to acquire memory.

If you use this function, you will tend to get pages allocated just from the specified controller.

If memory is not available from the preferred controller, memory from another controller will be used instead.

- `tmc_alloc_t * tmc_alloc_set_nodes_interleaved(tmc_alloc_t *alloc, unsigned long nodes)`

Set up a mask of memory nodes across which to interleave memory.

This function does strict interleaving of all the specified nodes, at page-size granularity, on consecutive allocated pages.

- `tmc_alloc_t * tmc_alloc_set_node_default(tmc_alloc_t *alloc)`

Set up to use the default strategy for choosing memory nodes.

- `tmc_alloc_t * tmc_alloc_set_node_local(tmc_alloc_t *alloc)`

Use the local memory controller for the allocating CPU for memory.

5.1.4 For More Information About `tmc_alloc()`

For more information, see:

- “Specified-Attribute Memory Page Allocation” section of the “Tilera Multicore Components (TMC)” chapter of the *Application Libraries Reference Manual* (UG527)
- `<tmc/alloc.h>` header

5.2 Using the `tmc_mspace()` API to Allocate Memory Spaces

You can manage memory by means of `malloc`-like operations by using `libc` functions to operate on *mspace* (memory space) objects. (An *mspace* is also known as a heap or arena.) The Tilera implementation of the `tmc_mspace()` API is layered on top of the `tmc_alloc()` API, so that you can customize the type of memory returned by the `tmc_mspace()` API in the same way you can customize memory returned from `tmc_alloc_map()`.

5.2.1 Allocating Separate Memory Spaces

The functions shown below create, destroy, and manage separate areas of memory:

```
#include <tmc/mspace.h>

tmc_mspace tmc_mspace_create_special(size_t capacity, unsigned int flags,
    const tmc_alloc_t *)
tmc_mspace tmc_mspace_create_with_base_special(void *base, size_t capacity,
    unsigned int flags, const tmc_alloc_t *);

tmc_mspace tmc_mspace_create(size_t capacity, unsigned int flags);
tmc_mspace tmc_mspace_create_with_base(void *base, size_t capacity, unsigned int
    flags);
size_t tmc_mspace_destroy(mspace msp);
tmc_mspace (void);
```

```

void* tmc_mspace_malloc(tmc_mspace msp, size_t bytes);
void* tmc_mspace_calloc(tmc_mspace msp, size_t n_elements, size_t elem_size);
void* tmc_mspace_memalign(tmc_mspace msp, size_t alignment, size_t bytes);
struct mallinfo tmc_mspace_mallinfo(tmc_mspace msp);
void tmc_mspace_malloc_statistics(tmc_mspace msp);
void tmc_mspace_statistics(tmc_mspace msp, size_t* maxfp, size_t* fp, size_t*
used);

```

The `tmc_mspace_create()` routine creates and returns a new independent space with the given initial capacity, or, if 0, the default granularity size. It returns `NULL` if there is no system memory available to create the space. The capacity of the space will grow dynamically as needed to service `tmc_mspace_malloc` requests. You can control the sizes of incremental increases of this space by dynamically setting the granularity option, for example:

```
mallopt(M_GRANULARITY, value).
```

5.2.2 Flags Specific to the `tmc_mspace()` API

The `flags` parameter is a bitfield. Several flags specific to the `tmc_mspace()` API are defined:

`TMC_MSPACE_LOCKED`

The space uses a separate lock to control access.

You should normally set this flag any time multiple threads or processes might allocate or free from the same `mspace` at the same time. Typically, this means that you should set this flag in any Pthreads program, any `cmem`-using `tmc_alloc_t` for `tmc_mspace_create()` or `tmc_mspace_create_special()`, and any `mspace` constructed using any form of shared memory. It is always set for the shared memory `mspaces` managed by the `tmc_cmem` functions (such as `tmc_cmem_malloc`).

Note: When specifying `TMC_MSPACE_LOCKED`, in order for the locking to work correctly across tiles, you cannot use a `tmc_alloc_t` with `ALLOC_HOME_INCOHERENT`.

`TMC_MSPACE_NOGROW`

The space will not allocate additional memory above and beyond the initial allocation.

With `tmc_mspace_create()` and `tmc_mspace_create_with_base()`, the initial allocation is the `capacity` argument (or the default granularity if 0, typically 64KB). With `tmc_mspace_create_special()` and `tmc_mspace_create_with_base_special()`, the initial allocation is passed explicitly, and no further allocations will be performed.

This flag is most useful for `tmc_mspace_create_with_base()` for hand-mapped memory.

5.2.3 Setting Allocation Attributes in `mspaces`

The `tmc_mspace()` API provides generic memory, typically locally-cached or hash-for-home, depending on the default settings for the process. To create an `mspace` that allocates memory of a particular type, use `tmc_mspace_create_special()`. This function is just like `tmc_mspace_create()`, but it takes an additional third argument that is a pointer to an `tmc_alloc_t` object. The `tmc_alloc_t` specifies the allocation attributes of the memory that will be returned by the `tmc_mspace` API calls.

For example, if you want to create a heap that used *huge pages*, you might specify:

```
tmc_alloc_t attr = ALLOC_INIT;
tmc_alloc_set_huge(&attr);
tmc_mspace msp = tmc_mspace_create_special(0, 0, &attr);
```

5.2.4 Functions for Multiple Heap Allocation

Some of the functions in the `tmc_mspace()` API are:

- `tmc_mspace_create()` creates and returns a new independent space with a specified initial capacity or the default granularity size.

In general, you should use this function if you want to allocate a heap with default memory allocation.

- `tmc_mspace_create_special()` creates a heap just like `tmc_mspace_create()` does with a `tmc_alloc_t` allocator initialized with `TMC_ALLOC_INIT` passed as the third argument. This lets you allocate memory of a particular type.

This function allows application code to choose an allocation strategy for a heap, including huge page support as well as memory homing and memory controller selection.

- `tmc_mspace_create_with_base()` uses the memory supplied as the initial base of a new space.

In general, you should use this function if you need to provide a preallocated region of memory to the allocator.

- `tmc_mspace_create_with_base_special()` creates a heap using the memory supplied as the initial base of a new space, and allocates additional memory of the type specified by the passed `tmc_alloc_t` object.

In general, you should use this function if you want to control the allocation of the additional memory that you created with `tmc_mspace_create_with_base()`.

- `tmc_mspace_destroy()` destroys the given space and attempts to return all of its memory back to the system, returning the total number of bytes freed.

After destruction, the results of access to all memory used by the space become undefined.

- `tmc_mspace_malloc()`, `tmc_mspace_calloc()`, and `tmc_mspace_memalign()` behave as `malloc()`, `calloc()`, and `memalign()`, respectively, but operate within the given space. Allocated memory can be freed with `tmc_mspace_free()` or reallocated with `tmc_mspace_realloc()`.

Note that memory allocated with the `tmc_mspace` routines cannot be freed with the `libc` `free()` API, and cannot be reallocated with the `libc` `realloc()` API.

- `tmc_mspace_mallinfo()` and `tmc_mspace_malloc_statistics()` behave as `mallinfo()` and `malloc_stats()`, respectively, but report properties of the given space.

5.2.5 Using the `tmc_malloc` Library to Support `malloc()` by Means of `tmc_mspace`

The `tmc_malloc` library allows you to mix and match the TMC `mspace` library allocation routines with the standard `malloc()` and `free()` API routines. The usual `libc` routines are not compatible with TMC's `mspace` routines, so you need to be careful which API to use for any given object.

The `tmc_malloc` library overrides the `malloc()` routine in the `libc` library to make it use the TMC allocator, so objects can be freed and reallocated without concern for their original source.

For more information, see the section on “Using mspaces for Standard malloc/free” in the *Application Libraries Reference Manual* (UG527).

5.2.6 For More Information About `tmc_mspace()`

For more information, see:

- “Multiple Heap Allocation” section of the “Tilera Multicore Components (TMC)” chapter of the *Application Libraries Reference Manual* (UG527)
- `<tmc/mspace.h>` header

5.3 Using the `tmc_cmem()` API to Access Common Memory

The `tmc_cmem()` API provides access to common memory, which is shared memory that is mapped on all participating processes at the same address.

See [Section 4.7 “Allocating Shared Memory” on page 30](#) for background information on shared memory and this API.

The `tmc_cmem()` API is a method for collaborating processes to have thread-like memory access and allocation. Any participating process can call `tmc_cmem_malloc()` or other `tmc_cmem` allocation APIs and all the processes will immediately have access to the allocated memory, at the same address.

All the participating processes must share some common ancestor that created the common memory region using `tmc_cmem_init()`, and must themselves have called `tmc_cmem_init()`.

To allocate common memory at the page level, you can create a `tmc_alloc_t` object initialized to `TMC_CMEM_ALLOC_INIT`, or you can dynamically initialize a `tmc_alloc_t` object by calling `tmc_cmem_init()`. After doing this, you can call `tmc_alloc_map()` with a pointer to that `tmc_alloc_t` to allocate common memory.

Assuming that a process has called `tmc_cmem_init()`, it can allocate the common memory region as follows:

```
tmc_alloc_t alloc = TMC_CMEM_ALLOC_INIT;
tmc_alloc_set_home(&alloc, TMC_ALLOC_HOME_HERE);
void *p = tmc_alloc_map(&alloc, getpagesize())
```

The `tmc_cmem()` API also includes a `malloc` API so processes can easily allocate and free common memory. For example, a process can allocate a few bytes of common memory from the default `mspace` as follows:

```
thing_t* thing = tmc_cmem_malloc(sizeof(*thing));
```

Additional `mspaces` can be set up by passing a “common memory” `tmc_alloc_t` object to `tmc_mspace_create_special()`, as discussed in [Section 4.6 “Multiple Heap Allocation \(mspaces\)” on page 30](#).

The `tmc_cmem()` API also provides one default common-memory pool (using an `mspace`) to simplify allocating and freeing common memory:


```
#include <tmc/cmем.h>

void *tmc_cmем_malloc(size_t nbytes);
void *tmc_cmем_memalign(size_t align, size_t nbytes);
void *tmc_cmем_calloc(size_t n, size_t s);
```

These functions allow processes to allocate cmем memory. Each process gets its own shared memory mspace, cached on the tile running that process. Each process can call `tmc_cmем_malloc()` (or the other similar functions) to allocate memory from its shared memory mspace, and all the processes in the cmем group can then reference the memory that process has allocated. Any process can `free()` the memory back into the shared memory mspace of the process that allocated it.

The special mspace used by cmем uses `TMC_MSPACE_LOCKED`.

`tmc_cmем_malloc()`, `tmc_cmем_calloc()`, `tmc_cmем_realloc()`, and `tmc_cmем_memalign()` are thin wrappers around `tmc_mspace_malloc()`, `tmc_mspace_calloc()`, `tmc_mspace_realloc()`, and `tmc_mspace_memalign()`, using this common mspace.

As for other mspace-allocated memory, you can use the `tmc_mspace_free()` and `tmc_mspace_realloc()` functions to return or resize blocks of shared memory. `tmc_mspace_realloc()` will reallocate within the shared pool.

All accesses to the shared pools are locked so that it is safe for multiple processes to request, free, and resize memory at once.

The `TMC_ALLOC_HOME_TASK` setting for `tmc_alloc_set_home()` is not available for cmем memory, because this setting only works for memory mapped into a single process, and cmем by definition is intended to be used by multiple processes.

5.3.1 More Information About `tmc_cmем()`

For more information, see:

- “Common Memory” section of the “Tilera Multicore Components (TMC)” chapter of the *Application Libraries Reference Manual* (UG527)
- `tmc/cmем/simple.c` example in the “Example Documentation” in the guide
- `<tmc/cmем.h>` header

5.4 Memory-Mapping Flags to Control Caching

There are a number of additional Tilera-specific flags that work with any `mmap()` call. You can use them with the `tmc_alloc()` API to modify how the system caches the returned, allocated memory.

5.4.1 Flags for Setting the Home Cache

The *home cache* is the cache on a particular *home* processor, which proxies all cache line traffic to memory and is responsible for invalidation operations. The home cache is normally set by the operating system. Linux by default will set the home cache for a given page to be hash-for-home, that is, with the home cache for each cache line distributed across the available cores. This can be changed by the `ucache_hash` boot option or the `LD_CACHE_HASH` environment variable. See [Section 6.2.2 “Per-Application Cache Defaults” on page 67](#).

The following flags allow an application to specify where the home cache for pages will be located:

```
MAP_CACHE_HOME(n)
```

Specifies a specific processor's cache to be the home cache for the mapping. The value of *n* should correspond to one of the valid processors available to Linux. See `/proc/cpuinfo` for the list of valid processors on a given chip configuration.

`MAP_CACHE_HOME_HASH`

Allocate memory with the `hash-for-home` attribute set.

`MAP_CACHE_HOME_HERE`

Shorthand for specifying that the home cache should be set to that of the processor calling `mmap()`.

`MAP_CACHE_HOME_TASK`

Specify that the home cache of the allocated memory will follow the thread or process that called `mmap()` if it migrates to a new processor. Used by threads that need to allocate an mspace whose home cache will always be the cache of the tile executing that thread. This is the default behavior for each thread's stack.

`MAP_CACHE_HOME_SINGLE`

Requests that the OS provide some single home cache for the mapping, but leaves the issue of which CPU to use up to the OS. This overrides the default `hash-for-home` behavior without forcing any specific processor's cache to be the home cache.

`MAP_CACHE_HOME_NONE`

Tells the OS not to use a home cache at all. For a read-only mapping, this tells each processor to fetch cache lines directly from RAM. If the local cache is enabled, as it is by default for read-only mappings, data read from RAM will be cached in the usual way in the local processor's cache.

5.4.2 Flags for Disabling Local Caching

By default, the OS always tries to use the on-chip caches to store data. Sometimes this is not possible for the local per-processor cache. You can specify these flags to disable use of the on-chip caches:

`MAP_CACHE_NO_L1`

Allocate memory but do not allow use of the local L1 cache.

`MAP_CACHE_NO_L2`

Allocate memory but do not allow use of the local L2 cache.

`MAP_CACHE_NO_LOCAL`

Allocate memory but do not allow use of any local cache. The resulting mappings will only be cached in the home cache, if any.

The combination of `MAP_CACHE_NO_L1` and `MAP_CACHE_NO_L2` is equivalent to `MAP_CACHE_NO_LOCAL`. Use of both the local L1 cache and the local L2 cache is suppressed.

`MAP_CACHE_NONE`

Disables use of all caches, so reads and writes go directly to RAM.

The combination of `MAP_CACHE_NO_LOCAL` and `MAP_CACHE_HOME_NONE` (see [Section 5.4.1 “Flags for Setting the Home Cache” on page 39](#)) is equivalent to `MAP_CACHE_NONE`.

Note: Even when disabling the L2 cache and the L1 data cache (L1-Dcache) with these flags, the instruction cache (L1-Icache) is always enabled.

5.4.3 Flags for Requesting User-Managed Shared Memory

All of the modes described in [Section 5.4.1 “Flags for Setting the Home Cache” on page 39](#) and [Section 5.4.2 “Flags for Disabling Local Caching” on page 40](#) preserve *cache coherency*. Cache coherency enables processors to read and write memory at the same time safely (assuming appropriate use of locking). It is possible with the TILE architecture to disable cache coherency, which can lead to improved memory performance under some circumstances. When this is done, the application must carefully flush data from its local cache back to RAM after stores, and/or invalidate data in its local cache prior to loads. Allowing applications to have this very fine-grain control over the use of the memory network can be helpful for performance, though it can also be challenging to use it correctly.

To enable this mode for a read-write mapping, you must specify the `MAP_CACHE_INCOHERENT` flag as one of the memory-mapping flags. The definition of this flag includes `MAP_CACHE_HOME_NONE`, so that cache lines transferred to and from the local cache go directly to RAM.

Synchronization primitives based on the test-and-set instruction TNS do not work correctly with user-managed shared memory, because the memory is not coherent. For example, attempting to build a spinlock (mutex) primitive from this type of memory fails because every tile gets its own cached copy of the lock; therefore, every tile could take and release the lock without synchronizing with the others. Thus, TNS should only be used with coherent shared memory and never with user-managed shared memory.

You can use mspaces for user-managed shared memory. However, all `tmc_mspace_malloc()`, `tmc_mspace_free()`, `tmc_mspace_realloc()` and so forth calls must be performed on the tile where the mspace is created, and only the actual data in the allocated memory can be shared.

5.4.4 Flags for Sharing Mappings Without the `tmc_cmem()` API

For sharing memory between applications, the `tmc_cmem()` API provides the simplest API. However, you can also do the same kind of sharing manually. This can be desirable if the size of memory to be shared is fixed at compile time, or if the memory should be persistent from one invocation of the application to another.

Multiprocess applications that do not use `tmc_cmem` should instead `mmap()` a fixed-size region of memory and use that region for shared allocation. Depending on how the processes are related, two approaches can be used:

- If the processes are all simply `fork()`ed from a single parent, the parent can `mmap()` memory with the `MAP_ANONYMOUS` and `MAP_SHARED` flags, and all the child processes will share it.
- If the processes are started separately, or call `exec()` after `fork()`, you will lose any pre-existing shared mappings. You should use the first process to create a suitable file to serve as *backing store* for the memory (typically using `open()` and `ftruncate()` to set the size), then `mmap()` the file into memory specifying a known base address for the mapping, then initialize the contents of the memory region. The child processes can then `mmap()` the file in to the same base address.

File mappings for the second approach should likely be on a RAM file system such as `tmpfs`, to ensure that the OS does not spend time writing dirty mspace pages out to disk. On the other hand, if a persistent mspace is desired (to support multiple runs of an application), a “real” file system can be an appropriate choice.

When specifying a known base address for the mapping, it is important to choose a region of the address space that does not conflict with other normal Linux mappings, such as the executable itself or the program stack. Such non-conflicting addresses are not guaranteed to remain the same from release to release. Currently, however, a good address to place fixed shared mappings is just above the end of the loaded executable area. For example, you might choose to place such mappings at address `0x40000000`, leaving the bottom 1GB of address space for loaded code and data.

5.5 Using Hash-for-Home

The hash-for-home memory homing strategy allows software to define a set of CPUs to be used as a distributed L3 cache. Hash-for home is described in [Section 4.4 “Hash-For-Home Strategy” on page 28](#). Also see [Chapter 4: Memory Homing and Allocation Concepts](#).

By default, hash-for-home is used for all dynamically allocated memory.

The hash-for-home strategy is one way to configure the system. This configuration is done per page of memory (64KB typically, or 16MB for huge pages). For hash-for-home, the hardware uses a simple algorithm to round-robin among all the specified hash-for-home tiles, and each cache line on that page is assigned a different one of the tiles as its home.

You can also configure a given page to be homed in its entirety on a single specified tile. This is done, for example, with the stack page, because it is almost entirely used by the task (process or thread) that is running on a given tile. Thus, that same tile should be the *home* for all the data on each stack page associated with that task.

The set of CPUs to be used by hash-for-home memory is defined by the `hfh_tiles` subcommand in the `client` command in the hypervisor configuration file. By default, the hypervisor fills the hash-for-home CPU set with all tiles that are not running dedicated-tile hypervisor drivers. For more information, see the “Hypervisor Customization” chapter in the *Multicore Development Environment System Programmer’s Guide* (UG509).

5.5.1 Hash-for-Home in Applications

The `LD_CACHE_HASH` environment variable controls the cache homing of application memory. This mechanism allows independent control of the default mapping mode for different types of application data. In particular, it controls the default mapping mode for the following categories of application memory:

- Program text and read-only data
- Static, writable data — the program’s `.data` and `.bss` sections
- Stack
- Heap memory — any memory allocated by means of `malloc()`, `mspaces`, or `alloc_map()`, but not `mmap()`

There are five possible values for `LD_CACHE_HASH`. Each value enables hash-for-home for a particular permutation of data types; data which is not hash-for-homed is *locally homed* and homed on whatever CPU is running the task.

- `all` — hash-for-home everything
- `allbutstack` — leaves the stack(s) locally-cached and hashes everything else
- `static` — hash-for-home for loaded sections (`.text`, `.data`, `.bss`) only
- `ro` — only hash program text and read-only data
- `none` — do not use hash-for-home for anything

The kernel provides `allbutstack` semantics by default if `LD_CACHE_HASH` is not set. To change the default when `LD_CACHE_HASH` is not set, use the `ucache_hash` boot option; for example: `ucache_hash=ro`.

5.5.2 Hash-for-Home in Linux

The Linux kernel accepts a `kcache_hash` boot argument that specifies how the kernel should home its memory. This argument accepts the same values as `LD_CACHE_HASH` (`all`, `allbutstack`, `static`, `ro`, `none`). By default, the kernel runs in `allbutstack` mode.

5.5.3 Recommendations for Hash-for-Home Settings

The optimal choice of hash-for-home settings depends on an application's shared memory access patterns. We recommend the following settings for particular categories of applications:

- Large shared memory programs with random memory access patterns should use the default `allbutstack` settings, thus using hash-for-home for everything except user and kernel stacks. This is the most appropriate setting for large, pthreaded applications.
- To optimize for mostly single-threaded processes with little or no shared memory access, set `LD_CACHE_HASH` to `ro`. This leaves the static data and heap cached on the tile running each individual process, rather than randomly homing the data on other tiles and competing for cache capacity that is best used by the process(es) running on those tiles.

To set up tiles 0–25 as hash-for-home reserved for use of a particular application, add these settings to the hypervisor configuration file under the `client` command line: Add `hfh_tiles 0-25` and then add `dataplane=0-25 LD_CACHE_HASH=ucache_hash kcache_hash=none kdata=26,29-34` to the `args` line to arrange for no default use of hash-for-home and to cache kernel data on a few non-dataplane tiles. Then, either run the specific application with `LD_CACHE_HASH=allbutstack`, or use the `MAP_CACHE_HOME_HASH` flag when allocating memory, as appropriate. This guarantees that only that application homes data using hash-for-home.

Instead of setting `LD_CACHE_HASH` for individual applications, you might want to simply add the `ucache_hash` option to the kernel's boot command line; for example, `ucache_hash=ro`. This will change the system defaults without the requirement of passing an environment variable down to every application that might need it.

5.6 Understanding Virtual Memory and Page Size

The hardware converts program addresses (*virtual addresses*) into the corresponding offset in the RAM (*physical addresses*). This conversion allows the operating system to offer each program a range of addresses starting at zero, while actually keeping each program's data at different places in memory. The hardware makes this conversion in blocks of memory of fixed size, but the size can be chosen for each virtual address. The sizes supported by the hardware start at 16KB and go up in powers of four (64KB, 256KB, 1MB, etc.) up to a maximum of 64GB. For a list of the permissible page sizes, see [Table 5-1](#).

5.7 Configuring Memory Page Sizes under Linux

Linux on TILE-Gx supports a variety of memory page sizes, and provides the usual boot-time options for declaring and allocating them. These options are described below.

5.7.1 Page Tables and Page Sizes

The Linux OS manages memory via a three-level page table mechanism. The span of an entry at each of these levels determines a natural page size that can be used to allocate and refer to memory regions.

On TILE-Gx the span of entries at each of these levels is 64K (small), 16M (huge), and 4G (jumbo). These spans are used to determine the allowable page sizes on TILE-Gx.

5.7.1.1 Small Pages

The normal, default page size in Linux is the *small* page size. In Linux on TILE-Gx this is 64K, though one can optionally lower the small page size to 16K by doing a custom Linux build and setting the `CONFIG_PAGE_SIZE_16KB` option in the kernel configuration. Small pages avoid fragmentation (e.g. small files take only 16K of memory in the page cache) but can decrease performance due to more pressure on the TLB when doing memory accesses.

5.7.1.2 Huge Pages

Linux also supports allocating memory in larger page sizes, known collectively as *huge pages*. At boot time you may request one or more huge page sizes, using the `hugepagesz` boot option, subject to certain constraints, which are covered in the following sections.

5.7.1.3 Default Huge Page Size

One huge page size is treated as the default size for huge pages by Linux. This is normally the 16M page size, but you can set the default page size using the `default_hugepagesz=` boot option. This is the page size associated with the various huge page flags and information—for instance, the `mmap()` `MAP_HUGETLB` flag, the `shmget()` `SHM_HUGETLB` flag, and the information reported in `/proc/meminfo`.

5.7.2 Configuring Huge Page Sizes

Configuring huge pages is not difficult but there is some implicit defaulting involved that you need to keep in mind. Read this section completely before using these options.

To request a huge page size, use the `hugepagesz=size` boot time option, where *size* is a number followed by a unit letter (k/K, m/M, or g/G).

For example, to request a 16GB page pool, specify:

```
hugepagesz=16g
```

You can request more than one huge page size, subject to the following limits:

- a. You can request a given page size at most once
- b. You can request at most one pool for each of the 16MB and 4GB base page spans

Note: The 64KB span is already available as the *small* page size.

- c. You can request at most one additional power-of-4 multiple for each of the base page spans (64KB, 16MB, and 4GB).

The valid huge page sizes on TILE-Gx are shown in [Table 5-1](#).

If you specify no `hugepagesz` options, then Linux automatically declares a default huge page size of 16MB, and one additional huge page size of 1MB.

If you specify one or more `hugepagesz` options, then only those huge page sizes are declared, except that the default huge page size is still 16MB; if you do not explicitly specify 16MB as a huge page size, Linux configures the 16MB page size automatically, prior to any of the sizes you have explicitly requested. Specify a different default huge page size by using the boot option `default_hugepagesz=size`.

For example, to declare a huge page size of 4M, and also set it as the default, use the following options:

```
hugepagesz=4m default_hugepagesz=4m
```

The `default_hugepagesz` option must refer to an existing huge page size; this can be either an explicit `hugepagesz` option on the command line or, if you supply no `hugepagesz` options, Linux declares one of the automatically declared page sizes (1MB or 16MB).

For example, specify the option `default_hugepagesz=1m`, and Linux allocates only the 1M huge page size, since 1MB is now the default huge page size as well.

If you specify only the option `default_hugepagesz=16m`, this is essentially the same as if you had not specified any options (hugepage or default size) `default_hugepagesz` setting, since 16MB is already the default hugepage size, and Linux declares the additional 1M page size as well.

As an extreme example, you could simultaneously request five different huge page sizes:

- 256KB pages (4 times the 64KB page size)
- 16MB pages
- 256MB pages (16 times the 16MB page size)
- 4GB pages
- 16GB pages (4 times the 4GB page size)

To accomplish this allocation, specify the following boot options:

```
hugepagesz=256K hugepagesz=256M hugepagesz=4G hugepagesz=16G
```

The default huge page size (16MB) is automatically declared. If you need a different default huge page size, you must explicitly declare the 16M size, as in this example:

```
hugepagesz=256K hugepagesz=16M hugepagesz=256M \
hugepagesz=4G hugepagesz=16G default_hugepagesz=256M
```

You cannot simultaneously request 256M and 1G as huge page sizes, since both of these sizes are managed as multiples of the 16M base page size.

Note: If you specify an invalid `hugepagesz` value, or specify a `default_hugepagesz` value that doesn't refer to a previously declared huge page size, Linux silently ignores these options. Hence, if you request a page size other than the default huge page sizes, verify that you actually got the sizes and defaulting that you wanted. Some ways of doing this are discussed later in this chapter.

Table 5-1 shows the memory page sizes available to you in Linux on TILE-Gx:

Table 5-1. Page Sizes Allowed in Tilera Linux

Page Size	Boot Option	Description
256KB	<code>hugepagesz=256K</code>	4 of the 64KB pages
1MB	<code>hugepagesz=1M</code>	16 of the 64KB pages (the default multiple of 64KB pages)
4MB	<code>hugepagesz=4M</code>	64 of the 64KB pages
16MB	<code>hugepagesz=16M</code>	The default huge page size
64MB	<code>hugepagesz=64M</code>	4 of the 16MB pages
256MB	<code>hugepagesz=256M</code>	16 of the 16MB pages
1GB	<code>hugepagesz=1G</code>	64 of the 16M pages
4GB	<code>hugepagesz=4G</code>	The default “jumbo” page size
16GB	<code>hugepagesz=16G</code>	4 of the 4GB pages
64GB	<code>hugepagesz=64G</code>	16 of the 4GB pages

5.7.3 Allocating Huge Pages

By default, the Tilera Linux kernel does not pre-allocate any huge pages when it boots. For each huge page size, so long as it is 16MB or less, Linux can be configured to automatically allocate pages of that size on request (up to the limit of free memory) by setting the `sysctl` value `vm.nr_overcommit_hugepages` to a large value.

If the default huge page size is 16MB or less, this configuration is done automatically for you by the Tilera startup scripts, so if you are only using the default huge page size, you do not have to take other any special action at boot time; the kernel allocates such pages on demand.

While pre-allocating huge pages is not required for page sizes of 16MB or less, it can be helpful in order to get the maximum possible number of such pages, since after the system has been up for a while, fragmentation of memory makes it harder for the kernel to find contiguous unused regions of physical memory.

Huge page sizes greater than 16MB cannot be automatically allocated, and you must request that such pages be pre-allocated at boot time. The boot time option for requesting huge page pre-allocation is `hugepages=count`, where `count` is the number of pages to allocate.

The `hugepages` option may appear multiple times on the command line; it always refers to the most-recently-specified page size. So for example, to allocate two different page sizes, each with its own count, you might use the following options:

```
hugepagesz=4G hugepages=10 hugepagesz=16G hugepages=3
```

If you use the `hugepages` option prior to any explicit `hugepagesz` options, for example:

```
hugepages=100
```


Linux sets the number of pages to be pre-allocated for the automatically-declared default page size (16M size), since in this case there is no alternate `default_hugepagesz` setting.

5.7.4 Allocating Huge Pages on Specific Memory Controllers

You can request huge pages allocated to specific NUMA nodes by specifying `hugepages=x, y, z` to request the specified number of pages on each NUMA node, i.e. `x` pages on node 0, `y` on node 1, and `z` on node 2.

Note: By default the operating system stack stripes RAM at a low level across the various NUMA nodes, so there is no benefit by default in specifying the NUMA node. However, you can boot with `tile-monitor --hvd STRIPE_MEMORY=never` and the hypervisor does not stripe memory, in which case being able to assign huge pages to individual memory controllers can be helpful.

By default, Linux uses a small amount of memory at the beginning and end of each memory controller for book-keeping (the page array at the front, and kernel per-CPU data at the end). This means that it is not possible by default to allocate huge pages at the very beginning and very end of each controller. To allow allocating all the memory on a controller as huge pages, specify the `isolnodes` argument on the kernel boot command line. For example, to allow allocating all of the memory on controller 1 as huge pages, you would specify `isolnodes=1`. The argument is a comma-separated list of NUMA node numbers (or of hyphen-separated ranges of NUMA node numbers).

Additionally, since the Linux kernel itself is located at the beginning of controller zero, and the hypervisor is located at the end of controller zero, you cannot allocate huge pages at the very beginning or end of that controller.

5.7.5 Determining Current Huge Page Configuration

On Linux, there are a number of `/proc` and `/sys` files that provide information about the huge page state.

First, the classic `/proc/meminfo` file provides information on default size huge pages. Booting with `hugepages=20` might show the following before allocating any huge pages to an application:

```
# cat /proc/meminfo
[...]  
HugePages_Total:      20  
HugePages_Free:       20  
HugePages_Rsvd:       0  
HugePages_Surp:       0  
Hugepagesize:        16384 kB
```

In addition, the `/sys/kernel/mm/hugepages/*` directory hierarchy offers information about all the different huge page sizes. For example, booting with the Linux kernel configuration options `hugepagesz=1g hugepages=4` configures the 1GB page size and allocates four such pages, in addition to the default huge page size. In this case, the `/sys/kernel/mm/hugepages/` directory includes the following subdirectories:

```
# ls /sys/kernel/mm/hugepages/  
hugepages-1048576kB hugepages-16384kB
```

In each directory is an `nr_hugepages` file that holds a single number representing that value. To view it, use the following command:

```
# cat /sys/kernel/mm/hugepages/hugepages-1048576kB/nr_hugepages  
4
```

As an alternative, use the `hugeadm` tool to discover information about the huge page status of the system. For example, the command `hugeadm --pool-list` provides information about each configured pool of huge pages, and the command `hugeadm --page-sizes` lists all supported page sizes. For more information, see the `hugeadm(8)` man page.

5.7.6 Using Huge Pages Programmatically in Generic Linux

You can allocate huge pages through a few standard (cross-platform) techniques:

- Using the `mmap()` routine with the `MAP_ANONYMOUS | MAP_HUGETLB` flags is the most straightforward generic way to allocate huge pages. The `MAP_HUGETLB` flag was first supported in Linux kernel version 2.26.32.
- Using the `SHM_HUGETLB` flag with the `shmget()` routine allocate huge pages. The `SHM_HUGETLB` flag was first supported in Linux kernel version 2.6.0.
- You can also use the `hugetlbfs` file system for allocating huge pages, and importantly, for sharing them between unrelated processes. The system must be configured with suitable `hugetlbfs` mount-points for this approach to work. By default, Tiler Linux is configured with a `/dev/hugepages/` mount point for default size huge pages, and with a series of `/dev/hugetlbfs/pagesize-*` directories corresponding to all the available huge page sizes.

By default, these directories are only writable by `root`. In order for non-`root` processes to allocate huge pages via the filesystem, you must change the directory permissions. Run `chmod` and `chgrp` commands as `root` on a booted system (either interactively, or from, for example, `/etc/rc.local`). Or, additional mount points can be created with new instances of the `hugetlbfs` file system, configured as desired. See the "Documentation/vm/hugetlbpage.txt" file in the kernel sources for more details of mounting and using `hugetlbfs` files systems.

Use the `hugeadm` tool to create and manipulate huge page mount points and system options. See the `hugeadm(8)` man page.

5.7.7 Allocating Huge Pages using `tmc/alloc.h`

The `<tmc/alloc.h>` header provides an API for allocating memory, including huge-page memory.

You can pass a `tmc_alloc_t` object (an *allocator*) to the `tmc_alloc_set_huge()` routine to enable allocating default-size huge pages.

Use the `tmc_alloc_clear_huge()` routine to disable the allocator. Use the `tmc_alloc_get_huge()` routine to query the allocator.

The `tmc_alloc_get_huge_pagesize()` method returns the size in bytes of the default huge page size.

For non-default size pages, the `tmc_alloc_set_pagesize_exact()` method requests allocating the specified page size. If that page size is not configured in the kernel, the routine returns `NULL`. Alternately, if userspace simply needs to request that its allocations are contiguous in physical memory in chunks of a particular size, use the `tmc_alloc_set_pagesize()` routine, to set the allocator to use page sizes of at least as big as the argument. The routine returns `NULL` to signal that multiple pages would be required, and the allocator is set to use the largest possible pages in that case. The `tmc_alloc_get_pagesize()` method queries an allocator as to what size pages it will return when you use it.

The `tmc_alloc_get_pagesizes()` routine returns a bit-mask of the available sizes so a program can discover what pages are (in principle) available to it. When you call this routine, there is no guarantee that pages are actually available.

The `tmc_alloc_set_huge()` method configures the memory allocator to allocate default-size huge memory pages, which are typically 16MB in size; you can set the default huge memory page size using the `default_hugepagesz=` kernel boot option.

For more information about these APIs, see the *Application Libraries Reference Manual* (UG527).

5.7.7.1 Using TILE-Gx APIs Which Require 64KB Pages

Some of the TILE-Gx APIs require pages of 64KB size, such as the `gxio_mpipe_register_page()` routine. For these APIs, if you are using a kernel built with 16KB small pages, you must allocate a huge page to pass to those APIs. Use the `hugepagesz=64k` argument on the boot command line to have the kernel can allocate pages of appropriate size without excessive waste. To have the kernel choose the smallest page that is big enough to give 64K of physically-contiguous memory, use this call:

```
tmc_alloc_set_pagesize(&alloc, 64*1024)
```

5.7.8 Allocating Huge Pages using hugetlbfs

Use the `hugetlbfs` library to explicitly allocate huge pages. If you include `<hugetlbfs.h>` and link with `-lugetlbfs`, your application can:

- Allocate huge pages using the `get_huge_pages()` routine
- Free huge pages using the `free_huge_pages()` routine
- Allocate entire regions of huge page memory using the `get_hugepage_region()` routine
- Free up those huge page regions from memory using the `free_hugepage_region()` routine

Only a single size of huge pages are provided via these APIs. Further information is available in the man pages.

5.7.9 Using malloc() to Allocate Huge Pages

By default, the standard C library only uses small pages for heap allocation. There are two ways to make `malloc()` return huge pages:

- You can link or preload the `hugetlbfs` shared object into the executable, using the `hugectl` program front-end. For example, to run program `/app` but request that huge pages be used for the heap, use the command:

```
hugectl --heap /app
```

Under the covers, the `hugectl` command just sets environment variables that preload the `libhugetlbfs.so` file and request the desired hugepage semantics. For more information, see the `hugectl(8)` man page for more information.

You can also override the `shmget()` API using `hugectl --shm`. Use the `--heap=PAGE-SIZE` argument to set a particular page size for `libhugetlbfs`. Specify the `PAGESIZE` value in bytes, plus an optional K, M, or G suffix to indicate, respectively, that the value is in kilobytes, megabytes or gigabytes.

- You can link or preload the TMC `malloc` library into the program. This library transparently converts libc `malloc` calls into calls to per-thread default TMC `mspace` objects.

If you set the `$HUGETLB_MORECORE` variable in the environment, and `libtmc_malloc.so` is present, then the `malloc()` routine gives you huge pages as well.

Experiment with both approach to see which works better in your application.

5.7.10 Basing Program Text, Data, and BSS on Huge Pages

The `libhugetlbfs` library supports the basing of program text, data, and BSS in huge pages for dynamically-linked executables. To link your program with support for placing program text (code), data, and bss on huge pages, use the following GCC command:

```
gcc -B/usr/share/libhugetlbfs -Wl,--hugetlbfs-align foo.o ...
```

If you are cross-compiling (and thus have set `$TILERA_ROOT` in the environment and are running `tile-gcc`) you must specify the first argument as `-B$TILERA_ROOT/tile/usr/share/libhugetlbfs`. These arguments invoke an `ld` wrapper that passes `page_size` arguments to the linker to tell the linker to align the program segments on 16MB boundaries, and includes `add -lhugetlbfs` to the linker arguments.

After you run `tile-gcc`, set environment variables to tell `libhugetlbfs` to replace the normal memory pages used by the kernel to load the binary with huge pages. To run application `/app` with huge pages for text or for data/bss, you can do:

```
HUGETLB_ELFMAP=R /app      # text (code)
HUGETLB_ELFMAP=W /app      # data/bss
HUGETLB_ELFMAP=R:W /app    # both
HUGETLB_ELFMAP=no /app     # neither
```

You may specify an optional `=PAGESIZE` value after the `R` or `W` to use a different huge page size. Specify `PAGESIZE` in bytes, plus an optional `K`, `M`, or `G` suffix.

You can also use the `hugectl` program to do this. For example, specify `hugectl --no-pre-load --text /app` uses huge pages for the text. For more information, see the `hugectl(8)` man page.

If you wish to run multiple copies of a program with huge pages, set the `HUGETLB_SHARE=1` environment variable, to allow multiple programs to reuse the same huge pages for copies of program text and data.

To make the use of huge pages for text or data persistent, run the `hugeedit` program. The command `hugeedit --text /app` backs text segments with huge pages by default, add using the `--data` specifies the data segments. You may also run `hugeedit --disable /app` to remove all huge page markings. A `tile-hugeedit` binary exists for cross-compilation scenarios. For more information, see the `hugeedit(8)` man page.

The `/usr/share/doc/libhugetlbfs/HOWTO` documentation file contains more detailed information, along with the `libhugetlbfs(7)` man page.

5.7.11 Caveats for Using Huge Pages

Some functionality was supported in the MDE 2.x release and is not currently supported. These include the following issues:

There is no way to programmatically indicate that a program would like huge pages for the heap. Older releases supported a `MALLOC_USE_HUGE_PAGES(1)` declaration that could be placed at file scope. There is no equivalent support in current releases of the MDE.

Placing program text or data on huge pages does not work for statically-linked binaries at this time.

Placing the program stack on a huge page (typically the same huge page used for the data/bss) can not be requested at this time.

5.8 Allocating Too Much Memory

The system responds in various ways when application programs attempt to allocate too much memory. If you use the TMC library's `tmc_mspace` or `tmc_alloc_map()` APIs, you will always receive a `NULL` return value if the system is out of memory.

As discussed in [Section 5.7.3 “Allocating Huge Pages” on page 46](#), you can request that these APIs return huge pages in various ways. When all the free huge pages in the system pool have been used, the `tmc_mspace` and `tmc_alloc_map()` APIs return `NULL` if you attempt to allocate more. In generic code, when an allocation failure occurs, you might want to check the relevant `tmc_alloc_t` by means of `tmc_alloc_get_huge()` and modify any error message appropriately. Note that the `tmc_mspace_allocator()` API returns the `tmc_alloc_t` for a `tmc_mspace` object.

It is possible for some memory allocation requests to succeed but later cause a `SIGBUS` if enough memory is not available to satisfy page faults. As discussed above, this is not possible for the `tmc_mspace` APIs, or for the default `tmc_alloc_map()`, because the default `tmc_alloc_t` used by these APIs sets the `MAP_POPULATE` flag in the `mmap_flags` field. This flag makes the library code validate that newly memory-mapped memory is in fact backed by RAM; if not, the system sends a `SIGBUS` to the library, which it silently traps and converts to a `NULL` return. You might elect to omit the `MAP_POPULATE` flag if you are creating your own special-purpose `tmc_alloc_t`, for example if you want to set up a “sparse” region of memory. In this case you need to be prepared to catch `SIGBUS` in your own application. This is also true if you use `mmap()` with `MAP_ANONYMOUS` directly in your application.

Another way that the system can run out of memory is by calling `fork()` and then writing to pages in both the parent and child. In this case, the system must make a second copy of every page of memory that is touched in both parent and child. If a huge page needs to be copied in this manner, and no huge pages are left in the free pool, the application is terminated with a `SIGKILL`.

If the system is still too low on memory to satisfy a page request, it will activate the out-of-memory killer (also known as the oom-killer), which will terminate some process in the hopes of freeing up memory. In addition, the system will print a detailed inventory of the amount of memory on the system to help with diagnosis.

5.9 Configuring TILE-Gx OpenJDK 1.6 Support for UseLargePages

The TILE-Gx version of OpenJDK 1.6 now supports the `-XX:+UseLargePages` option, which can improve performance on multi-threaded applications with large Java heap requirements by using Linux hugepages for the Java heap.

To make use of `UseLargePages` with the OpenJDK 1.6, you must:

- Ensure that the `shmmax` system parameter is large enough to create a shared memory region that can contain your largest Java heap.
- Notify the Linux kernel to reserve an appropriate number of huge pages.

Note: You can set the `shmmax` and `nr_hugepages` values in `/etc/sysctl.conf`.

5.9.1 Setting the shmmax Parameter

Set the `shmmax` system parameter slightly larger than you need, in order to contain the default class data region (approximately 160MB) as well. The value does not need to be exact. To allow a 6GB heap (plus the 160MB class data region), set `shmmax` to 8GB using this command:

```
echo 8589934592 > /proc/sys/kernel/shmmax
```

5.9.2 Setting the nr_hugepages Size

Notify the Linux kernel to reserve an appropriate number of huge pages. Since a huge page is 16MB by default, a 1GB heap would require 64 huge pages. You also need to add 10 for the class data region, bringing the total to 74. You could set this value using the command:

```
echo 74 > /proc/sys/vm/nr_hugepages
```

If you run multiple instances of Java with `-XX:+UseLargePages` simultaneously, the `nr_hugepages` value should reflect the total of their heaps. If you do not have enough huge pages reserved, huge pages might still be used if memory utilization is such that the kernel can allocate them on the fly.

Examine the file `/proc/meminfo` to see how many huge pages are reserved and in use. Do not make the `nr_hugepages` amount any larger than necessary, because the huge pages are then unavailable to the system for other purposes.

5.10 Unaligned Memory Accesses

The Tile Processor provides 1-, 2-, 4-, and 8-byte load and store instructions. These instructions require that the dereferenced address be aligned modulo the size. Any reference through an improperly aligned address triggers an interrupt.

When the default unaligned fixup handler overhead is unacceptable, the code accessing unaligned memory needs to be modified. There are three general techniques to avoid problems: annotating source code, using `<vec_ops.h>`, and writing custom code.

5.10.1 Vector Library Macros

The `<vec_ops.h>` header contains functions for various SIMD operations as well as macros for reading unaligned memory efficiently. The `<vec_ops.h>` routines are just macros, so if they do not suit your needs, you can create your own custom versions.

```
vec_unaligned_load
```

The easiest way to load an unaligned four-byte value is to invoke the `vec_unaligned_load` macro, which would have this signature if it were a function.

```
int vec_unaligned_load(const void* ptr);
```

This macro always loads two adjacent `int` variables and combines their values to produce the desired unaligned `int`. It is guaranteed not to access any unnecessary memory, so it will not cause a segmentation violation even if used to access the very last word on a page.

Although `vec_unaligned_load` is easy to use, it is also much less efficient than necessary when used inside a loop. When loading `N` consecutive unaligned values, only `N+1` aligned loads are needed, but a simple loop using `vec_unaligned_load` will naively issue `2N` loads. Furthermore, the setup overhead for loading unaligned values is the same for multiple words with the same alignment, but this will not necessarily be recognized by the compiler.

To improve performance for loading consecutive values, `<vec_ops.h>` provides some slightly harder to use but more efficient abstractions.

`vec_UnalignedPointer` and `vec_unaligned_load_N`

`vec_UnalignedPointer` is a data structure representing a pointer to unaligned memory. It contains an underlying aligned pointer as well as shift and mask values that depend on the low two bits of the original unaligned pointer. This struct is initialized by calling the `vec_init_unaligned_pointer` macro.

Once you have a `vec_UnalignedPointer`, you can pass it to the `vec_unaligned_load_1`, `vec_unaligned_load_2` and `vec_unaligned_load_4` macros, which efficiently load a fixed number of consecutive words starting at `uptr`. This comes up frequently for video codecs. When looping over an arbitrary number of unaligned values, `vec_UnalignedArray` is a better choice.

Here is an example of loading four consecutive unaligned words:

```
/* Initialize uptr. */
vec_UnalignedPointer uptr;
vec_init_unaligned_pointer(uptr, start_address);

/* Load four unaligned values starting at 'start_address'. */
int v0, v1, v2, v3;
vec_unaligned_load_4(v0, v1, v2, v3, uptr);
```

You can also efficiently stride a `vec_UnalignedPointer` to another address with the same misalignment without the overhead of calling `vec_init_unaligned_pointer` again, by modifying its aligned `ptr` field:

```
uptr.ptr += 16; /* move ahead 16 ints, i.e. 64 bytes. */
```

Putting these ideas together, a video codec could process a 16x16 block of unaligned bytes, where each row is identically aligned, like this:

```
/* Initialize uptr. */
vec_UnalignedPointer uptr;
vec_init_unaligned_pointer(uptr, start_address);

for (int row = 0; row < 16; row++)
{
    /* Load four consecutive unaligned values. */
    int v0, v1, v2, v3;
    vec_unaligned_load_4(v0, v1, v2, v3, uptr);

    /* Process the four values. */
    do_something(v0, v1, v2, v3);

    /* Move on to the next row. */
    uptr.ptr += row_stride_measured_in_ints;
}
```

Caution Because the `vec_unaligned_load_N` macros always load $N+1$ words and extract N words of data from them, when the original pointer is already aligned, they will access one unused word beyond the last word required. Programs that use these macros need to ensure there is one extra word of storage at the end of the block of memory being processed to avoid a possible segmentation violation from reading too far.

`vec_UnalignedArray`

Although `vec_UnalignedPointer` is useful when loading just a few words from a given unaligned address, often you need to loop over many unaligned values in an array. `vec_UnalignedArray` makes that process efficient by keeping track of the word loaded

by the previous loop iteration so it can be combined with the following word. In this way it only issues one load per loop iteration, rather than the two you would get by using `vec_unaligned_load`.

For example:

```
vec_UnalignedArray uarray;
vec_init_unaligned_array(uarray, start_address);

while (...)
{
    uint32_t x = vec_unaligned_array_load_next(uarray);
    ...
}
```

`vec_unaligned_array_load_next` actually loads the value to be used on the *next* loop iteration, as a form of simple prefetching. If you happen to know an unaligned access is the final one, you can use `vec_unaligned_array_load_last` to avoid prefetching the next value. This is typically not important, however, as the cases where this is legal tend to be the cases where the compiler can optimize away the prefetching load as dead code.

Because of the prefetching, `vec_unaligned_array_load_next` can access two words past the end of the array in the worst case, so programs using it need to make sure the allocated array has extra space at the end to avoid potential segmentation violations.

5.10.2 Loading One Unaligned Value

This section shows C and assembly code sequences that efficiently load one int from an arbitrary unaligned address. `<vec_ops.h>` provides a `vec_unaligned_load` macro that performs this operation, but this section explores the details of how you would write your own.

Note that when loading a sequence of consecutive unaligned values, the techniques described in this section are not optimal. See [Section 5.10.3 “Loading Multiple Unaligned Values” on page 56](#) for information about that case.

Overview of the Problem

[Figure 5-1](#) represents a segment of word-aligned memory at some arbitrary base address, which we call `BASE` and number as 0. The numbers along the top indicate the byte-addressed offset from the base address. Each box represents one byte in memory and its contents are displayed in hex. For this simple example, the bytes are sequentially numbered starting at 0x10. So, a 32-bit word at $(BASE+8)$ is 0x1B1A1918.

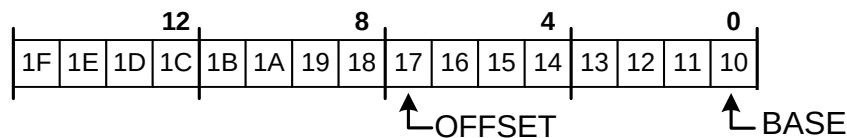


Figure 5-1: Segment of Word-Aligned Memory at a Base Address

If we need to access a full word of data starting at $(BASE+OFFSET)$, the desired data word is 0x1A191817. If we try to access this memory directly using standard methods


```
int* ptr = (BASE+OFFSET);
int data = *ptr;
```

The processor sends an unaligned memory access interrupt and automatically invokes the fixup routine (unless it is disabled). One way to avoid this problem is to load the value using only aligned load instructions. One simple algorithm is:

1. Load the first aligned word below the desired address. In our example, the *lower word* starts at offset 4.
2. Load the first aligned word above the desired address. Here, the *upper word* starts at offset 8.
3. Shift each word so that the desired data is in the proper byte-wise position.
4. Bit-wise OR the two words together.

5.10.2.1 Accelerating Unaligned Memory Accesses

The Tile Processor provides two special instructions to accelerate unaligned memory accesses:

ldna

Load an eight byte quantity from memory into the destination register. The address of the value to be loaded is read from the source operand and the bottom three bits are set to zero. No Unaligned Data Reference interrupts are caused by this instruction.

This instruction is useful because it avoids the need to mask off the low two bits of an unaligned input address before issuing a load.

dblalign

This instruction extracts an unaligned int variable from the two adjacent aligned int variables that it might span. It determines which bytes to extract by examining the low two bits of a third operand, which can just be the original unaligned pointer. The operation can be thought of as taking the low 64 bits of a 128-bit shift:

```
dblalign op0, op1, op2
```

is equivalent to the following C code:

```
op0 = (uint32_t)((op0 | ((uint64_t)op1 << 32)) >> ((op2 & 3) * 8));
```

In other words, when the low two bits of the pointer are 0x0, meaning it was already aligned, `dblalign` returns `op0` (the first operand). When the low two bits are 0x1, it right shifts the 64-bit concatenation by 8 and therefore returns the high three bytes of `op0` and the low byte of `op1`, and so on.

At the assembly level, `dblalign`'s first input register is also its output register, meaning it always overwrites one of its inputs. When using C intrinsics for instructions like this, it is good form to assign the result value to the same variable passed as the first argument, to both make this restriction clear to the reader and help the compiler generate better code.

The overwritten input corresponds to the aligned int at the lower of the two addresses. In a simple loop through unaligned memory from low to high addresses, this value is no longer needed after it is passed `dblalign`, so overwriting it is harmless.

The Tile Processor instructions allow an unaligned pointer to be de-referenced efficiently:

```

unsigned int
unaligned_load(const void* ptr)
{
    // Load the aligned words spanned by 'ptr'.
    // Note that we add 3, not 4, when loading 'hi', so that
    // if 'ptr' is already aligned we load the same int twice
    // rather than accessing potentially invalid memory at ptr + 4
    // and getting a SEGV.
    unsigned int lo = __insn_ldna(ptr);
    unsigned int hi = __insn_ldna(ptr + 3);

    // Combine the two adjacent words using the low 2 bits
    // of 'ptr' to decide which bytes to combine from
    // 'lo' and 'hi'.
    lo = __insn_dblalign(lo, hi, ptr);
    return lo;
}

```

This technique compiles into the following instruction sequence:

```

{
    ldna    r1, r0
    addi    r2, r0, 3
}
{
    ldna    r2, r2
}
{
    ## Final result in r1.
    dblalign r1, r2, r0
}

```

5.10.2.2 Using Packed Struct Fields

There is another way to write code that loads a single int from an arbitrary unaligned address. Normally the compiler guarantees that struct fields are properly aligned, but when a field has the *packed* attribute it is allowed to be stored at an unaligned address. To avoid unaligned load faults when accessing such fields, the compiler automatically generates the optimized code sequences described above. This “packed field” technique for unaligned loads is therefore the same speed as the chip-specific techniques, but is recommended because it is concise and optimal on both chips.

The `vec_unaligned_load` macro in `<vec_ops.h>` is implemented this way:

```

unsigned int
portable_unaligned_load(const void* ptr)
{
    // Create a pointer to a struct with one "packed" field and dereference it.
    const struct { unsigned int f __attribute__((packed)); }* s = ptr;
    return s->f;
}

```

5.10.3 Loading Multiple Unaligned Values

A common case for unaligned memory accesses is to load several consecutive unaligned values. It is tempting to write such loops by using one of the techniques described in the previous section to load each unaligned value one at a time:

```
// A slow way to sum n unaligned ints starting at 'ptr'.
unsigned int
unaligned_sum(const void* ptr, int n)
{
    unsigned int sum = 0;
    while (n--)
    {
        // BAD! THIS PERFORMS TWICE THE LOADS WE NEED!
        sum += vec_unaligned_load(ptr);
        ptr += 4;
    }
    return sum;
}
```

Although simple, this is slow and should be avoided when performance matters. The problem is that each standalone unaligned load requires two aligned loads, meaning that processing an array of N unaligned values requires $2N$ loads. However, only $N+1$ loads are actually necessary, because the high bits of one unaligned value are stored in the same aligned memory value as the low bits of the next unaligned value. The key is to use each loaded value twice, once for each of the two unaligned values that overlap it.

In this example, only one load is issued per loop iteration:

```
unsigned int
unaligned_sum(const void* ptr, int n)
{
    const unsigned int* x = (const unsigned int*)((long)ptr & ~3);

    unsigned int a = *x++;
    int sum = 0;

    while (n--)
    {
        unsigned int b = *x++;
        a = __insn_dblalign(a, b, ptr);
        sum += a;

        // Reuse this load result for the next iteration.
        a = b;
    }

    return sum;
}
```

With various helpful loop optimizations (for example, unrolling and software pipelining) disabled to keep the example small, the inner loop compiles to this sequence:

```

loop:
{
    addi    r2, r2, 4
    move    r5, r1
}
{
    lw      r1, r2
    addi    r6, r6, -1
}
{
    seqi    r4, r6, -1
    dblalign r5, r1, r3
}
{
    add     r0, r0, r5
    bzt     r4, loop
}

```

As written, this implementation has one minor drawback. Because it always issues $n+1$ aligned loads, in the case where `ptr` is already aligned it will read one int past the end of memory it was supposed to access. In theory, this could cause a segmentation fault. There are several workarounds to this problem, such as allocating enough array space so that one extra load is safe, looping $n-1$ times and handling the last value specially, or using a specialized (and faster) alternative loop in the case where `ptr` is already aligned.

5.10.4 Shifting Over Values Already in Registers

Sometimes you need to examine the same memory locations starting at multiple unaligned offsets. For example, suppose you want to scan through a range of memory looking for a particular 32-bit value at any byte offset.

Here is a simple way to write it:

```

// Slow way to check for a 32-bit value at any byte offset.
int
find_int(const void* ptr, int size, unsigned int val)
{
    for (int offset = 0; offset <= size - 4; offset++)
        if (vec_unaligned_load(ptr + offset) == val)
            return offset;

    // Not found.
    return -1;
}

```

Unfortunately, this algorithm ends up loading each aligned int four times, once for each offset modulo 4. You can reduce the loads by a factor of 4 by only loading each aligned int once and then shifting it several times to different alignments.

```

// Search for a 32-bit value, minimizing loads.
// This uses intrinsics.
//
// NOTE: This assumes 'ptr' and 'size' are aligned mod 4
// to remove edge-case checks that clutter the example.
int
find_int(const void* ptr, int size, unsigned int val)
{
    const unsigned int* x = ptr;

    unsigned int a = *x++;
    if (a == val)
        return 0;

    for (int offset = 0; offset <= size - 8; offset += 4)
    {
        unsigned int b = *x++;

        if (__insn_dblalign(a, b, 1) == val)
            return offset + 1;
        else if (__insn_dblalign(a, b, 2) == val)
            return offset + 2;
        else if (__insn_dblalign(a, b, 3) == val)
            return offset + 3;
        else if (b == val)
            return offset + 4;

        a = b;
    }

    // Not found.
    return -1;
}

```

This technique can be generalized to “sliding over” larger windows of registers one byte at a time, and so forth. This comes up in video codec motion estimation algorithms. For example, to shift over a 16-byte window held in local variables a, b, c, d, by one byte, you could write:

```

// Sequence to shift over 16 bytes by one byte position.
a = __insn_dblalign(a, b, (const void*)1);
b = __insn_dblalign(b, c, (const void*)1);
c = __insn_dblalign(c, d, (const void*)1);
d >>= 8;

```

Some DSP algorithms work on 16-bit values and consequently need to shift over a sliding window by two bytes. This is of course a trivial change to the above algorithm.

The 32-bit sliding step can be avoided altogether in one important case. If the shifted values are simply feeding into multiply instructions, choosing the correct multiply opcode eliminates the need to shift the values over at all. There are many distinct 16x16 -> 32 multiply opcodes, which include support for all combinations of multiplying together the high and low halves of two registers. For example, `__insn_mulll_ss(a, b)` multiplies the low 16 bits of a and b, `__insn_mulhl_ss(a, b)` multiplies the high 16 bits of a by the low 16 bits of b, and `__insn_mulhh_ss(a, b)` multiplies the high 16 bits of a and b. There are further variants of these instructions for unsigned multiplies and accumulating results. Using these instructions rather than shifting multiply inputs by 16 bits yields more efficient code. For more details on multiply instructions, see the *Tile Processor and I/O Device Guide for the TILE-Gx Family of Processors* (UG404).

5.10.5 Unaligned Stores

Unfortunately, there is no clever trick to implement a single 32-bit store to an arbitrary unaligned address. Issuing two 32-bit aligned stores (by analogy with unaligned load techniques) does not work because it will overwrite bytes adjacent to the 32 bits being stored. If you know the address equals two mod 4 you can of course issue two aligned 16-bit stores, but if you do not know anything about the alignment in advance the best technique is simply to store the four individual byte values one at a time.

When issuing a sequence of consecutive unaligned stores, as opposed to just one, it is possible to do better than using byte stores. The key is to use “sliding over” techniques on the input values to produce values suitable for aligned 32-bit stores. The “edges” of the consecutive sequence still require individual byte stores, but the “middle” can be done with 32-bit stores.

For example, when `memcpy` is given source and destination pointers with different alignments, it cannot use simple 32-bit loads and stores because either the source or the destination must be misaligned. So it copies individual bytes until the destination pointer is 4-byte aligned, then uses unaligned load techniques described earlier to load and align source values and 4-byte aligned stores to write to the destination. If necessary, it does a few byte stores at the end to finish.

5.11 Additional Tips on Managing Alignment of Data

We provide the following additional information concerning data alignment.

5.11.1 Using the `memalign()` Function

For dynamically allocated data, you can choose the alignment by using the `memalign()` function in the usual way for Linux.

For example, to get cacheline aligned memory:

```
#include <arch/chip.h>
void* my_data = memalign(CHIP_L2_LINE_SIZE(), sizeof(struct foo));
```

5.11.2 Using Linker Scripts

Linker scripts allow much more control over data placement, particularly in combination with the `__attribute__((section))` directive in the `tile-gcc/g++` compiler. Contact Tlera technical support for more information on using this approach.

5.11.3 Using Page Table Special Files to Confirm Allocation

The `/proc/PID/pgtable` family of special files allows users to see which virtual addresses are mapped to which physical addresses, and with what caching characteristics. This can be useful for diagnosing performance or correctness issues and for confirming that you have allocated memory as you wanted.

For more information, enter `tile-man pgtable` at the command line.

5.12 Examples of Dynamic Memory Allocation

- To allocate 64KB on memory controller #2:

```
#include <tmc/alloc.h>
...
tmc_alloc_t alloc = TMC_ALLOC_INIT;
alloc_set_nodes(&alloc, 2);
void *ptr = alloc_map(&alloc, 65536);
```
- To allocate 64K of memory that is remotely cached by processor #3:

```
#include <tmc/alloc.h>
...
tmc_alloc_t alloc = TMC_ALLOC_INIT;
alloc_set_home(&alloc, 3);
void *ptr = alloc_map(&alloc, 65536);
```
- To create an mspace and use it to allocate some memory that is constrained to controller #2:

```
#include <tmc/mspace.h>
#include <tmc/alloc.h>
...
tmc_alloc_t alloc = TMC_ALLOC_INIT;
alloc_set_nodes(&alloc, 2);
tmc_mspace msp = tmc_mspace_create(0, 0, &alloc);
...
void* ptr = tmc_mspace_malloc(msp, 100);
```
- To create an mspace and use it to allocate some memory that is homed on tile #3 and is safe for threaded applications to use:

```
#include <tmc/mspace.h>
...
tmc_alloc_t alloc = TMC_ALLOC_INIT;
alloc_set_home(&alloc, 3);
tmc_mspace msp = tmc_mspace_create(0, MSPACE_LOCKED, &alloc);
...
void* ptr = tmc_mspace_malloc(msp, 100);
```


CHAPTER 6 DYNAMIC DISTRIBUTED CACHE TECHNOLOGY

This chapter describes DDC™ (Dynamic Distributed Cache), the caching subsystem for the Tile Processor:

- [Section 6.1 Overview of DDC](#)
- [Section 6.2 Default Caching Policy](#)
- [Section 6.3 Page Attributes in DDC](#)

See also:

- [Chapter 4: Memory Homing and Allocation Concepts](#)
- [Chapter 5: Controlling Memory Allocation in an Application](#)

6.1 Overview of DDC

The TILE-Gx Processor uses DDC (Dynamic Distributed Cache) technology to enhance on-chip caching and coherence performance.

Some features of DDC are:

- Fully coherent distributed caching for all memory pages. Any tile can cache any cache block in its local caches. Hardware maintains coherence for all cache blocks.
- Allocation control: The user can optionally control the level of cache hierarchy for allocation.
- Distribution of cache blocks: Cache blocks can optionally be distributed over the entire array of L2 caches in a fine-grained manner.
- Coherent and direct-to-cache I/O: The Tile Processor can maintain coherence for I/O direct memory accesses (DMA) in hardware, without software support. Additionally, the I/O DMA engines can deliver packet data directly to the on-chip caches, saving bandwidth and latency for packet processing applications.

The hardware uses a distributed directory to record which cores are caching which blocks. On a write to a cache line, the hardware performs the necessary invalidations of the stale cached copies to maintain cache coherence.

[Table 1](#) highlights the features of the caching subsystem.

Table 1. TILE-Gx Processor Cache Subsystem

L1 instruction (L1I) cache	32 KB, two-way associative
L1 instruction translation lookaside buffer	16 entries, fully associative
L1 data (L1D) cache	32 KB, two-way associative
L1 data translation lookaside buffer	16 entries, fully associative

Table 1. TILE-Gx Processor Cache Subsystem

L2 unified cache	256 KB, eight-way associative
L3 coherent cache	9MB
Latency (load to use)	L1 Dcache hit: 2 cycles L1 Dcache miss, L2 Cache hit: 11 cycles L2 Cache miss, neighbor L3 Cache hit, Dword 0: 32 cycles L2 Cache miss, neighbor L3 Cache hit, Dword 1 to 7: 41 cycles L2 cache miss, DDR3 page open (typical): 67 cycles L2 cache miss, DDR3 page miss (typical): 88 cycles
Architecture	Non-blocking, out-of-order, stall-on-use

Among all Tiles, there is an aggregate of 9MB coherent L3 cache.

The per-tile L1 caches are *private* caches. The only way cache lines are allocated into the private caches is by means of load and store and istream references generated by the local tile.

The L2 cache is managed as a hybrid cache. The L2 cache at tile X can allocate two different types of cache lines:

- Cache lines that are requested by tile X (due to cache misses in the local L1 and L2 caches) but homed remotely (L2 cache lines).
- Cache lines that are requested by remote tiles (other than X) and *homed* by tile X (L3 cache lines).

Each L2 cache dynamically determines which one of the two types of cache lines listed above should be displaced on an eviction from the L2 cache. This displacement decision is determined by a per-set pseudo-LRU algorithm.

The per-tile translation buffers store the home tile number on a memory-page-by-memory-page basis. All cache lines on a given page can either be homed on a specific tile, or can be spread across multiple tiles (a *hashed* page).

L3 cache lines are inclusive: Displacement of an L3 cache line invalidates all copies of the cache line (unless the `P.NonInclusive` page attribute is used).

Figure 6-1 shows a read (for example, due to the execution of a load instruction) from core A (the remote requesting core) to a cacheline X, where cacheline X is homed at core B (the home core):

1. Core A first checks its local caches for the cacheline X and, if cacheline X is not present, sends a request for cacheline X to core B.
2. Core B receives the request for cacheline X and retrieves cacheline X from its L2.
3. Core B then sends the full cacheline X back to core A. Core A installs cacheline X in its local L1 and L2 caches.

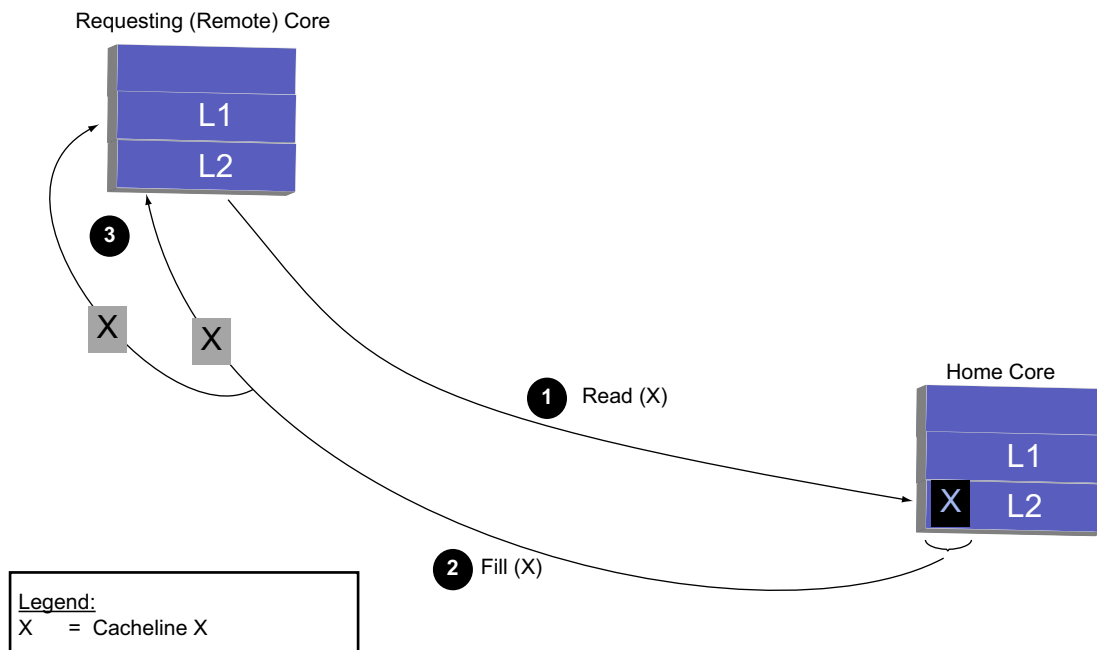


Figure 6-1: Request to Home Core/Fill L2/L1 with Cacheline X

Figure 6-2 shows a write from core A to a word (X[0]) in cacheline X, where cacheline X is again homed at core B. In this example, a third core (C) also has a copy of cacheline X.

1. Core A sends the write address and data to core B.
2. Core B receives the write address and data and checks the directory information for cacheline X. The directory indicates that core C (the sharing core) has a copy of cacheline X. Core B updates cacheline X with the new value for word X[0].
3. Core B sends an invalidate message to core C.
4. Core C receives the invalidation and invalidates cacheline X from its caches.
5. Core C then sends an invalidation acknowledgement back to core B.
6. Core B receives the invalidation acknowledgement and sends a write acknowledgement back to core A.
7. Core A receives the write acknowledgement message and thus knows that the write to word X[0] has completed.

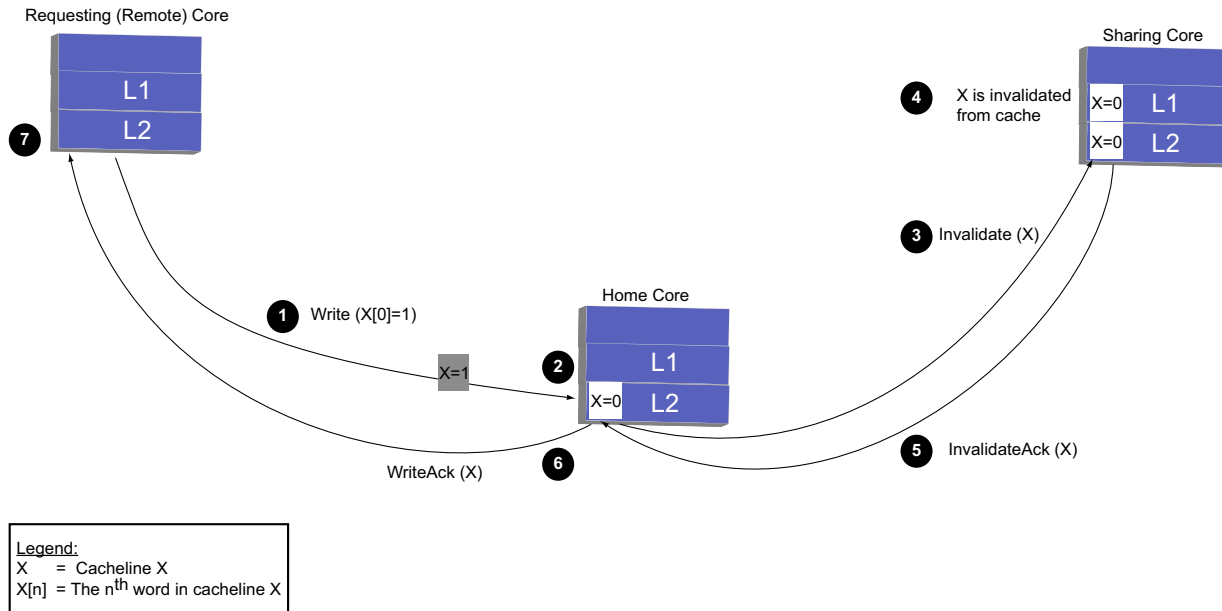


Figure 6-2: Write from Core A to Word [0] in Cacheline X

Because the Tile architecture does **not** wait for the write acknowledgement before it continues to issue instructions, this latency is rarely visible to application code.

6.2 Default Caching Policy

The default caching policy takes advantage of the Tile Processor's distributed L3 cache, similar to the caching hierarchy for which pre-existing SMP Linux applications are usually tuned.

By default, all applications and system code take advantage of the dynamic cache functionality. The dynamic cache brings cache lines into the local cache automatically, and maintains coherence even for locally-cached data.

Read-only data sections in the program executables (the *text* or code, and the read-only data) are cached using *hash for home*, so that the cache lines are homed scattered across all the hashing tiles. They are cached with the *non-inclusive* attribute, so that if the L3 cache line on the home cache is evicted, the L2 cache lines on other tiles are not evicted.

By default, the Linux kernel (code) is handled like user text, using hash-for-home. The kernel data cache homing is distributed at cache-line granularity by hash-for-home.

See:

- [Section 6.2.1 System-Wide Cache Defaults](#)
- [Section 6.2.2 Per-Application Cache Defaults](#)
- [Section 6.2.3 Specifying Tiles for Hash-for-Home](#)

6.2.1 System-Wide Cache Defaults

By default, the Tile Processor caches everything except *stack* memory using hash-for-home. All stack memory is cached with each page of memory set to have its home cache on the tile that allocates it.

The `ucache_hash` and `kcache_hash` options can be used to change the default. For example, `kcache_hash=ro` makes the kernel use hash-for-home only for read-only parts of the kernel, that is, code and read-only data; `kcache_hash=none` makes the kernel use no hash-for-home pages.

The `cache_hash` boot option will set both `kcache_hash` and `ucache_hash` to the specified value.

For more information, see the *Multicore Development Environment System Programmer's Guide* (UG509).

6.2.2 Per-Application Cache Defaults

The Tile Processor uses an environment variable, `LD_CACHE_HASH`, to determine which parts of a user process will use hash-for-home by default. The settings for the environment variable work just like the `kcache_hash` settings discussed above, with possible values such as `ro` (use hash-for-home for read-only sections) or `none` (do not use hash-for-home at all).

If `LD_CACHE_HASH` is not specified for a particular process, the kernel uses internal defaults to choose the use of hash-for-home, by default choosing hash-for-home for everything except the stack. The `ucache_hash` boot option can be used to change this default system-wide.

The `cache_hash` boot option will set both `kcache_hash` and `ucache_hash` to the specified value.

For more information, see the *Multicore Development Environment System Programmer's Guide* (UG509).

6.2.3 Specifying Tiles for Hash-for-Home

By default, the hypervisor places all the tiles available to the client supervisor in the “hash for home” set, so hashing will be done over all of them approximately equally.

To modify the set of tiles used, you must set the `hfh_tiles` subcommand in the `client` section of the hypervisor configuration (`.hvc`) file. This subcommand sets the Address Mapping Table (AMT) to the desired set of tiles.

For example, you could set `hfh_tiles` to be the same as the dataplane set specified in the Linux boot arguments and then use hashing for all the dataplane allocations. Or you could set `hfh_tiles` to be the non-dataplane tiles, and use only per-page caching for the dataplane tiles, to avoid evicting any cache lines on dataplane tiles.

For more information, see the section on the `hfh_tiles` subcommand in the “Hypervisor Customization” chapter in the *Multicore Development Environment System Programmer's Guide* (UG509).

6.3 Page Attributes in DDC

Several per-memory-page attributes allow different cache configurations to be constructed. For certain workloads, choosing the optimal configuration can result in higher performance. These attributes enable multiple memory modes such as hardware-coherent, user-managed coherence, and pinned memory.

6.3.1 Page Attributes for Allocation Control

These attributes specify which level in the cache hierarchy a page (P) can be cached:

- `P.NoAllocateL1`: Cache lines on page P will not allocate in the requesting tile's L1 Dcache. This attribute can be controlled by means of the `MAP_CACHE_NO_L1` flag for `mmap()` or `tmc_alloc_set_caching()`.
- `P.NoAllocateL2`: Cache lines on page P will not allocate in the requesting tile's L2 cache. This attribute can be controlled by means of the `MAP_CACHE_NO_L2` flag for `mmap()` or `tmc_alloc_set_caching()`.
- `P.NonInclusive`: Cache lines on page P are treated as non-inclusive. Hardware will not maintain inclusion or coherence for cache lines on page P. This attribute can be used only for read-only or instruction pages. By default, all such pages have this attribute; it is not settable in software.

6.3.2 Page Attribute for Distribution of Cache Blocks

The `P.HashPage` attribute allows control of the distribution of cache lines on a page. When the `P.HashPage` attribute is set, the home for the cache lines on page P are distributed uniformly across all the L2 caches in the system. This distribution function is determined by a hardware hash operation. The set of tiles included in this distribution can be further modified by system software programming of the per-tile Address Mapping Table (AMT).

The flexibility of the page attributes allows advanced users to configure application-specific caching schemes. The following example illustrates this.

Use the `P.HashPage` attribute to assign 16 tiles as a dedicated, shared 1MB L3 cache. No application code executes on these 16 tiles; they are used as cache only. I/O data is coherently delivered directly into this 1MB cache by means of direct-to-cache I/O. Forty-eight separate compute/worker tiles, each with its own 64K private L2 cache, read data from the 1MB L3. All caches are kept coherent by hardware.

CHAPTER 7 ZERO OVERHEAD LINUX APPLICATIONS

This chapter describes the capabilities of Zero Overhead Linux™ (ZOL):

- [Section 7.1 Understanding Zero Overhead Linux](#)
- [Section 7.2 Using the Linux Scheduler on Dataplane Tiles](#)
- [Section 7.3 Running a Task on a Dataplane Tile](#)
- [Section 7.4 Interrupting a Dataplane Core](#)
- [Section 7.5 Programming Models for Dataplane Mode](#)
- [Section 7.6 Using the `set\_dataplane\(\)` API](#)
- [Section 7.7 Spin-Based Shared Memory Synchronization](#)
- [Section 7.8 Avoiding System Calls](#)
- [Section 7.9 User Dynamic Network \(UDN\)](#)
- [Section 7.10 Troubleshooting ZOL Applications](#)
- [Section 7.11 Dataplane Example](#)

7.1 Understanding Zero Overhead Linux

The Tile Processor™ Linux distribution includes support for Zero Overhead Linux™ (ZOL). ZOL allows a programmer to specify a set of *dataplane* tiles, each of which will run a single, user-space task without incurring any Linux system overhead.

You specify a dataplane tile by using the `dataplane=` boot argument.

Dataplane mode is a per-cpu setting that marks cpus as primarily intended to run jitter-sensitive or very-low-latency userspace code. The Linux kernel does not generally interrupt dataplane CPUs, thus they run just userspace code.

ZOL allows developers to take advantage of the full Linux environment during application startup, then stop invoking Linux system calls on the dataplane tiles and settle into the real-time execution mode while running the main processing loop. Standard Linux tiles run the Linux preemptive task scheduler, which takes periodic interrupts to allow task preemption and take care of miscellaneous operating system tasks. ZOL allows the programmer to specify a subset of the processor cores to run in *dataplane* mode. As long as a task running on a dataplane core does not make any system calls, it will not suffer any jitter due to Linux interrupts.

The primary intent of ZOL is to allow developers to write all or part of their application in a style that maintains real-time guarantees. For example, networking applications are often split into a fast path that touches every packet and a slow path that works on only a subset of the packets. Because XAUI devices can generate up to 15 million packets per second, any Linux interrupt could consume hundreds or even thousands of packets worth of cycles, causing packet drops. Running the fast path on dataplane tiles can eliminate this difficulty.

On the other hand, running on a dataplane tile increases the overhead of Linux system calls made on that tile (dataplane tiles queue up certain kernel work until the next system call) and severely limits the ability of the Linux scheduler to migrate tasks to available cores. Thus, applications that do not have severe real-time requirements or that need to run multiple threads per core should be run on standard, not dataplane, tiles. For applications that make system calls and do not specifically require the interrupt prevention behavior of dataplane tiles, it is better to use process/thread tile affinitization instead.

Tilera recommends using the `dataplane` boot argument for any application that does not tolerate latency well, such as 10 Gbps networking applications.

To enable cores for dataplane mode, specify the `dataplane=` boot argument to the kernel, using the normal kernel list syntax (a list of comma-separated CPUs or hyphen-separated ranges of CPUs). If you specify nonexistent CPUs, the kernel silently ignores the non-existent CPUs.

Note: You must configure at least one CPU as a non-dataplane CPU, to provide the kernel space to run.

Simulator images are not generally configured with dataplane enabled; the exception is the `gx8036-dataplane` image, which configures all cores except core 0 as being in dataplane mode.

Note: Once you have booted the system, you cannot change cores from dataplane to non-dataplane mode and vice versa.

Dataplane tiles can still use all normal Linux functionality, but as soon as they enter the kernel they are treated like normal Linux CPUs again. On return to userspace, Linux will resume ZOL mode.

If a dataplane tile calls one of the atomic operations defined in `<atomic.h>`, this does not cause the tile to leave ZOL mode. Dataplane tiles can also call any of the `tmc_spin_*` functions, which do not make any kernel calls.

For more information about `<atomic.h>` and about the `tmc_spin_*` functions, see the *Application Libraries Reference Manual* (UG527).

For more information about using the `dataplane` boot argument, see the “Linux Interfaces and Customization” chapter of the *Multicore Development Environment System Programmer’s Guide* (UG509).

7.2 Using the Linux Scheduler on Dataplane Tiles

The Linux scheduler is responsible for load balancing tasks across multiple processors. ZOL *dataplane* tiles modify the standard scheduler algorithm so that a dataplane tile running a single task will avoid taking timer interrupts to run the scheduler.

This section provides a detailed explanation of how the Linux scheduler works and how it interacts with dataplane tiles. Programmers who want to simply treat dataplane tiles as a black box can skip this material and move on to [Section 7.3 “Running a Task on a Dataplane Tile” on page 72](#).

7.2.1 Run Queues

Each processor has a run queue, consisting of two arrays of tasks:

- **ready tasks** — Ready tasks are tasks that are ready to execute more instructions. At any moment, one ready task can be running and the rest are waiting their turn. Linux is a time-sharing system, meaning that if there are more ready tasks than available processors, the scheduler automatically *context switches* back and forth between tasks so that each task makes progress on its workload. A context switch between two ready tasks can consume five to ten thousand cycles.

- **sleeping tasks** — Sleeping tasks are stopped in a blocking system call, such as a `read()` on a socket that currently has no data. The vast majority of processes on a typical Linux system are in this state at any point, waiting for something to happen. Because they are not ready to run, sleeping tasks do not consume any processor resources, just memory. When the blocking condition ends, for example because data shows up on the socket it was reading, a task becomes ready to run again and it moves into the ready list.

7.2.2 Scheduler Interrupts

Each non-dataplane processor takes a timer tick interrupt 100 times a second, during which it runs the scheduler algorithm to determine which ready task should execute next. Each timer tick takes roughly 30,000 cycles (although it can take much longer if, for example, a previous system call has deferred some work until the next timer tick). The total interrupt overhead is well under 1%.

During the timer tick, the scheduler determines which task should be running on that processor during the next 0.01 seconds, starting with ready tasks in that processor's local run queue. The scheduler might decide to continue running the same task, context-switch to another ready task in its run queue, or even steal a ready task from some other processor's run queue. The exact algorithm is complicated, but the scheduler's goal is to be fair in how it allocates processing time to competing tasks. A task can be given more or less processing time by altering its priority, as with the `nice` and `renice` commands.

7.2.3 Task Migration

Although the scheduler tries to keep tasks running on the same processor to optimize cache locality, sometimes it must migrate tasks between processors. One of the scheduler's goals is to balance work across all processors, which is why processors sometimes steal ready tasks from other run queues. A processor is of course only allowed to steal a task whose affinity set allows it.

The other way for a task to end up in a processor's run queue is if its affinity set is modified to exclude the processor in whose run queue it currently resides. In this case, Linux immediately migrates the task to the lowest-numbered processor in its affinity set.

7.2.4 Dataplane Tiles

Dataplane tiles are just like standard Linux tiles, except that they disable operating system overhead that can be problematic for certain real-time applications.

For example, consider an application that needs to analyze incoming Ethernet packets with a guaranteed worst-case latency of 10,000 cycles per packet. It simply cannot tolerate a 30,000 cycle timer tick interrupt to run the Linux scheduler, even if such interrupts only consume a tiny fraction of overall time. Dataplane tiles solve this problem.

Dataplane tiles differ from standard tiles in several ways:

- Dataplane tiles disable the 100 Hz scheduler interrupt to avoid interrupting a task running on that tile. Therefore, the normal scheduler algorithm does not run.

The one exception is if more than one task ends up in a dataplane processor's ready run queue, in which case the timer is re-enabled so it can multitask between those tasks. Of course, placing multiple tasks on a dataplane tile is normally not done because it defeats the purpose of using dataplane tiles.

If the process itself requests timer functionality, such as by calling `setitimer`, that works normally. It is only scheduler-induced timer interrupts that are disabled.

- No task will ever enter a dataplane run queue unless its affinity set prevents it from running on a non-dataplane tile. In other words, dataplane tiles only run tasks that you force them to run. Dataplane tiles never steal work from the run queues of other tiles, even if the affinity set allows the task to run on either tile.

- When running user code, dataplane tiles are shielded from kernel memory-management events that normal tiles handle immediately. Instead, when such events occur, the dataplane tile is tagged such that the next time it enters the kernel, it catches up with any such pending events at that time. This technique eliminates overhead that would otherwise occur when, for example, a non-dataplane tile remaps a kernel shared memory region that is also visible to a dataplane tile.

In addition, several normal timer-driven tasks are disabled for dataplane tiles, such as the *watchdog* timer that watches for bugs in the kernel and scheduler, and the *slab reaper* that returns memory to the global allocator from the per-CPU freelists.

Dataplane cores suppress a number of standard kernel book-keeping tasks to avoid interrupting user space:

- The kernel soft lockup watchdog is disabled, so soft lockups (when the kernel blocks internally and stops scheduling new tasks) are not detected on dataplane cores.
- Dataplane cores skip the `vmstat` update, along with the usual draining of per-cpu free pages back to the global allocator.
- TLB flush events that target kernel pages are suppressed when they target cores running dataplane tasks. Instead, the tasks flush all the kernel TLB entries when they finally enter the kernel.

Dataplane cores also try to stop the scheduler core if possible whenever they leave the kernel to return to userspace, which makes syscalls somewhat slower than they are on non-dataplane cores.

The kernel places a task in dataplane mode if it is running on a core that is enabled for dataplane and the task meets a number of criteria:

- No other run-able task is on the same core. In general, you should use the standard kernel affinity mechanism to bind only a single task (process or thread) to a single dataplane core at a time. The cores do not attempt to steal work from other cores to load-balance.
- No kernel events are pending for that core.

A task leaves dataplane mode when it enters the kernel for any reason. The kernel re-enters normal (non-dataplane) mode when the task enters the kernel. It may schedule future kernel events (scheduler ticks, etc.) to run even after returning to userspace, which in turn may cause the core to re-enter the kernel a few times before quiescing back to dataplane mode in userspace.

7.3 Running a Task on a Dataplane Tile

The standard way to use dataplane tiles is for an application to bind each of its tasks to a distinct dataplane tile by setting that task's affinity to include only the desired tile. Once there, the tasks run normally and can take advantage of the full range of Linux services, although they need to stop making system calls in order to settle into ZOL mode.

A tile is designated as a dataplane tile by being listed in the `dataplane=` list in the Linux (client) boot arguments in the hypervisor configuration (`.hvc`) file. This decision cannot be changed after boot time. With standard `.hvc` files, the easiest way to select dataplane tiles is by using the `--hvx` flag, which by convention adds to the list of Linux boot arguments. For example, on an 6x6 chip this would designate all tiles except the top left one as dataplane tiles:

```
$ tile-monitor --hvx dataplane=1-35
```

It is a good idea to always leave at least one non-dataplane tile to run miscellaneous Linux daemons.

A hypervisor configuration file will designate some tiles as *dedicated* to running nothing but hypervisor device drivers. These tiles are not available for Linux to use at all. For convenience, they can be mentioned in the dataplane list but they will nonetheless be unavailable.

Once Linux has been booted with a set of dataplane tiles, the application needs to explicitly bind its tasks to the dataplane tiles. For example:

```
// Get the set of dataplane cpus, then bind this task to the 'rank'th cpu.
cpu_set_t dataplane;
if (tmc_cpus_get_dataplane_cpus(&dataplane) != 0)
    die("Failed to get dataplane cpus.");
if (tmc_cpus_set_my_cpu(tmc_cpus_find_nth_cpu(&dataplane, rank)) != 0)
    die("tmc_cpus_set_my_cpu() failed.");
```

The spinner example code, located in the MDE at \$TILER_ROOT/examples/dataplane/spinner/, demonstrates a program that run several dataplane tasks and verifies that each CPU does not take any Linux interrupts.

7.4 Interrupting a Dataplane Core

A task can get interrupts from kernel book-keeping for a few ticks after its last return to userspace, as the kernel cleans up various bits of internal state (for example, the kernel's RCU locking mechanism may require waiting a timer tick duration before a cpu is no longer involved in the RCU locking protocol). However, see the DP_QUIESCE flag below.

Note: A task obviously enters the kernel when it makes a system call, but it also will enter the kernel when it touches a mapped page of memory for the first time and has to create a suitable page table entry. See the `mlockall (MCL_CURRENT)` call below.

If a task sets up a request for an interrupt, obviously the kernel will track that and deliver the interrupt as scheduled; for example, see the `alarm()` and `timer_create()` kernel APIs. Due to hardware timer limitations, this may require multiple brief interrupts where the kernel resets the hardware timer to a bit further out each time; the Tiler timers only count down over a period of a few seconds, so multiple interrupts might be required to get a single specified time requested of the kernel.

User-space TLB flushes interrupt all threads that share a given address space. These can be caused by the `munmap()` API, which in turn can be caused by a call to `free()`. It can also be caused by threads migrated to different cpus if kernel home-caching is enabled and the threads are configured to home the cache for their stacks on the current cpu, as is true by default; in this case the kernel has to do a TLB flush to all the other threads so they know to reload their TLBs if they have to access the migrating task's stack.

Other global kernel events will also cause interrupts, such as kernel module unload; a list is currently being maintained at <https://github.com/gby/linux/wiki> with sources of CPU interference in core Linux code.

7.5 Programming Models for Dataplane Mode

In general, the dataplane cores should run standalone processes, or else threads of a process that does not run threads on non-dataplane cores. Otherwise, the threads running on non-dataplane cores can cause TLB flushes to the dataplane threads somewhat unpredictably. While it's possible to make this work, it's a difficult discipline and not recommended.

To communicate between dataplane and non-dataplane processes, the best mechanism is typically some form of shared memory: for example, shared anonymous memory if one process is forked from the other, or else a shared memory-mapped file.

Using locks of any kind directly between dataplane and non-dataplane processes is not recommended. Pthread locks use kernel support (futexes) which will cause dataplane tasks to enter the kernel, which is generally not appropriate. However, memory-based spin locks can cause problems when run on non-dataplane cores, since if the lock is taken and the lock holder is scheduled out by the kernel for any reason, there can be very long latencies imposed on any other task (dataplane or non-dataplane) trying to acquire the lock.

A good model is a shared memory circular queue, such as the `<tmc/queue.h>` API. In this case, locking is done separately at each end of the queue, so dataplane processes only contend with each other.

7.6 Using the `set_dataplane()` API

The `<sys/dataplane.h>` header provides the `set_dataplane()` API. Tiler Linux has a `set_dataplane(2)` man page.

The `DP_QUIESCE` flag quiesces the timer interrupt before returning to user space after a system call. Normally if a task on a dataplane core makes a system call, the system will run one or more timer ticks after the system call has completed, causing unexpected interrupts in userspace. Setting `DP_QUIESCE` avoids that problem by having the kernel hold the task in kernel mode until the timer ticks are complete. This will make syscalls dramatically slower. If multiple dataplane tasks are scheduled on a single core, this in effect silently disables `DP_QUIESCE`, which allows the tasks to make progress, but without actually disabling the timer tick.

`DP_STRICT` disallows the application from entering the kernel in any way, unless it calls `set_dataplane()` again without this bit set. Issuing any other system call or causing a page fault would generate a kernel message, and `kill -9` the process. Setting this flag automatically sets `DP_QUIESCE` as well, to hold the process in kernel space until any timer ticks due to the `set_dataplane()` call have completed. This is essentially a development debugging aid.

`DP_DEBUG` provides support to debug dataplane interrupts, so that if any interrupt source attempts to involve a dataplane CPU, a kernel message and stack back-trace will be generated on the console. As this warning is a slow event, it may make sense to avoid this mode in production code to avoid making any possible interrupts even more heavyweight. Setting this flag automatically sets `DP_QUIESCE` as well. This is also intended as a development debugging aid, though in this case its primary use is to uncover kernel or driver bugs.

Note: `DP_DEBUG` generates a message if a task does something that makes a dataplane task be non-dataplane, even if the offender itself is a non-dataplane thread.

Use the `mlockall(MCL_CURRENT)` call to populate all memory mappings in the page table. You should call `mlockall(MCL_CURRENT)` before calling `set_dataplane()`, to ensure that no future page fault events will occur to cause interrupts into the Linux kernel, as long as no new mappings are installed by `mmap()` or other similar routines.

Since the hardware TLB is of finite size, there is still the potential for TLB misses that the hypervisor handles, either via its software TLB cache (fast path) or by walking the kernel page tables (slow path), so touching large amounts of memory will still incur hypervisor interrupt overhead. This is essentially the same as `mlockall(MCL_CURRENT)`, but without the pages being locked into memory.

Note: The `mlockall()` routine interrupts every core when you run it, so you must finish running it on every core before any core enters dataplane mode.

7.7 Spin-Based Shared Memory Synchronization

The TMC library provides spinning shared memory synchronization primitives that should only be used by dataplane tiles. These include a spinlock (`tmc_spin_mutex_t`), a fairer *queued* lock (`tmc_spin_queued_mutex_t`), and a spinning barrier (`tmc_spin_barrier_t`).

These implementations have lower overhead than those provided by the Pthreads library, but when a resource is unavailable, they spin in userspace rather than yielding to the kernel scheduler. As a result, they are inappropriate for environments where more than one task is running on each core, because a task might spin in userspace trying to acquire a lock that is held by a task that is currently swapped-out on the same core.

The spinning primitives work properly on both dataplane and standard tiles. However, sharing a lock between a task on a dataplane tile and a task on a standard tile can introduce non-determinism. This is because the standard tile might take a timer tick while holding the lock, forcing the dataplane task to wait 30,000+ cycles for the lock. For more information about these synchronization primitives, see the TMC chapter of the *Application Libraries Reference Manual* (UG527).

7.8 Avoiding System Calls

Dataplane tiles provide a complete Linux system interface and will properly execute any system calls an application makes. However, dataplane tiles only maintain zero Linux overhead if the task running on each tile avoids making any Linux system calls. Any Linux system call, particularly for memory allocation or I/O services, might result in system overhead on the calling tile (or other tiles) at some point in the future. For example, a call to invoke an I/O device driver might lead to jitter at some point in the future when that I/O driver needs to handle hardware interrupts. Thus, dataplane applications generally use standard Linux APIs during startup but then require a non-Linux I/O mechanism while performing real-time work.

The MDE software environment provides a direct-to-hypervisor I/O API for communicating with the high-speed, on-chip I/O devices. Because this API goes directly to the hypervisor layer without running any Linux code, it is ideally suited for use on dataplane tiles.

The “packet capture” example described in [Section 7.11 “Dataplane Example” on page 78](#) demonstrates how to use both the XAUI and PCIe interfaces in a dataplane application.

7.9 User Dynamic Network (UDN)

The User Dynamic Network (UDN) allows applications to send small data packets directly between tiles by means of a dedicated, on-chip hardware network. Applications generally use the UDN for one or two performance-critical communication operations. The UDN requires careful attention to potential network deadlocks, so we generally recommend using shared memory for bulk data transfer and reserving the UDN as an optimization tool for the most critical communication operations.

The UDN is accessible to both standard and dataplane tiles, but it is particularly well suited to dataplane use. The UDN provides 118 words of storage at each receiver, so taking a timer interrupt (or being context-swapped out) can cause the entire hardware network to block up because one task was unable to drain an incoming packet.

The remainder of this section assumes that you have read the UDN hardware description in [Section 1.5 “Inter-Tile Networks” on page 3](#).

7.9.1 Basic UDN Usage

When running under Tile Linux, access to the UDN is guarded by the operating system. The TMC library provides a two-phase process that allows applications to access the UDN.

First, the `tmc_udn_init()` function reserves a rectangle of tiles (referred to as a *UDN hardwall*) for use by the calling application. This function should be called by the first task in the application, before creating any child tasks. It takes a `cpu_set_t*` parameter specifying all the CPUs that will access the UDN and allocates the minimum rectangle of tiles that contains those CPUs. If the `cpu_set_t*` is `NULL`, the routine allocates a rectangle that contains all the CPUs in the calling task's affinity set (see the `tmc_cpus_get_my_affinity()` routine).

For example, the parent process should do something like the following to create a UDN hardwall:

```
if (tmc_udn_init(NULL) != 0)
    tmc_task_die("Failure in 'tmc_udn_init()'");
```

After creating the UDN hardwall, the application can create threads or fork processes as necessary and bind them to particular CPUs using `tmc_cpus_set_my_cpu()`. Once a task is bound to a single CPU, it should call `tmc_udn_activate()` to enable that CPU to send and receive packets within the previously created UDN hardwall. For example:

```
if (tmc_cpus_set_my_cpu(my_cpu) < 0)
    tmc_task_die("Failure in 'tmc_cpus_set_my_cpu()'");
if (tmc_udn_activate() < 0)
    tmc_task_die("Failure in 'tmc_udn_activate()'");
```

Once the UDN has been activated, the application can use `tmc_udn_header_from_cpu()` to create packet headers and send packets by means of the `tmc_udn_send_<N>()` functions. For example, the following code sends a 5-word data packet to demux queue #2 at `dest_cpu`:

```
DynamicHeader header = tmc_udn_header_from_cpu(dest_cpu);
tmc_udn_send_5(header, UDN2_DEMUX_TAG, data0, data1, data2, data3, data4);
```

When this packet arrives at the destination core, the UDN hardware strips off the header and tag words and places the five data words into demux queue 2. To receive those five data words, do the following:

```
data0 = tmc_udn2_receive();
data1 = tmc_udn2_receive();
data2 = tmc_udn2_receive();
data3 = tmc_udn2_receive();
data4 = tmc_udn2_receive();
```

For more detailed documentation of the UDN interface, see the TMC documentation in the *Application Libraries Reference Manual* (UG527). An example program using the UDN is available in `$TILERA_ROOT/examples/tmc/multi_tile/`.

7.9.2 Avoiding UDN Deadlock

The UDN has a limited amount of internal buffering and will never drop packets. As a result, applications can run into deadlock situations if they fill all the available buffering and thus block trying to send or receive a packet.

For example, consider an application in which two tiles try to send a stream of several 128-word UDN packets to each other at the same time. Because the hardware only has storage for ~118 words, the demux buffer in each tile will overflow and back up into the network. This back pressure will clog up the UDN links all the way back to the sending tile, eventually forcing it to stop injecting messages. Then, since both tiles were sending to the other simultaneously, and both are blocked by the demux buffer overflow, neither will be able to finish the packet sends so that they can run receive their incoming packets. Thus, the application will deadlock.

In general, avoiding UDN deadlock requires that the application writer be able to guarantee that each tile will always be able to drain incoming packets without needing to complete any UDN send operation. One way to do this is to have each tile only send or only receive. Another is to guarantee that the maximum amount of packet data that can be injected will never overflow the demux buffer. That way, no UDN send will ever stall due to back pressure, and all cores can guarantee that they will finish any UDN send operations and get back to receiving packets.

Designing UDN protocols that meet these requirements can be challenging. Thus, we recommend that developers use shared memory for most communication and only use the UDN for optimizing one or two performance-critical operations.

Several common patterns can be implemented on the UDN in a deadlock-free fashion. If the UDN-level performance is needed for particular communication flows, we recommend that developers stick to one (and only one) of the following patterns:

- **Nearest-Neighbor Pipelines** — In this pattern, tiles are arranged into a UDN pipeline in which each tile sends to exactly one other tile. If the tasks are arranged so that each task sends to one of its neighbors, each network link can only be used by one of the core-to-core packet routes. When arranged like this, traffic can overflow a receive demux and block a network link, but the system will not deadlock, because blocking that link cannot inhibit progress on any of the tiles later in the pipeline (that is, the tiles that need to make progress in order to drain the packet that is overflowing the demux).
- **UDN Load Balancer** — In this pattern, a single tile is responsible for sending *work descriptor packets* to a (potentially large) number of worker tiles. Each worker waits for a work packet to arrive, performs the work, and sends an ack back to the load balancer tile. If the programmer guarantees that the maximum number of work packets sent to a worker tile will not overflow the worker's demux buffer, the load balancer tile is guaranteed to make forward progress and the system will not deadlock.

A system with multiple load balancer tiles can be created by also guaranteeing that the acks coming back to each server tile can never overflow its demux buffer.

- **UDN Server Tile** — In this pattern, a single *server* tile receives requests from a (potentially large) number of *client* tiles. Each client sends a request to the server and then waits for the response. If there is only one server tile, deadlock is avoided so long as the application guarantees that the response packets will never overflow the demux buffer at the client tiles. To describe this in another way, each client should never send more requests than it has buffering to store responses.

A system with multiple server tiles can be created by also guaranteeing that the requests going to the server tile will never overflow its demux buffer.

7.10 Troubleshooting ZOL Applications

If your application is occasionally suspended by the operating system even when it is running on dataplane CPUs, there are several steps you can take to troubleshoot the problem:

1. Ensure that you have correctly configured the set of dataplane CPUs using the dataplane boot option.

You can cat the contents of `/sys/devices/system/cpu/dataplane` to confirm the set of CPUs. You can check which CPU your process is running on by using the `ps` command, for example using `ps -o cpu,psr,comm` to see the process ID and the assigned CPU for each process. (You can use `sgi_p` instead of `psr` to see if the process is actively running; if it is suspended in the kernel it will show an `*` (asterisk) symbol instead.)

2. Ensure that your process is not making any system calls, because each entry and exit to the kernel will perturb the dataplane state of the tile.

The `/proc/tile/interrupts` file contains per-CPU information about the number of syscalls made on each processor. This value should not be increasing as your program runs.

3. Ensure that your program has touched every page of memory it will need prior to starting to run in dataplane mode.

Touching a page of memory for the first time causes a kernel trap to enter that page into the process's page table. You can examine the process's page table periodically by examining `/proc/PID/pgtable` for your process's PID. It should not change while the program is running in dataplane mode.

4. Ensure that you are not causing periodic *un-sequestering* of the set of sequestered pages on the system.

Linux *sequesters* pages that are incoherent when they are freed back to the system, because making them coherent again involves a cache flush on every CPU. When the system runs low on memory, it will release all sequestered pages back to the general-purpose memory pool and perform a cache flush on every CPU, which will interrupt dataplane tiles.

Sequestering can occur with executable files that are run and then deleted; the FUSE interface does this frequently, so care should be taken when using FUSE and dataplane mode together. The `/proc/tile/interrupts` will show increments in the `hvflush` column when the hypervisor performs a remote cache flush (or TLB flush) on a CPU; you will also see the `/proc/sys/tile/homecache/sequestered_purges` count increasing when *un-sequestering* is performed.

7.11 Dataplane Example

The MDE distribution includes a Zero Overhead Linux example program called `spinner`, located at `$TILERA_ROOT/examples/dataplane/spinner/`.

This application demonstrates how an application can bind its tasks to dataplane CPUs. It shows how ZOL will minimize system overhead by disabling kernel timer ticks so long as there is one task per dataplane CPU and the dataplane tasks do not make any system calls.

INDEX

Symbols

/dev/hugepages/ 48
/etc/rc.local 48
/proc/cpuinfo 40
/proc/meminfo 44, 47
\$HUGETLB_MORECORE variable 49

A

accessing common memory 38
alarm() 73
alignment of data
 managing 60
all 42
allbutstack 42
allbutstack mode 43
allocating memory
 functions for 34
allocating memory pages 33
allocating memory spaces 35
allocating separate memory spaces 35
allocating too much memory 51
allocation control
 page attributes for 68
ALLOC_HOME_INCOHERENT 36
alloc_map() 42, 61
alloc_set_home() 61
alloc_set_nodes() 61
application
 use of memory 33
application layer 4
atomic.h 70
__attribute__((section)) directive 60
audience of this guide vii

B

bare metal 6
Bare Metal Environment
 See BME
BME 6
BME tile 20
boot argument
 default_hugepagesz 44, 45, 47
 hugepages 46, 47
 hugepagesz 44, 45, 46

C

cache blocks
 page attribute for distribution of 68
cache defaults
 per application 67
 system-wide 67
cache subsystem 25, 63
cache-coherent programming model 15
cache-homing mechanism 16
caching control 39
calloc() 37
checksums 3
chmod command 48
clearing shared memory
 functions for 34
client tiles 77
CONFIG_PAGE_SIZE_16KB 44
confirming allocation 60
context switches 70
controlling memory allocation 33
conventions
 notation viii
CPU affinity set 19
cpu_set_t structure 19
cpu_set_t* parameter 76
creating huge pages 36
customer support viii

D

data alignment
 managing 60
data decomposition 7
dataplane 6
dataplane boot argument 69, 72
dataplane Linux tile 20
dataplane tile 19
dataplane tiles 17, 23, 70, 72, 73
DDC 25, 63
 overview 63
 page attributes 67
 See Dynamic Distributed Cache 63
DDR3 memory 20
default_hugepagesz 44, 45, 47
demux fifo 4
demux queue 76
disabling

- local cache [40](#)
- dividing tasks among binary programs [12](#)
- DMA [63](#)
- DMA subsystem [2](#)
- document organization [vii](#)
- DP_DEBUG [74](#)
- DP_QUIESCE [74](#)
- DP_QUIESCE flag [73](#)
- DP_STRICT [74](#)
- Dynamic Distributed Cache [63](#)
- dynamic memory allocation
 - examples [61](#)

E

- encryption [3](#)
- examples
 - of dynamic memory allocation [61](#)
- exec() [12](#), [41](#)
- execve() [22](#), [31](#)

F

- feedback-based optimization [13](#)
- FIFO data structures [16](#)
- flags
 - for disabling local cache [40](#)
 - for requesting user-managed shared memory [41](#)
 - for setting home cache [39](#)
 - for sharing mappings [41](#)
 - to control caching [39](#)
- flags for disabling [40](#)
- flags for setting home cache
 - MAP_CACHE_HOME(n) [39](#)
 - MAP_CACHE_HOME_HASH [40](#)
 - MAP_CACHE_HOME_HERE [40](#)
 - MAP_CACHE_HOME_SINGLE [40](#), [41](#)
 - MAP_CACHE_HOME_TASK [40](#)
- flow hash [10](#)
- fork() [7](#), [12](#), [13](#), [18](#), [22](#), [41](#), [51](#)
- free() [30](#), [37](#), [39](#), [73](#)
- free_hugepage_region() [49](#)
- free_huge_pages() [49](#)
- freeing memory
 - functions for [34](#)
- ftruncate() [41](#)
- functional decomposition [7](#)

G

- get_hugepage_region() [49](#)
- get_huge_pages() [49](#)
- gigabit ethernet [20](#)
- gxio_mpipe_register_page() [49](#)

H

- hardware networks [14](#)
- hash-for-home [36](#), [40](#), [42](#), [67](#)

- in applications [42](#)
 - specifying tiles for [67](#)
- hashing [3](#)
- hfh_tiles [67](#)
- hfh_tiles subcommand [42](#)
- home cache [39](#)
- home tile [26](#)
- huge pages
 - creating [36](#)
- hugeadm command [48](#)
- hugectl command [49](#), [50](#)
- hugectl --heap [49](#)
- hugectl --shm [49](#)
- hugeedit command [50](#)
- hugepages [46](#), [47](#)
- hugepagesz [44](#), [45](#), [46](#), [47](#), [49](#)
- hugetlbfs [48](#), [49](#)
- hugetlbfs.h [49](#)
- hvx flag [72](#)
- hypervisor [4](#)
- hypervisor dedicated tile [20](#)
- hypervisor layer [4](#)

I

- IDE online help [viii](#)
- IDN [4](#)
- IDN Internal Dynamic Network
 - See IDN
- info pages [viii](#)
- init process [20](#)

J

- jitter [69](#)

K

- kcachefilter boot option [43](#)
- kcachefilter environment variable [67](#)

L

- L1 cache [40](#)
- L2 cache [26](#), [27](#), [28](#), [40](#), [64](#), [66](#)
- L3 cache [26](#), [42](#), [64](#), [66](#), [68](#)
- LD_CACHE_HASH environment variable [39](#), [42](#), [43](#), [67](#)
- LD_CACHE_HASH options
 - all [42](#)
 - allbutstack [42](#)
 - none [42](#)
 - ro [42](#)
 - static [42](#)
- level 1 data cache [3](#)
- libc library [35](#), [37](#)
- libhugetlbfs [49](#), [50](#)
- linker script
 - for controlling data placement [60](#)
- linux commands

- chmod [48, 50](#)
- hugeadm [48](#)
- hugectl [49, 50](#)
- Linux scheduler [70](#)
- load balancing tasks [19](#)
- local cache [40](#)
- M**
- macroblocks [8](#)
- mallinfo() [37](#)
- malloc() [20, 30, 37, 42](#)
- malloc_stats() [37](#)
- MALLOC_USE_HUGE_PAGES [50](#)
- man pages [viii](#)
- managing
 - alignment of data [60](#)
- MAP_ANONYMOUS [41](#)
- MAP_ANONYMOUS flag [48, 51](#)
- MAP_CACHE_HOME(n) [39](#)
- MAP_CACHE_HOME_HASH [40, 43](#)
- MAP_CACHE_HOME_HERE [40](#)
- MAP_CACHE_HOME_SINGLE [40, 41](#)
- MAP_CACHE_HOME_TASK [40](#)
- MAP_CACHE_INCOHERENT [41](#)
- MAP_CACHE_NO_L1 [40](#)
- MAP_CACHE_NO_L2 [40](#)
- MAP_CACHE_NO_LOCAL [40](#)
- MAP_CACHE_NONE [40](#)
- MAP_HUGETLB [44](#)
- MAP_HUGETLB flag [48](#)
- MAP_POPULATE flag [51](#)
- MAP_SHARED [30, 31, 41](#)
- mbind() [33](#)
- MDE Document Index [viii](#)
 - location of [viii](#)
- MDE documentation
 - location of [viii](#)
- memalign() [37, 60](#)
- memory
 - allocating too much [51](#)
- memory allocation
 - controlling [33](#)
 - dynamic
 - examples [61](#)
- memory homing [25](#)
- memory pages
 - allocating [33](#)
- memory spaces
 - allocating [35](#)
 - allocating separate [35](#)
- mlockall() [74](#)
- mlockall(MCL_CURRENT) [74](#)
- mmap() [5, 30, 31, 33, 40, 42, 48, 51, 74](#)
 - MAP_ANONYMOUS [41, 48, 51](#)
 - MAP_HUGETLB flag [44, 48](#)
 - MAP_POPULATE [51](#)
 - MAP_SHARED [41](#)
- mmap_flags field
 - MAP_ANONYMOUS [48, 51](#)
 - MAP_HUGETLB [48](#)
 - MAP_POPULATE [51](#)
- MPMD [12](#)
- multiple heap allocation
 - functions for [37](#)
- Multiple Program Multiple Data
 - See MPMD
- munmap() [73](#)
- N**
- nearest-neighbor pipelines [77](#)
- network packets [8](#)
- nice command [71](#)
- non-dataplane tiles [20](#)
- none [42](#)
- notation conventions [viii](#)
- nr_hugepages [47](#)
- O**
- open() [41](#)
- organization of this guide [vii](#)
- P**
- page attribute
 - for distribution of cache blocks [68](#)
- page attributes
 - for allocation control [68](#)
 - in DDC [67](#)
- page table special files [60](#)
- PAGESIZE [49, 50](#)
- parallelization
 - process [7](#)
 - strategy [18](#)
- PC [2](#)
- PCIe [20](#)
- pgtable [60](#)
- physical addresses [43](#)
- pipeline decomposition
 - illustrated [8](#)
- point-to-point
 - communication [15](#)
 - hardware messages [14](#)
- POSIX Threads (pthreads) [22](#)
- program counter
 - See PC
- pthread_barrier_t structure [22, 23](#)
- pthread_cond_signal() [23](#)
- pthread_cond_t structure [22](#)
- pthread_cond_wait() [23](#)
- pthread_create() [7, 13, 18](#)
- pthread_mutex_lock() [22](#)

pthread_mutex_t structure [22](#)
 pthread_mutex_unlock() [22](#)
 Pthreads [31](#)
 Pthreads library [22](#)
 Pthreads synchronization [22](#)

Q

queued lock [75](#)

R

read() [71](#)
 ready run queue [71](#)
 ready tasks [70](#)
 realloc() [37](#)
 renice command [71](#)
 ro [42](#)
 run-time software stack [4](#)

S

SAD [3](#)
 sched_setaffinity() [22](#)
 scheduler-induced timer interrupts [71](#)
 scheduling processes and threads [13](#)
 server tile [77](#)
 set_dataplane() [74](#)
 DP_DEBUG flag [74](#)
 DP_QUIESCE flag [73, 74](#)
 DP_STRICT flag [74](#)
 setitimer [71](#)
 setting
 home cache [39](#)
 setting allocation attributes [36](#)
 setting dataplane mode [74](#)
 setting memory nodes
 functions for [34](#)
 setting shared memory
 functions for [34](#)
 shared memory [14](#)
 communication [14](#)
 user-managed [41](#)
 shared memory regions [30](#)
 shepherd [21](#)
 shmget() [48, 49](#)
 SHM_HUGETLB flag [44, 48](#)
 SIGBUS [51](#)
 SIGKILL [51](#)
 SIMD instructions [3](#)
 SIMD operations [52](#)
 Single Program Multiple Data
 See SPMD
 slab reaper [72](#)
 sleeping task [71](#)
 spinning barrier [75](#)
 SPMD [12](#)
 standard Linux tile [20](#)

static [42](#)
 sub-word parallelism [3](#)
 sum of absolute differences
 See SAD
 supervisor layer [4](#)
 support
 technical or customer [viii](#)
 synchronization primitives [41](#)
 sys/dataplane.h [74](#)

T

taskset [21](#)
 taskset command [20, 21, 22](#)
 technical support [viii](#)
 test-and-set instruction [41](#)
 threads [7](#)
 tile classes [6](#)
 --tile option [20](#)
 Tile Processor
 architecture [1](#)
 tile-gcc [50](#)
 tile-hugeedit [50](#)
 tile-monitor [20, 21](#)
 timer tick interrupt [71](#)
 timer_create() [73](#)
 TLB [3](#)
 TMC [23](#)
 library [75](#)
 tmc/alloc.h [29, 35, 48, 61](#)
 tmc/cmем.h [39](#)
 tmc/mospace.h [35, 38, 61](#)
 tmc/queue.h [74](#)
 tmc/task.h [21](#)
 tmc/udn.h [20](#)
 tmc_alloc() [33, 35](#)
 tmc_alloc() API [33](#)
 tmc_alloc_clear_huge() [48](#)
 tmc_alloc_clear_shared() [34](#)
 tmc_alloc_get_available_nodes() [34](#)
 tmc_alloc_get_huge() [48, 51](#)
 tmc_alloc_get_huge_pagesize() [48](#)
 tmc_alloc_get_pagesize() [48](#)
 tmc_alloc_get_pagesizes() [48](#)
 TMC_ALLOC_HOME_TASK [39](#)
 TMC_ALLOC_INIT [29, 37](#)
 tmc_alloc_map() [30, 31, 34, 35, 38, 51](#)
 tmc_alloc_map_at() [34](#)
 tmc_alloc_remap() [34](#)
 tmc_alloc_set_home() [29, 38, 39](#)
 tmc_alloc_set_huge() [29, 37, 48](#)
 tmc_alloc_set_node_default() [35](#)
 tmc_alloc_set_node_local() [35](#)
 tmc_alloc_set_node_preferred() [35](#)
 tmc_alloc_set_nodes() [34](#)
 tmc_alloc_set_nodes_interleaved() [35](#)

tmc_alloc_set_pagesize() 48, 49
 tmc_alloc_set_pagesize_exact() 48
 tmc_alloc_set_shared() 31, 34
 tmc_alloc_t structure 29, 30, 31, 33, 34, 35, 36, 37, 38, 48, 51
 tmc_alloc_unmap() 34
 tmc_cmem() 30, 31, 38, 41
 tmc_cmem() API 38
 TMC_CMEM_ALLOC_INIT 38
 tmc_cmem_calloc() 39
 tmc_cmem_init() 38
 tmc_cmem_malloc() 38, 39
 tmc_cmem_memalign() 39
 tmc_cmem_realloc() 39
 tmc_cpus_count() 19
 tmc_cpus_find_nth_cpu() 19
 tmc_cpus_get_my_affinity() 19
 tmc_cpus_set_my_affinity() 22
 tmc_cpus_set_my_cpu() 19, 76
 tmc_mspace or tmc_alloc_map() 51
 tmc_mspace() 35, 36, 37
 flags specific to 36
 tmc_mspace() API 35
 tmc_mspace_allocator() 51
 tmc_mspace_calloc() 36, 37, 39
 tmc_mspace_create() 35, 36, 37, 61
 tmc_mspace_create_special() 30, 35, 36, 37, 38
 tmc_mspace_create_with_base() 35, 36, 37
 tmc_mspace_create_with_base_special() 35, 36, 37
 tmc_mspace_destroy() 35, 37
 tmc_mspace_free() 37, 39, 41
 TMC_MSPACE_LOCKED 36, 39
 tmc_mspace_mallinfo() 36, 37
 tmc_mspace_malloc() 36, 37, 39, 41, 61
 tmc_mspace_malloc_statistics() 36, 37
 tmc_mspace_memalign() 36, 37, 39
 TMC_MSPACE_NOGROW 36
 tmc_mspace_realloc() 37, 39, 41
 tmc_mspace_statistics() 36
 tmc_spin_barrier_t structure 23, 75
 tmc_spin_mutex_t structure 23, 75
 tmc_spin_queued_mutex_t structure 23, 75
 tmc_sync_barrier_t structure 23

tmc_sync_mutex_t structure 23
 tmc_udn_activate() 76
 tmc_udn_header_from_cpu() 76
 tmc_udn_init() 76
 tmpfs 41
 TNS 41
 translation lookaside buffer
 See TLB

U

ucache_hash boot option 39, 43, 67
 ucache_hash environment variable 67
 UDN 4, 75
 load balancer 77
 server tile 77
 unaligned memory accesses 52
 User Dynamic Network
 See UDN
 user-managed shared memory 41

V

vec_ops.h 52, 54, 56
 vec_UnalignedArray macro 53
 vec_unaligned_load macro 52, 54, 56
 vec_unaligned_load_N macro 53
 vec_UnalignedPointer structure 53
 Very Long Instruction Word
 See VLIW
 virtual addresses 43
 VLIW 2

W

watchdog timer 72
 work descriptor packets 77
 work queue 7

X

XAUI 20

Z

ZOL 17, 69
 Linux applications 69

